

Generative AI and Its Applications



Unit 4

(UE23CS342BA4)

Multimodal Models: CLIP, BLIP & Whisper

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

GenerativeAI and Its Applications

Acknowledgement



The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

What is Multimodal AI?

What is Multimodal AI?

Definition:

Multimodal AI refers to artificial intelligence systems that can **process, understand, and combine information from multiple data types (modalities)**.

such as:

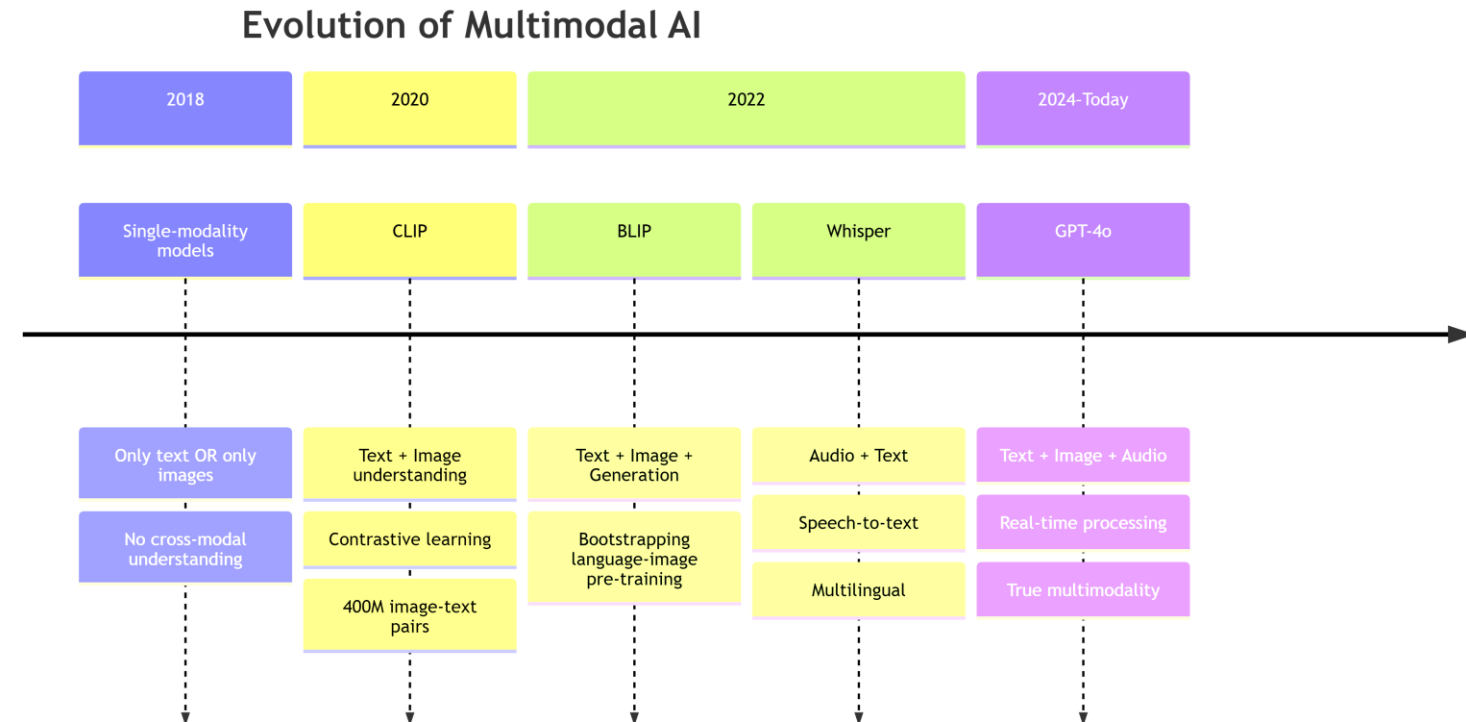
- Text
- Images
- Audio
- Video

within a **single unified model**.

What is Multimodal AI?

The Evolution of Multimodal AI

- 2018** → Single-modality models (only text *or* only images)
- 2020** → CLIP (Text + Image understanding)
- 2022** → BLIP (Text + Image + Generation)
- 2022** → Whisper (Audio + Text)
- 2024–Today** → GPT-4o (Text + Image + Audio in real time)



Why Multimodal AI Matters?

1. Human-like Understanding

Humans naturally combine senses—vision, hearing, language. Multimodal AI mimics this ability.

2. Richer Representations

Different modalities provide **complementary information**:

- Image → visual context
- Text → semantic meaning
- Audio → tone and intent

3. Real-World Relevance

Most real-world data is multimodal:

- Documents → text + images
- Videos → visuals + audio
- Conversations → speech + language

Analogy

- Unimodal AI** → Like a doctor who only reads reports
- Multimodal AI** → Like a complete physician who:
 - Listens to symptoms
 - Examines the patient
 - Reviews lab results

The Multimodal Landscape (Key Models)?

1. CLIP

- Modality: Image + Text
- Technique: Contrastive learning

Use cases:

- Zero-shot classification
- Image-text retrieval

2. BLIP

- Modality: Image + Text (+ generation)
- Technique: Bootstrapped captioning

Use cases:

- Image captioning
- Visual Question Answering (VQA)

3. Whisper

- Modality: Audio + Text
- Architecture: Encoder–Decoder Transformer

Use cases:

- Speech recognition
- Translation
- Language identification

The Pre-training Paradigm?

Multimodal models learn in two main stages:

1. Pre-training

Train on massive datasets:

- CLIP → ~400M image-text pairs
- Whisper → ~680K hours of audio

Learn a **shared embedding space** where:

- Similar concepts across modalities are close together

2. Fine-tuning (Optional)

- Adapt the model to specific tasks
- Improve performance for particular domains

What is CLIP??

Definition

CLIP (Contrastive Language–Image Pre-training) is a neural network developed by OpenAI that learns visual concepts from natural language by training on **400M image–text pairs**.

Core Idea

Instead of fixed labels:

- “A cat sitting on a windowsill”
- “A house in the countryside”
- “A red car on the highway”

Learns concepts directly from captions

Analogy

- Traditional CV: Learning from flashcards
- CLIP: Learning from picture books with captions

Why CLIP???

Problems with Traditional Models

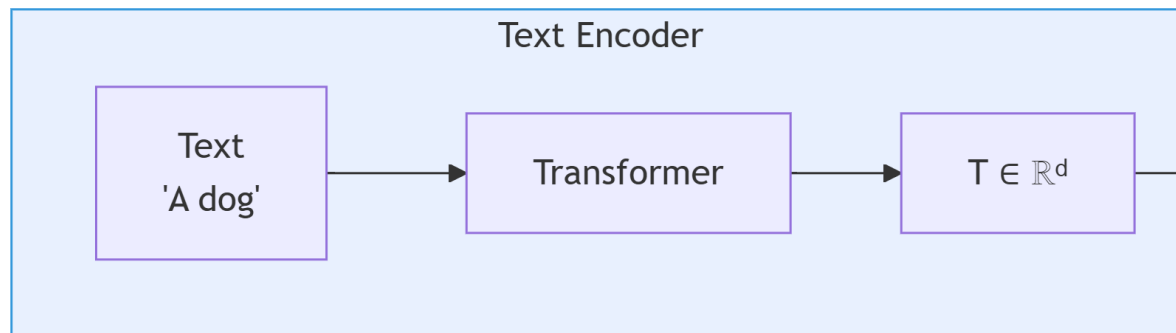
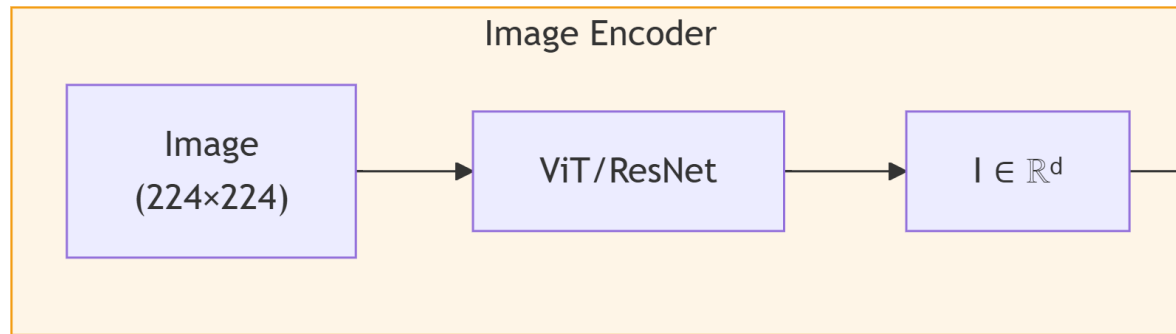
- Manual labeling required
- Fixed categories
- Poor generalization
- No use of language knowledge

CLIP Solution

- Train on internet-scale image text pairs
- Learns open vocabulary concepts
- Enables **zero-shot learning**

CLIP Architecture

An **image encoder** (such as ViT or ResNet) takes an input image (e.g., 224×224 pixels) and converts it into a numerical representation called an **embedding vector** ($I \in \mathbb{R}^d$).



A **text encoder** (a Transformer) processes a text input like “A dog” and converts it into another embedding vector ($T \in \mathbb{R}^d$)

Dual Encoder Design

- **Image Encoder:** ViT / ResNet

- **Text Encoder:** Transformer

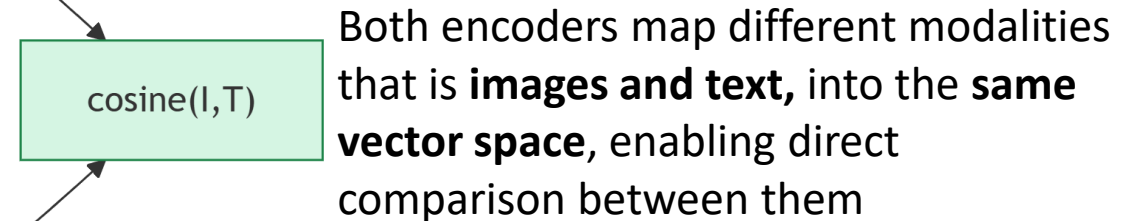
Both produce embeddings in same space:

- Image $\rightarrow$ vector ($\mathbb{R}^d$)

- Text $\rightarrow$ vector ($\mathbb{R}^d$)

Matching Mechanism

- Cosine similarity between image & text embeddings



During training, CLIP learns to **maximize similarity for correct image–text pairs** and minimize it for incorrect pairs.

CLIP - CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

This explains, How CLIP scales across different model sizes to balance performance, accuracy, and computational cost.

Model	Image Size	Vision Params (M)	Text Params (M)	Output Dim	Use Case
B/32	224	87.85	63.43	512	Lightweight, fast
B/16	224	86.19	91.16	512	Balanced
L/14	224	303.97	123.65	768	High accuracy
H/14	224	638.08	354.03	1024	Research
g/14	224	1012.65	354.03	1024	Maximum capacity
G/14	224	1840	590	1280	State-of-the-art

A higher value means the model can capture more complex visual patterns (textures, shapes, objects), but it also requires more memory and computation.



Naming Convention:

B = Base, L = Large, H = Huge, G = Giant

/32, /16, /14 = Patch size (smaller patches = more tokens = higher resolution)

Rule of Thumb: Start with B/16 for most applications, use L/14 when accuracy is critical

These parameters help the model understand language, semantics, context, and relationships between words. Larger text parameter counts allow better understanding of complex sentences,

GenerativeAI and Its Applications

CLIP - CONTRASTIVE LANGUAGE-IMAGE PRE-TRAIN

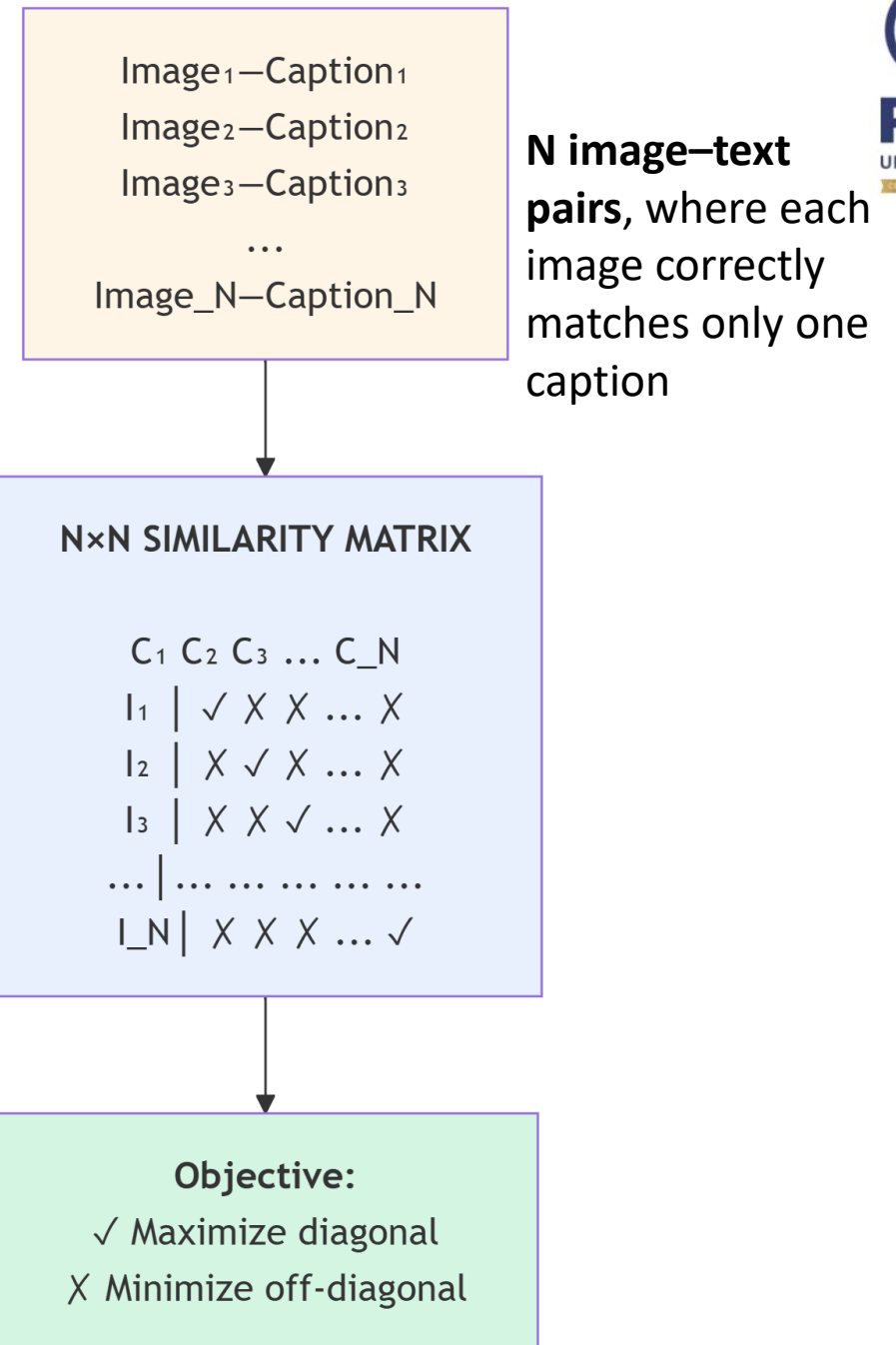
How CLIP Learns - Contrastive Learning

The Training Objective:

During pre-training, CLIP is trained on a batch of N image-text pairs

GOAL: Maximize similarity on diagonal (positive pairs), minimize on off-diagonal (negative pairs).

Both the images and captions are passed through their respective encoders to produce embeddings, and then CLIP computes a **similarity score (cosine similarity)** between every image and every caption. This forms an $N \times N$ **similarity matrix**, where rows represent images and columns represent captions.



Contrastive Loss - Numerical Example

A toy numerical example of how CLIP computes similarity and learns using contrastive loss.

Batch size $N = 3$

Image embeddings (normalized):

$I_1 = [0.9, 0.4]$ (cat photo)

$I_2 = [0.2, 0.9]$ (dog photo)

$I_3 = [0.7, 0.6]$ (car photo)

Text embeddings (normalized):

$T_1 = [0.8, 0.5]$ ("a cat")

$T_2 = [0.3, 0.8]$ ("a dog")

$T_3 = [0.6, 0.7]$ ("a car")

$I_1 T_1$

Contrastive Loss - Numerical Example

Step 1: Similarity Matrix

Since embeddings are normalized, cosine similarity = dot product.

$$S[i, j] = I_i \cdot T_j$$
$$S = \begin{bmatrix} 0.92 & 0.59 & 0.82 \\ 0.61 & 0.78 & 0.75 \\ 0.86 & 0.69 & 0.84 \end{bmatrix}$$

Diagonal = correct matches

Off-diagonal = negatives

Handwritten calculation for $P(T_1)$:

$$P(T_1) = \frac{e^{0.92/0.07}}{e^{0.92/0.07} + e^{0.59/0.07} + e^{0.82/0.07}}$$

Result: $P(T_1) \approx 0.80$

- If probability is **high (close to 1)** → loss is **small**

- If probability is **low** → loss becomes **large**

That's exactly why contrastive loss pushes correct pairs toward probability = 1.

Step 2: Softmax with Temperature ($\tau = 0.07$)

We compute row-wise softmax (image → text).

For Image I_1

$$\text{softmax}_j = \frac{e^{S_{1j}/\tau}}{\sum_k e^{S_{1k}/\tau}}$$

You computed:

- $e^{0.92/0.07} \approx 508,928$

- $e^{0.59/0.07} \approx 4,585$

- $e^{0.82/0.07} \approx 121,510$

Sum: 635,023

So probabilities:

- $P(T_1) = 0.80$

- $P(T_2) = 0.01$

- $P(T_3) = 0.19$

Handwritten calculation for $P(T_1)$:

$$P(T_1) = \frac{e^{0.92/0.07}}{e^{0.92/0.07} + e^{0.59/0.07} + e^{0.82/0.07}}$$

Result: $P(T_1) \approx 0.80$

CLIP - CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

Loss Calculation: I2&I3

Image I_2

Scores:

[0.61, 0.78 0.75]

Scaled:

$$\bullet 0.61/0.07 = 8.71 \Rightarrow e^{8.71} \approx 6,073$$

$$\bullet 0.78/0.07 = 11.14 \Rightarrow e^{11.14} \approx 69,000$$

$$\bullet 0.75/0.07 = 10.71 \Rightarrow e^{10.71} \approx 44,800$$

Sum:

119,873

Probability of correct match (T_2):

$$P = \frac{69,000}{119,873} \approx 0.576$$

Loss:

$$-\log(0.576) \approx 0.551$$

Final Loss:

$$\begin{aligned} \text{Loss} &= \frac{0.223 + 0.551 + 0.897}{3} \\ &= \frac{1.671}{3} \approx 0.557 \end{aligned}$$

Image I_3

Scores:

[0.86, 0.69 0.84]

Scaled:

$$\bullet e^{0.86/0.07} \approx e^{12.29} \approx 217,000$$

$$\bullet e^{0.69/0.07} \approx e^{9.86} \approx 19,200$$

$$\bullet e^{0.84/0.07} \approx e^{12.00} \approx 162,754$$

Sum:

398,954

Probability of correct match (T_3):

$$P = \frac{162,754}{398,954} \approx 0.408$$

Loss:

$$-\log(0.408) \approx 0.897$$

CLIP Zero-Shot Classification

How CLIP performs zero-shot inference

1. Prepare Label Prompts

Convert class names into natural language:

Instead of:

cat, dog, car

Use:

"a photo of a cat"

"a photo of a dog"

"a photo of a car"

This matters because CLIP was trained on **image-text pairs**, not raw labels.

2. Encode Text

Each prompt is passed through CLIP's **text encoder**:

$$T_1, T_2, T_3 = \text{text embeddings}$$

Example:

"a photo of a cat" → T_1

"a photo of a dog" → T_2

"a photo of a car" → T_3

3. Encode Image

Input image is passed through the **image encoder**:

$$I = \text{image embedding}$$

CLIP Zero-Shot Classification

How CLIP performs zero-shot inference

4. Compute Similarities

Use cosine similarity (dot product since normalized):

$$S_j = I \cdot T_j$$

This gives:

Similarity with "cat" $\rightarrow 0.91$

Similarity with "dog" $\rightarrow 0.65$

Similarity with "car" $\rightarrow 0.40$

5. Apply Softmax (Optional but Common)

$$P_j = \frac{e^{S_j/\tau}}{\sum_k e^{S_k/\tau}}$$

This converts similarities into probabilities.

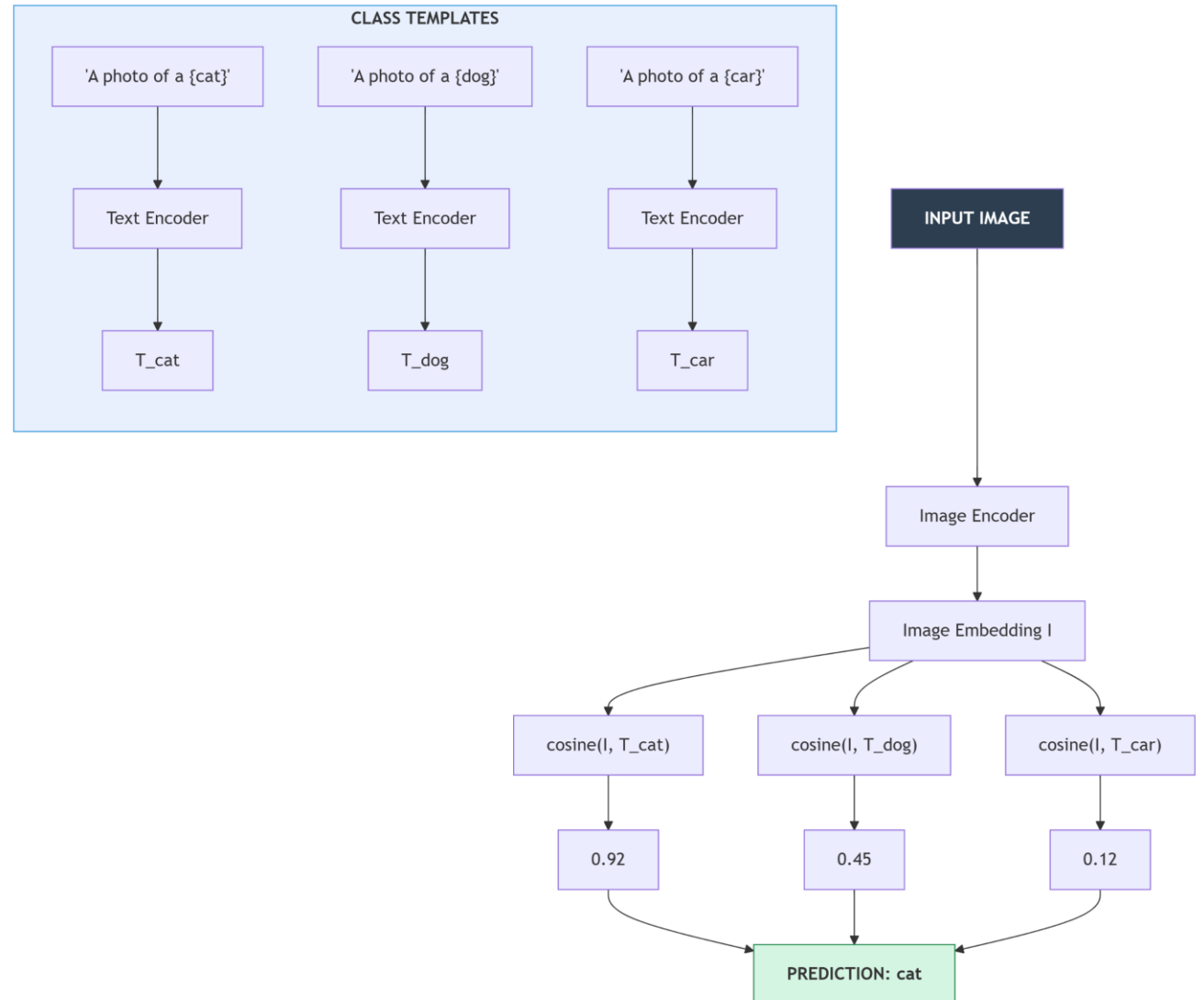
6. Pick the Best Match

$$\text{Prediction} = \arg \max_j S_j$$

Highest similarity $\rightarrow$ predicted class

CLIP Zero-Shot Classification

How CLIP performs zero-shot inference



CLIP Zero-Shot Classification PROBLEM:

Problem

Classify an image of a hedgehog

Step 1: Define Candidate Labels

labels = ["porcupine", "rat", "mole",
"hedgehog"]

Step 2: Create Text Prompts

T_1 = "A photo of a porcupine"
 T_2 = "A photo of a rat"
 T_3 = "A photo of a mole"
 T_4 = "A photo of a hedgehog"

Step 3: Encode Image and Text (2D Embeddings)

Image embedding:

$I = [0.85, 0.52]$

Text embeddings:

$T_{\text{porcupine}} = [0.72, 0.69]$

Similarity = $0.85 \times 0.72 + 0.52 \times 0.69 = 0.971$

$T_{\text{rat}} = [0.91, 0.41]$

Similarity = $0.85 \times 0.91 + 0.52 \times 0.41 = 0.987$

$T_{\text{mole}} = [0.63, 0.77]$

Similarity = $0.85 \times 0.63 + 0.52 \times 0.77 = 0.936$

$T_{\text{hedgehog}} = [0.88, 0.47]$

Similarity = $0.85 \times 0.88 + 0.52 \times 0.47 = 0.992$

CLIP Zero-Shot Classification PROBLEM:

Step 4: Apply Softmax with Temperature ($\tau = 0.07$)

Scaled scores:

$$0.971 / 0.07 \approx 13.87$$

$$0.987 / 0.07 \approx 14.10$$

$$0.936 / 0.07 \approx 13.37$$

$$0.992 / 0.07 \approx 14.17$$

Exponentials:

$$e^{13.87} \approx 1.05M$$

$$e^{14.10} \approx 1.34M$$

$$e^{13.37} \approx 0.64M$$

$$e^{14.17} \approx 1.43M$$

$$\text{Sum} \approx 4.46M$$

Step 5: Compute Probabilities

$$P(\text{porcupine}) \approx 0.24$$

$$P(\text{rat}) \approx 0.30$$

$$P(\text{mole}) \approx 0.14$$

$$P(\text{hedgehog}) \approx 0.32$$

Final Prediction

Predicted label: "hedgehog"
(Highest similarity & probability)



GenerativeAI and Its Applications

CLIP - CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

CLIP Applications:

1. Semantic Image Search

Query: "Vintage summer dress with floral pattern"

CLIP:

- Encodes query text
- Encodes all images
- Retrieves most similar images

Works even without tags or metadata

3. Open-Vocabulary Object Detection

Detect custom objects using text prompts:

```
model.set_classes([  
    "person wearing sunglasses",  
    "red backpack"  
])
```

No retraining required

2. Generative AI Control

Used in models like Stable Diffusion

CLIP:

- Interprets text prompt
- Evaluates generated images
- Guides generation toward prompt

Acts as a "critic" for image-text alignment

4. Facial Emotion Recognition (Zero-Shot)

Text prompts:

- "a happy face"
- "a sad face"
- "an angry face"

CLIP matches image to emotion without fine-tuning

CLIP - Strengths and Limitations

Strengths :

Zero-shot capabilities - recognize concepts not seen during training

Open vocabulary - any text prompt works, not fixed categories

Flexible - works for classification, retrieval, generation

Multilingual - text encoder can work with multiple languages

Foundation model - powers many downstream applications



Limitations:

Not fine-grained - struggles with subtle differences (e.g., car models)

Attribute binding - can confuse "red car" vs "car with red wheels"

Compositionality - "person riding horse" vs "horse riding person" similar

Counting - can't reliably count objects in images

Training data bias - inherits internet biases

Analogy:

CLIP is like a **well-read generalist**—they know a little about everything but may not have deep expertise in any specific domain

What is BLIP?

Definition

BLIP (Bootstrapping Language-Image Pre-training) is a unified vision-language framework developed by Salesforce that can handle both:

- Understanding tasks (e.g., retrieval, VQA)
- Generation tasks (e.g., image captioning)

Key Problem with Previous Models

1. Models specialize in ONLY one:

- Either understanding (CLIP)
- OR generation (captioning models)

2. Web data is noisy:

- Captions often do NOT match images
- Poor data → weak learning

What is BLIP?

Analogy

BLIP is like: Student (Captioner): Writes practice essays (captions)

Teacher (Filter): Checks and removes bad ones

Result: Only high-quality data is used, Both student and system improve over time

Why BLIP is Powerful

- Works for BOTH understanding and generation
- Improves noisy web data automatically
- More robust than earlier vision-language models oped by Salesforce that can handle both: Understanding tasks (e.g., retrieval, VQA), Generation tasks (e.g., image captioning)

The Problem BLIP Solves?

Web Data is Noisy:

Web image-caption pairs often look like:

Image:

Caption: "Click here for more information about our vacation packages!"

Image:

Caption: "Best pizza in New York - like and subscribe!"

Image:

Caption: "iPhone 15 Pro Max review #tech #gadgets #sponsored"

The Noisy Caption Problem •Web-scraped data contains **misaligned, irrelevant, or low-quality captions**

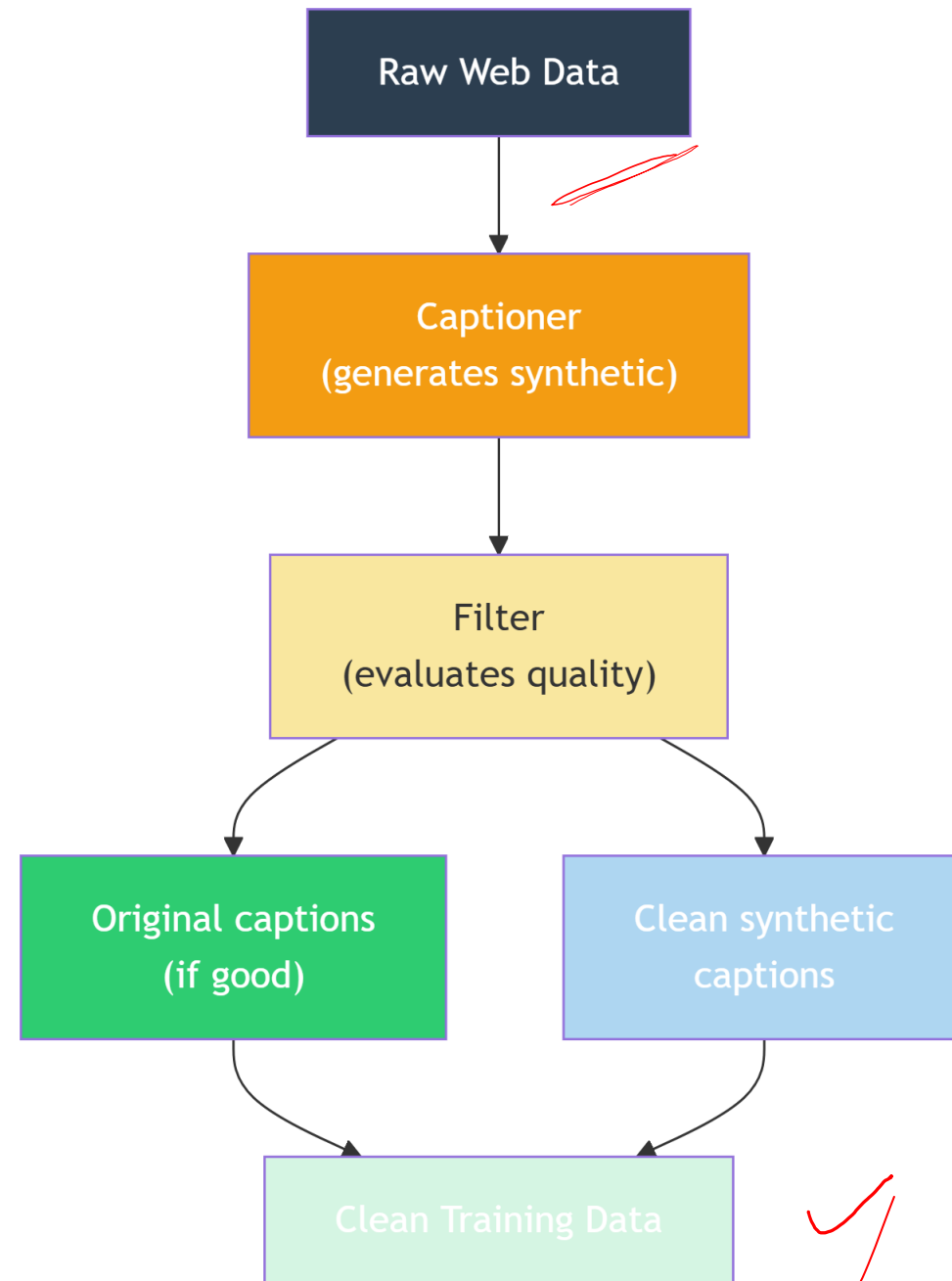
•Training on this noise leads to **suboptimal performance**

•Manual cleaning is **impossible at scale** (millions of images)

GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

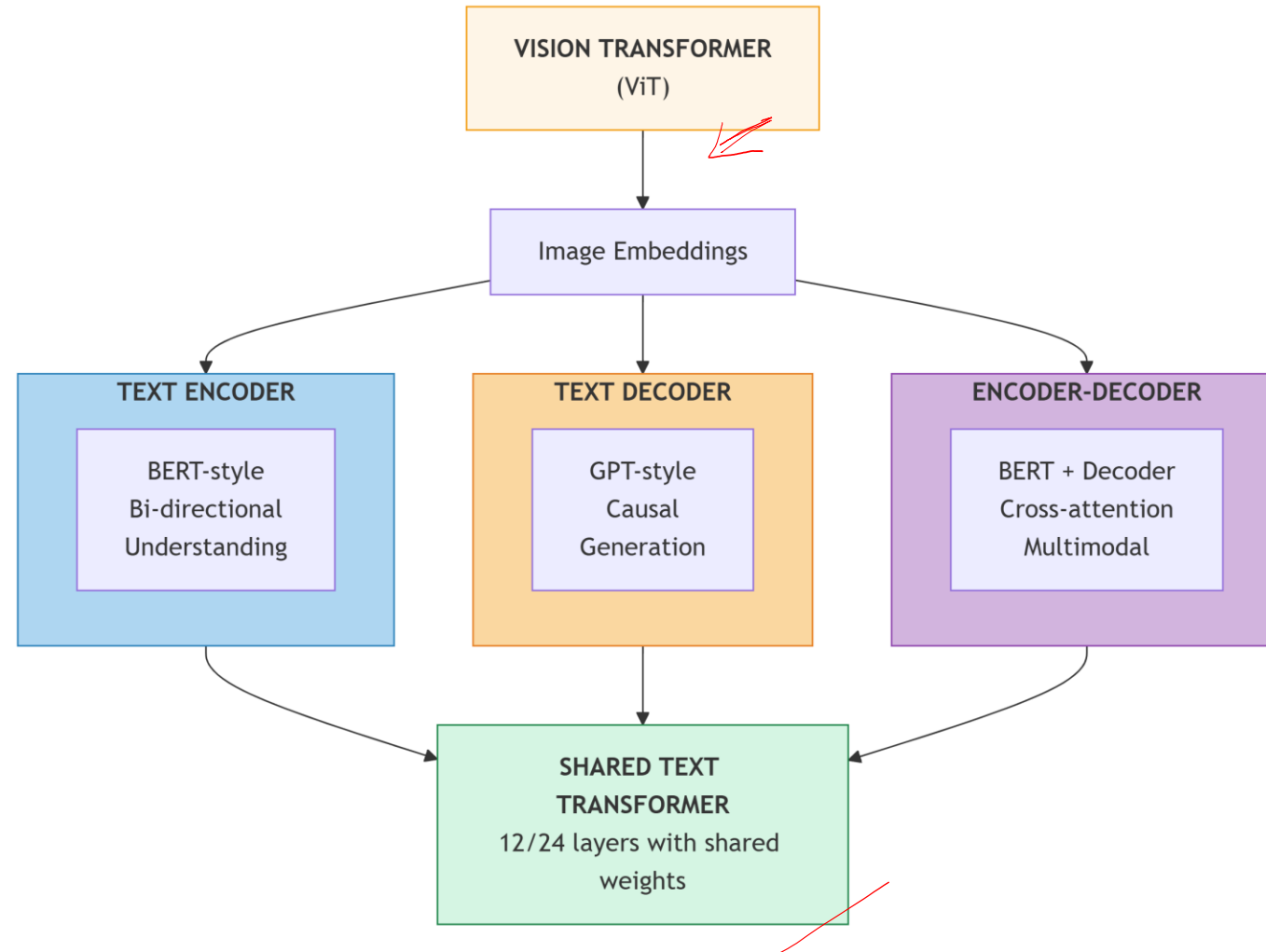
BLIP's Bootstrapping Approach?



GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

BLIP Architecture



GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

BLIP Model Variants

Pre-trained Checkpoints Available

Model	Vision Backbone	Pre-training Images	Tasks
BLIP Base	ViT-B/16	14M	Retrieval, VQA, NLVR2
BLIP CapFilt-L	ViT-B/16	129M	Captioning, VQA
BLIP Large	ViT-L/16	129M	High-accuracy tasks

Finetuned Models for Specific Tasks

Task	Dataset	BLIP Base	BLIP CapFilt-L	BLIP Large
Image-Text Retrieval	COCO	✓	-	✓
Image-Text Retrieval	Flickr30k	✓	-	✓
Image Captioning	COCO	-	✓	✓
VQA	VQA v2	✓	✓	-
NLVR2	NLVR2	✓	-	-

GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

BLIP - Understanding Tasks

How BLIP performs retrieval

Task: Given a query, find matching images or texts

Image-to-Text Retrieval:

Image —► Encode —► Compare with text embeddings —► Rank texts

Text-to-Image Retrieval:

Text —► Encode —► Compare with image embeddings —► Rank images

GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE-

IMAGE PRE-TRAINING Numerical Example - Image-Text Matching Score:

Example

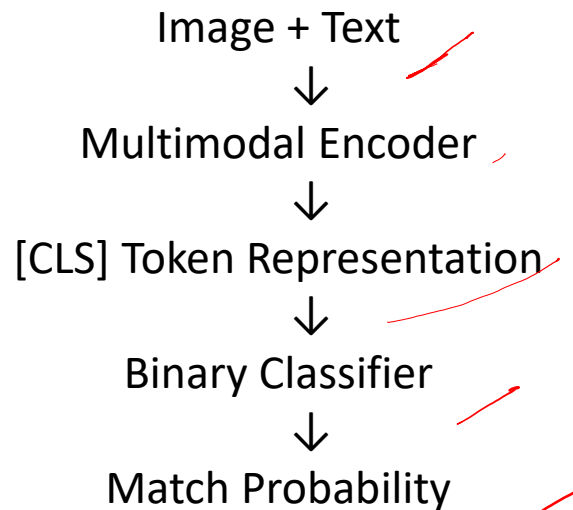
Image: (cat photo)

Text:

"A woman and a dog sitting together on a beach"

BLIP Computes Two Matching Scores

1. ITM Head Score (Learned Classifier)



Score = 0.12

→ Very LOW → Image and text do NOT match

2. Cosine Similarity Score (when *use_itm_head* = False)

Image Embedding · Text Embedding
= Cosine Similarity

Score = 0.23

→ LOW similarity → Semantically different

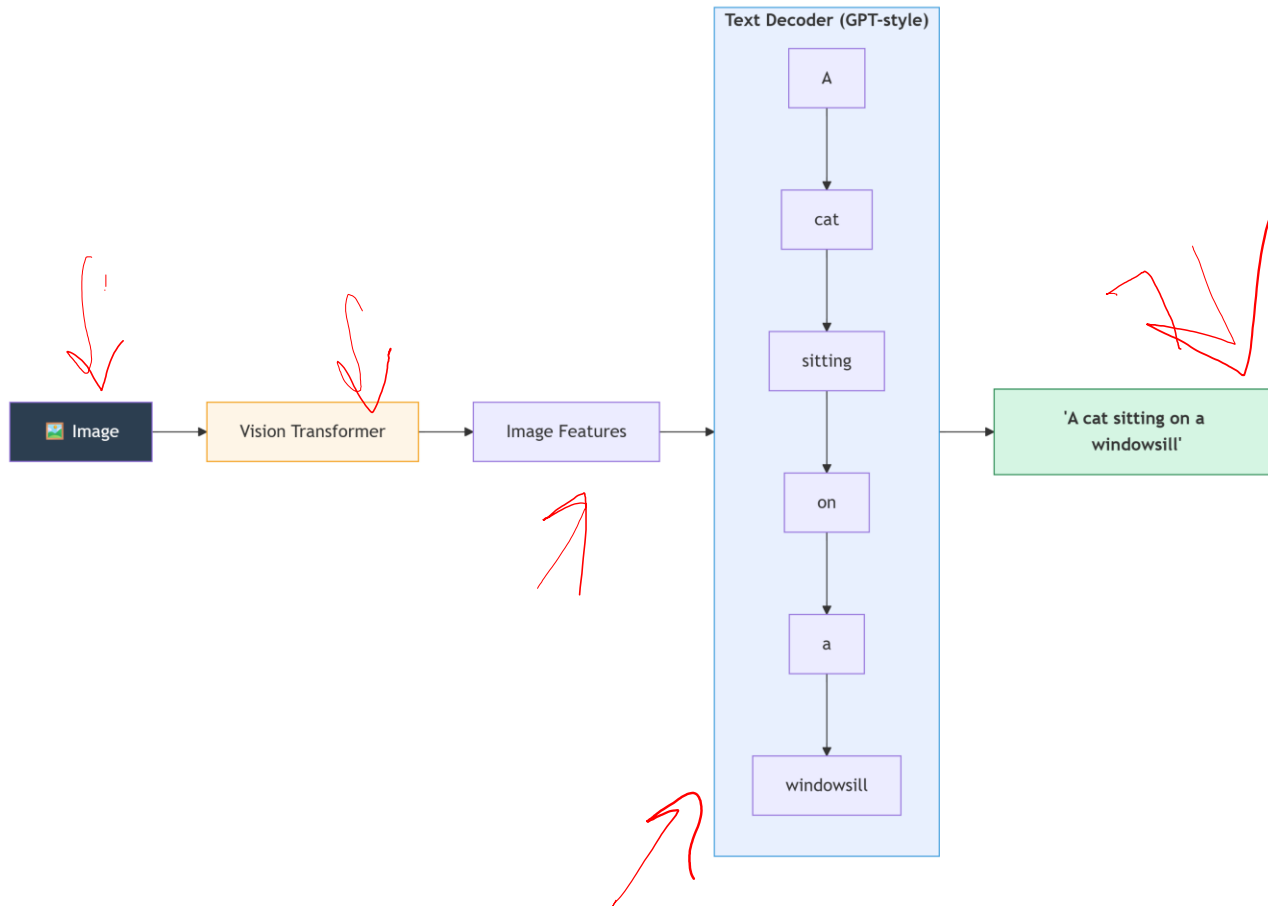
GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

How BLIP generates captions

BLIP - Generation Tasks (Image Captioning):

Image → Caption Generation



Generated So Far	Next Token Probability
<start>	$P("A") = 0.85$
"A"	$P("cat") = 0.78$
"A cat"	$P("sitting") = 0.72$
"A cat sitting"	$P("on") = 0.91$
"A cat sitting on"	$P("a") = 0.88$
"A cat sitting on a"	$P("windowsill") = 0.67$
"A cat sitting on a windowsill"	$P("<end>") = 0.95$

GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE- IMAGE PRE-TRAINING

BLIP for VQA

BLIP - Visual Question Answering (VQA)

BLIP treats VQA as answer generation, not classification:

Input: Image + Question

Output: Free-form text answer

Example:

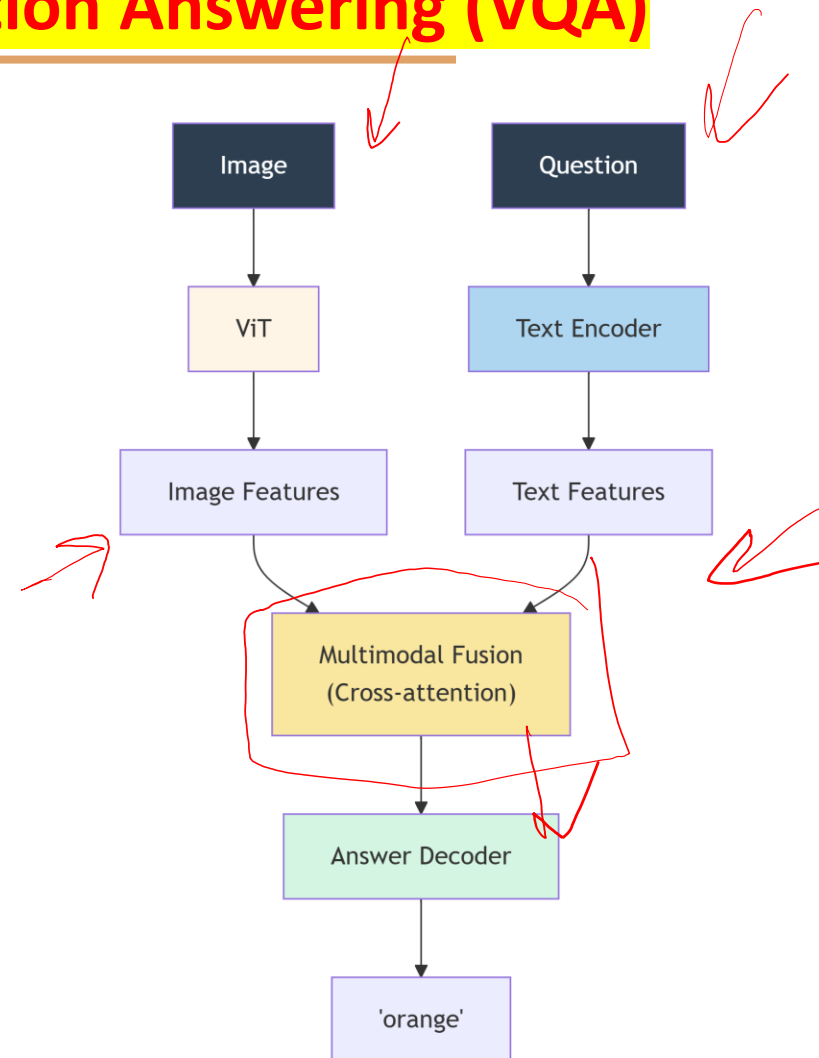
Q: "What is the cat doing?"

A: "sleeping on the couch"

Q: "How many people are in the image?"

A: "three"

This enables answering questions beyond fixed categories!



GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE-

IMAGE PRE-TRAINING **BLIP Applications**

Use Case	How BLIP is Used
E-commerce Product Catalog	Generates product descriptions, alt text, and searchable keywords from product images
Assistive Technology	Converts images into captions → Text-to-speech for visually impaired users
Content Moderation	Answers questions like “Does this image contain violence?” using VQA
Educational Tools	Explains diagrams or visual content uploaded by students
Social Media Accessibility	Automatically generates alt text for posts (e.g., Instagram images)

GenerativeAI and Its Applications

BLIP - BOOTSTRAPPING LANGUAGE-

IMAGE PRE-TRAINING **BLIP – Strengths vs Limitations**

Strengths	Limitations
Unified architecture (understanding + generation)	Higher computational cost than CLIP
Bootstrapping cleans noisy web data	Complex multi-task training
Strong performance on benchmarks	Requires large-scale data
Flexible: supports retrieval, captioning, VQA	Can inherit and amplify biases
Good generalization (even to video tasks)	Original repo deprecated

What is Whisper?

Definition:

Whisper is an automatic speech recognition (ASR) system developed by OpenAI, released as open source in 2022. It is trained on 680,000 hours of multilingual and multitask supervision data collected from the web .

Analogy:

Whisper is like a **universal translator** from Star Trek—it can understand almost any language, handle background noise, and even translate between languages, all in one system .

What is Whisper?

Key Capabilities

- Multilingual speech recognition (~99 languages)
- Speech translation (any language → English)
- Language identification
- Voice activity detection (speech vs silence)
- Timestamp prediction (word/segment timing)
- Robust to noise, accents, and recording conditions

GenerativeAI and Its Applications

WHISPER - SPEECH RECOGNITION MODEL

Whisper Architecture?

Encoder-Decoder Transformer

a visual-like
representation
of sound

special tokens

language tokens

task tokens

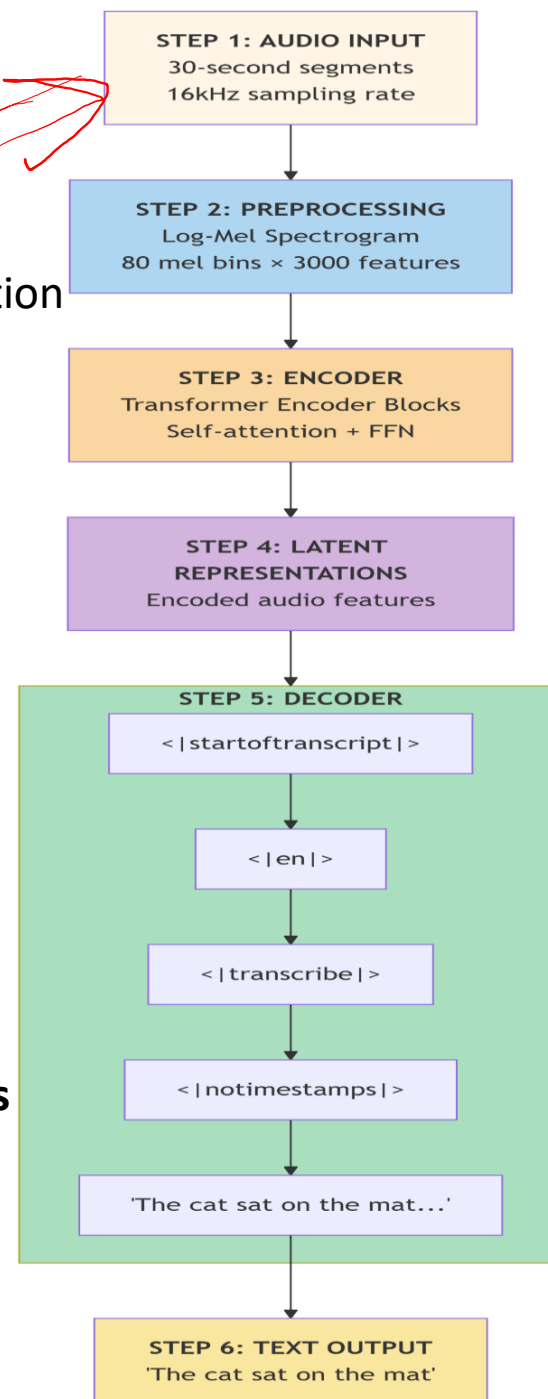
timestamps

produces the
transcription

raw audio is taken as
input (typically **30-second**
chunks at
16kHz sampling)



extracts
meaningful
patterns (like
phonemes, words,
accents) and
converts them into
latent
representations.



Whisper **listens** → **encodes** → **generates text sequentially.**

Whisper Preprocessing - Log-Mel Spectrogram

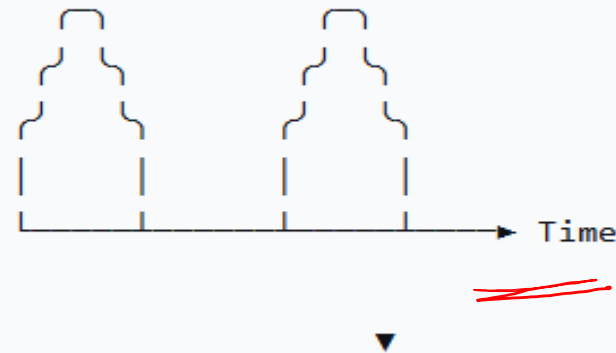
Converting Audio to Model Input

A **mel spectrogram** is a time-frequency representation that maps the frequency axis to the **mel scale**, which mirrors human auditory perception. **Humans are better at distinguishing lower frequencies than higher ones**, and the mel scale accounts for this.

Key Parameters:

- 80 mel frequency bins (frequency resolution)
- 25ms window with 10ms hop (time slicing of audio)
- 30-second segments = 3000 time steps
- Input shape: (80, 3000) per 30-second chunk

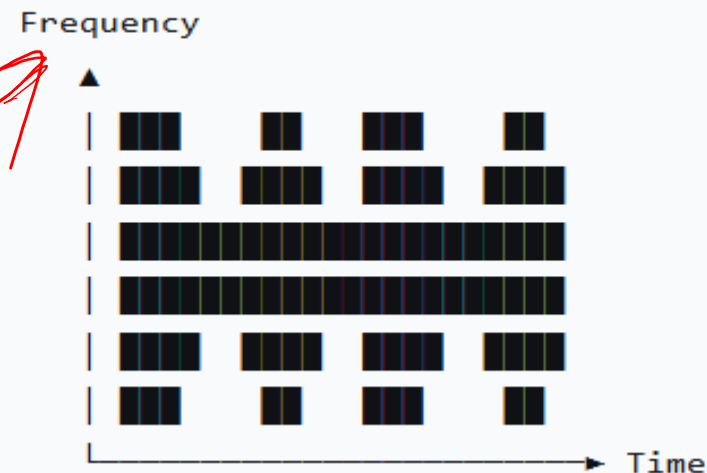
Raw Audio Waveform:



A **log-Mel spectrogram** is a 2D representation where:

- X-axis = time**
- Y-axis = frequency (on Mel scale)**
- Values represent energy (loudness)

Log-Mel Spectrogram (80 mel bands × 3000 time steps):



GenerativeAI and Its Applications

WHISPER - SPEECH RECOGNITION MODEL

Whisper Model Sizes

Key Observation

As model size increases:

- Accuracy improves
- Computational cost increases
- Inference speed decreases

Model	Parameters	English-only Version	Multilingual Version	Relative Speed	Typical Use Case
Tiny	39 Million	tiny.en	tiny	Very High	Embedded systems and edge devices
Base	74 Million	base.en	base	High	Mobile and lightweight applications
Small	244 Million	small.en	small	Moderate	General-purpose transcription
Medium	769 Million	medium.en	medium	Low	High-accuracy applications
Large	1.55 Billion	Not available	large	Very Low	Maximum accuracy requirements
Turbo	809 Million	Not available	turbo	Moderate–High	Optimized for faster inference

Whisper Training Data

Massive Scale and Diversity

Total training data: 680,000 hours

English Data – 563,000 hours (83%)

- YouTube videos
- Podcasts
- Audiobooks
- Lectures and interviews
- Real-world conversational speech

Non-English Data – 117,000 hours (17%)

• 98 languages represented

- European languages (Spanish, French, German, etc.)
- Asian languages (Chinese, Japanese, Korean, Hindi, etc.)
- Low-resource languages (Swahili, Welsh, Urdu, Tamil, etc.)

Whisper Training Data

Data Quality and Filtering

To handle noisy internet audio, OpenAI applied strict filtering:

Language mismatch filtering

- Removed clips where audio language $\neq$ subtitle language

Duration mismatch filtering

- Removed clips with **incomplete or partial transcripts**

Deduplication

- Removed repeated audio or **repeated captions**

Noise removal

- Deleted *non-speech text* such as:
 - “Please like and subscribe”
 - “Follow my channel”
 - “Don’t forget to share”

The scale and diversity allow Whisper to:

- Handle strong accents
- Work with noisy audio
- Understand multiple languages
- Recognize real-world conversational speech
- Perform speech recognition + translation

Context Tokens – How Whisper Controls Tasks

Special Tokens Guide Generation

Decoder Input Format

```
<|startoftranscript|> <|en|> <|transcribe|>  
<|notimestamps|>  
The cat sat on the mat...
```

These tokens tell the model:

- Which language to expect <|en|> (English)
- What task to perform <|transcribe|> (speech → text)
- Whether to generate timestamps <|notimestamps|>

Token Types:

START TOKEN

```
<|startoftranscript|>
```

Marks the beginning of text generation

LANGUAGE TOKENS

```
<|en|> – English
```

```
<|fr|> – French
```

```
<|es|> – Spanish
```

... up to **99 supported languages**

Context Tokens – How Whisper Controls Tasks

TASK TOKENS

<|transcribe|> – Speech → text in the same language

<|translate|> – Speech → English translation

TIMESTAMP TOKENS

<|notimestamps|> – Do not generate timestamps

Timestamp prediction tokens:

<|0.00|> <|2.34|> <|5.12|> ... actual time alignment

This allows Whisper to do:

- Transcription
- Translation
- Subtitle generation (with timing)

Whisper - Numerical Example

Audio: "the cat sat" (3 seconds)

Step 1: Audio → Log-Mel Spectrogram (80×300 for 3 seconds)

Step 2: Encoder produces hidden states $H \in \mathbb{R}^{(T \times d)}$ where
 T =sequence length, d =hidden dim

Step 3: Decoder generates tokens autoregressively

Time step 1:

Input: <|startoftranscript|>

Output logits over vocabulary (50,257 tokens)

Top predictions:

- <|en|>: 8.5
- <|fr|>: 3.2
- <|es|>: 3.1
- "the": 2.1

Selected: <|en|> (language identified as English)

Time step 2:

Input: <|startoftranscript|> <|en|>

Output logits:

- <|transcribe|>: 7.8
- <|translate|>: 4.2

Selected: <|transcribe|>

Time step 3:

Input: <|startoftranscript|> <|en|> <|transcribe|>

Output logits:

- <|notimestamps|>: 6.5
- <|0.00|>: 2.1

Selected: <|notimestamps|>

Whisper Decoding (Continued)

Time step 4:

Input: [$\langle \text{start} \rangle$, $\langle \text{en} \rangle$, $\langle \text{transcribe} \rangle$, $\langle \text{notimestamps} \rangle$]

Output logits for first word:

- "the": 7.2
- "a": 5.1
- "cat": 4.3
- "dog": 3.8

Selected: "the" (probability = $\text{softmax}(7.2) \approx 0.65$)

Time step 5:

Input: [..., "the"]

Output:

- "cat": 8.1
- "dog": 4.2
- "bird": 3.1

Selected: "cat" ($P \approx 0.78$)

Time step 6:

Input: [..., "the", "cat"]

Output:

- "sat": 7.9
- "slept": 5.2
- "ran": 4.8

Selected: "sat" ($P \approx 0.71$)

Time step 7:

Input: [..., "the", "cat", "sat"]

Output:

- $\langle \text{endoftext} \rangle$: 8.5
- "on": 3.2
- "and": 2.8

Selected: $\langle \text{endoftext} \rangle$ (end of transcription)

At each step, the model outputs **logits (scores)** over all possible words and uses **softmax** to convert them into probabilities, selecting the most likely one.

Final transcription: "the cat sat"

Whisper Performance Metrics

Word Error Rate (WER) Comparison

Lower WER = better speech recognition

Dataset	wav2vec 2.0	Whisper Large-v2	Improvement
LibriSpeech	1.8%	1.7%	+5.6%
TED-LIUM	4.2%	3.9%	+7.1%
Common Voice	8.1%	5.2%	+35.8%
VoxPopuli	9.3%	5.8%	+37.6%
CHiME-6	32.4%	18.9%	+41.7%

Whisper shines in **real-world conditions**, not just clean benchmarks.

Whisper matches top performance on clean datasets and **greatly improves accuracy on noisy / real-world audio**.

Why Whisper Generalizes Better

wav2vec 2.0 was mainly fine-tuned on **LibriSpeech** (clean audio).

Whisper:

- Trained on **680,000 hours** of diverse audio
- Includes noisy speech, accents, and real-world recordings
- Performs consistently across multiple datasets
- **Result:** Better generalization, not just benchmark performance.

Real-Time Factor (RTF)

RTF < 1 = real-time capable

RTF measures **how fast the model processes audio relative to real time:**

- **RTF < 1 → real-time capable** (faster than audio duration)
- Example: RTF = 0.5 → 10 sec audio processed in 5 sec

Model	RTF (CPU)	RTF (GPU)	Real-Time Capable?
tiny	0.02	0.004	Yes
base	0.03	0.006	Yes
small	0.07	0.012	Yes
medium	0.15	0.025	Near Real-Time
large	0.32	0.050	No
turbo	0.08	0.015	Yes

Whisper Applications

1. Real-Time Captioning (Video Conferencing)

Challenge: Less than 500 ms latency

Solution:

Use **tiny / base models**

Stream audio in small chunks

Use sliding-window inference

Examples: Zoom captions, Google Meet live captions

2. Medical Transcription

Challenge: Medical terminology + data privacy

Solution:

- Fine-tune on medical conversations
- Deploy on-premise (no cloud processing)
- Add medical term dictionary for post-processing

Result: Accurate doctor-patient transcription

3. Industrial Quality Control

Use case: Voice-based defect logging

Workers speak issues → system logs them

Workers say:

- “Scratch on panel 7”
- “Loose screw detected”

Workflow:

Speech → Whisper transcription → MES system →
Quality report

4. Multilingual Content Creation

Example workflow: Podcast → transcription → translation →
subtitles

Spanish podcast → Whisper transcription

→ English translation

→ Auto-generated subtitles in 10+ languages

Used for:

- Podcasts
- YouTube videos
- Online courses and Interviews

Whisper Applications

5. Edge Device Deployment

Whisper can run directly on embedded devices.

Example demos:

- 1.58-bit quantized Whisper models
- 4× memory reduction
- 2× faster inference

Devices demonstrated on:

- Synaptics Astra board
- Renesas RZ/V2H board

This enables:

- Offline transcription
- Privacy-sensitive applications
- Deployment on IoT / embedded boards

Whisper Optimization Techniques

1. Quantization

Goal: Reduce memory + speed up inference

Example (ONNX + 8-bit quantization)

```
from optimum.onnxruntime import ORTModelForSeq2SeqLM

model = ORTModelForSeq2SeqLM.from_pretrained(
    "openai/whisper-tiny",
    export=True,
    provider="CPUExecutionProvider"
)
```

Result:

- Memory: **156MB → 39MB** (75% reduction)
- Speed: **2–3× faster inference**
- Works well for edge devices

Goal: Make model smaller and faster

•Convert weights from **32-bit → 8-bit**

•Example:

- Memory: **156MB → 39MB (75% reduction)**
- Speed: **2–3× faster**

Whisper Optimization Techniques

2. Distillation

Idea: Train a small model to imitate a large model

Teacher Model: Whisper Large (1.55B parameters)

Student Model: Whisper Small (244M parameters)

Benefits:

- Accuracy loss: < 3%
- Speed gain: **up to 6× faster**
- Lower memory usage

Whisper Optimization Techniques

3. Streaming for Real-Time Applications

Goal: Process audio while it is still being recorded

Method:

- Split audio into chunks
- Process every few seconds
- Append results continuously

```
def stream_transcribe(audio_stream):  
    # Process audio chunks continuously  
    # Generate transcript in real-time  
    return transcript
```

Used in:

- Live captions
- Meetings
- Voice assistants
- Real-time subtitles

Whisper – Strengths and Limitations

Strengths

- Multilingual – Supports **99 languages in one model**
- Robust – Handles noise, accents, and different recording conditions
- Multitask – Transcription, translation, and language detection
- Zero-shot – Works without fine-tuning
- Open source – Multiple model sizes (tiny → large)
- Strong generalization – Performs well across diverse datasets

Limitations

- Long audio – Requires segmentation for audio longer than 30 seconds
- Real-time bottleneck – Large models are slow on CPU
- Uneven language accuracy – Some languages perform better than others
- No speaker diarization – Cannot distinguish speakers
- Timestamp precision – Not accurate enough for word-level timestamps

Whisper – Strengths and Limitations

Language Performance Variation

High Accuracy

English

Spanish

German

French

Medium Accuracy

Hindi

Arabic

Vietnamese

Lower Accuracy (low-resource languages)

Swahili

Welsh

Urdu

GenerativeAI and Its Applications

WHISPER - SPEECH RECOGNITION MODEL

Model Comparison Matrix

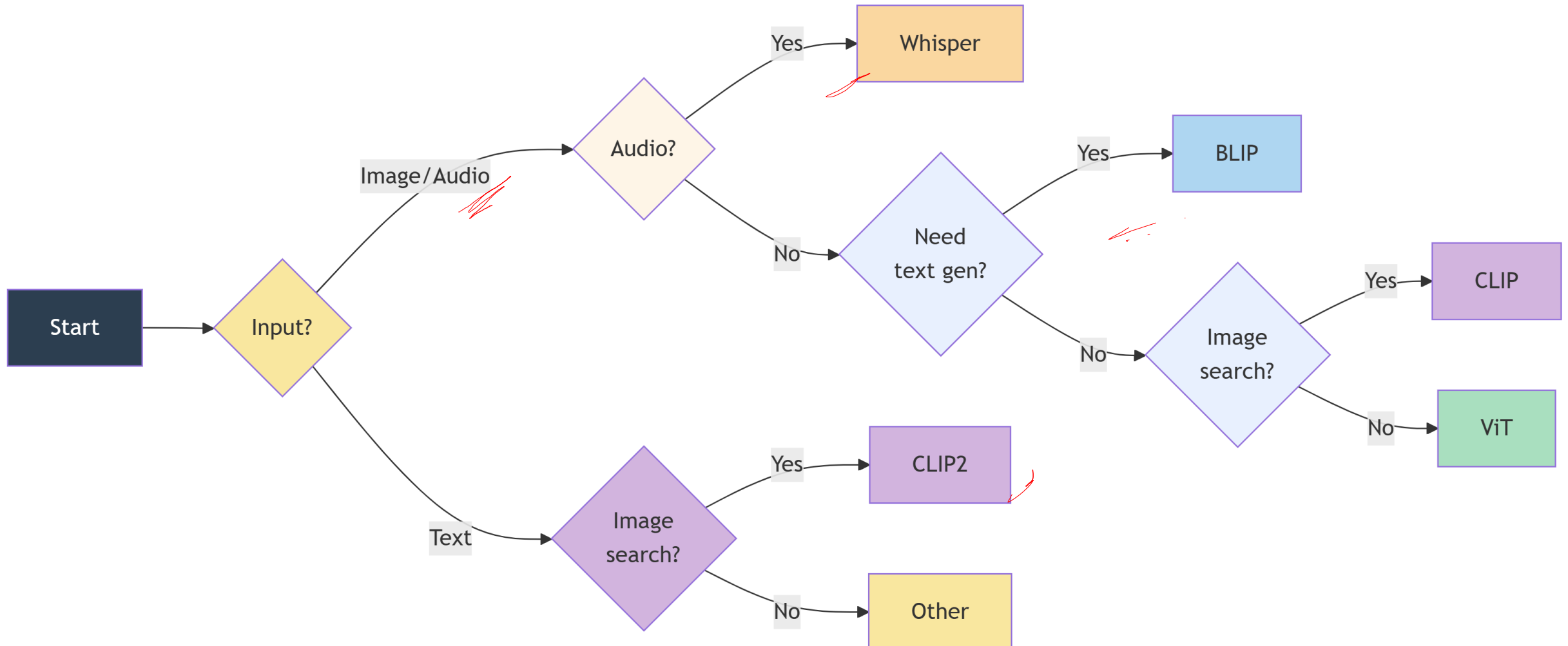
Feature	CLIP	BLIP	Whisper
Modalities	Image + Text	Image + Text	Audio + Text
Architecture	Dual Encoder	Unified Multi-task	Encoder-Decoder
Training Data	400M image-text pairs	129M image-text pairs	680K hours audio
Primary Tasks	Zero-shot classification, Retrieval	Captioning, VQA, Retrieval	Transcription, Translation
Output	Similarity scores	Text, scores	Text
Strengths	Zero-shot, open vocabulary	Unified understanding+generation	Multilingual, robust
Limitations	No generation, attribute binding	Complex training	Long audio, real-time
Release Year	2021	2022	2022
Company	OpenAI	Salesforce	OpenAI

GenerativeAI and Its Applications

WHISPER - SPEECH RECOGNITION MODEL : When to Use Which Model



Choose Your Multimodal Model



Practice Problems

Problem 1: CLIP Zero-Shot Classification

Given image embedding $I = [0.6, 0.8, 0.3]$ and text embeddings:

- $T_1 = [0.5, 0.7, 0.4]$ ("cat")
- $T_2 = [0.9, 0.2, 0.5]$ ("dog")
- $T_3 = [0.3, 0.6, 0.8]$ ("bird")

Calculate cosine similarities and determine the predicted class.

Problem 2: CLIP Contrastive Loss

For a batch of 2 image-text pairs:

$I_1 = [0.9, 0.2]$, $T_1 = [0.8, 0.3]$

$I_2 = [0.3, 0.9]$, $T_2 = [0.2, 0.8]$

Compute the similarity matrix and contrastive loss with $\tau=0.1$.

Problem 3: BLIP Image Captioning

Given the partial caption "A dog is" and next token logits:

- "running": 5.2
- "sleeping": 4.8
- "eating": 4.5
- "barking": 5.0

Calculate probabilities and determine the most likely next token.

Problem 4: Whisper Transcription

Audio contains: "Where is the library"

Language identification logits: English=7.5, Spanish=4.2, French=3.8

First word logits: "where"=6.8, "what"=5.2, "how"=4.9

Determine:

- Most likely language
- Most likely first word
- Complete transcription path

Problem 5: Model Selection

You need to build a system that:

- Takes photos of products
- Generates product descriptions
- Allows text search for products
- Works in 5 languages

Which models would you use and why?

Problem 1: CLIP Zero-Shot Classification

Given image embedding $I = [0.6, 0.8, 0.3]$ and text embeddings:

- $T_1 = [0.5, 0.7, 0.4]$ ("cat")
- $T_2 = [0.9, 0.2, 0.5]$ ("dog")
- $T_3 = [0.3, 0.6, 0.8]$ ("bird")

Calculate cosine similarities and determine the predicted class

Given:

- Image embedding $I = [0.6, 0.8, 0.3]$
- $T_1 = [0.5, 0.7, 0.4]$ ("cat")
- $T_2 = [0.9, 0.2, 0.5]$ ("dog")
- $T_3 = [0.3, 0.6, 0.8]$ ("bird")

Step 1: Cosine Similarity Formula

$$\cos(\theta) = (I \cdot T) / (\|I\| \times \|T\|)$$

Step 2: Calculate Norms

$$\|I\| = \sqrt{0.6^2 + 0.8^2 + 0.3^2} = \sqrt{0.36 + 0.64 + 0.09} = \sqrt{1.09} = 1.044$$

$$\|T_1\| = \sqrt{0.5^2 + 0.7^2 + 0.4^2} = \sqrt{0.25 + 0.49 + 0.16} = \sqrt{0.90} = 0.949$$

$$\|T_2\| = \sqrt{0.9^2 + 0.2^2 + 0.5^2} = \sqrt{0.81 + 0.04 + 0.25} = \sqrt{1.10} = 1.049$$

$$\|T_3\| = \sqrt{0.3^2 + 0.6^2 + 0.8^2} = \sqrt{0.09 + 0.36 + 0.64} = \sqrt{1.09} = 1.04$$

Step 3: Calculate Dot Products

$$I \cdot T_1 = (0.6 \times 0.5) + (0.8 \times 0.7) + (0.3 \times 0.4) = 0.30 + 0.56 + 0.12 = 0.98$$

$$I \cdot T_2 = (0.6 \times 0.9) + (0.8 \times 0.2) + (0.3 \times 0.5) = 0.54 + 0.16 + 0.15 = 0.85$$

$$I \cdot T_3 = (0.6 \times 0.3) + (0.8 \times 0.6) + (0.3 \times 0.8) = 0.18 + 0.48 + 0.24 = 0.90$$

Step 4: Calculate Cosine Similarities

$$\cos(I, T_1) = 0.98 / (1.044 \times 0.949) = 0.98 / 0.991 = 0.989$$

$$\cos(I, T_2) = 0.85 / (1.044 \times 1.049) = 0.85 / 1.095 = 0.776$$

$$\cos(I, T_3) = 0.90 / (1.044 \times 1.044) = 0.90 / 1.090 = 0.826$$

Step 5: Predicted Class

Highest similarity: $\cos(I, T_1) = 0.989 \rightarrow$ "cat"

Prediction: CAT

Answer: The image is most similar to "cat" (0.989)

Problem 2: CLIP Contrastive Loss

For a batch of 2 image-text pairs:

$$I_1 = [0.9, 0.2], T_1 = [0.8, 0.3]$$

$$I_2 = [0.3, 0.9], T_2 = [0.2, 0.8]$$

Compute the similarity matrix and contrastive loss with $\tau=0.1$.

• $I_1 = [0.9, 0.2]$, $T_1 = [0.8, 0.3]$ (matching pair)

• $I_2 = [0.3, 0.9]$, $T_2 = [0.2, 0.8]$ (matching pair)

• $\tau = 0.1$ (temperature)

Step 1: Normalize Vectors

$$\|I_1\| = \sqrt{0.9^2 + 0.2^2} = \sqrt{0.81 + 0.04} = \sqrt{0.85} = 0.922$$

$$I_{1_norm} = [0.9/0.922, 0.2/0.922] = [0.976, 0.217]$$

$$\|I_2\| = \sqrt{0.3^2 + 0.9^2} = \sqrt{0.09 + 0.81} = \sqrt{0.90} = 0.949$$

$$I_{2_norm} = [0.3/0.949, 0.9/0.949] = [0.316, 0.949]$$

$$\|T_1\| = \sqrt{0.8^2 + 0.3^2} = \sqrt{0.64 + 0.09} = \sqrt{0.73} = 0.854$$

$$T_{1_norm} = [0.8/0.854, 0.3/0.854] = [0.937, 0.351]$$

$$\|T_2\| = \sqrt{0.2^2 + 0.8^2} = \sqrt{0.04 + 0.64} = \sqrt{0.68} = 0.825$$

$$T_{2_norm} = [0.2/0.825, 0.8/0.825] = [0.242, 0.970]$$

Step 2: Compute Similarity Matrix

text

$$S[1,1] = I_{1_norm} \cdot T_{1_norm} = 0.976 \times 0.937 + 0.217 \times 0.351 = 0.915 + 0.076 = 0.991$$

$$S[1,2] = I_{1_norm} \cdot T_{2_norm} = 0.976 \times 0.242 + 0.217 \times 0.970 = 0.236 + 0.210 = 0.446$$

$$S[2,1] = I_{2_norm} \cdot T_{1_norm} = 0.316 \times 0.937 + 0.949 \times 0.351 = 0.296 + 0.333 = 0.629$$

$$S[2,2] = I_{2_norm} \cdot T_{2_norm} = 0.316 \times 0.242 + 0.949 \times 0.970 = 0.076 + 0.921 = 0.997$$

Similarity Matrix S =

$$\begin{vmatrix} 0.991 & 0.446 \\ 0.629 & 0.997 \end{vmatrix}$$

Step 3: Temperature Scaling ($\tau=0.1$)

text

$$S/\tau = S \times 10$$

$$S_{scaled} = \begin{vmatrix} 9.91 & 4.46 \\ 6.29 & 9.97 \end{vmatrix}$$

Step 4: Softmax for Image I_1

$$\exp(9.91) = 20,187$$

$$\exp(4.46) = 86.4$$

$$\text{Sum} = 20,187 + 86.4 = 20,273.4$$

$$P(I_1 \rightarrow T_1) = 20,187 / 20,273.4 = 0.996$$

$$P(I_1 \rightarrow T_2) = 86.4 / 20,273.4 = 0.004$$



Step 5: Softmax for Image I_2

$$\exp(6.29) = 540$$

$$\exp(9.97) = 21,392$$

$$\text{Sum} = 540 + 21,392 = 21,932$$

$$P(I_2 \rightarrow T_1) = 540 / 21,932 = 0.025$$

$$P(I_2 \rightarrow T_2) = 21,392 / 21,932 = 0.975$$

Step 6: Compute Loss

$$\text{Loss} = -[\log(P(I_1 \rightarrow T_1)) + \log(P(I_2 \rightarrow T_2))] / 2$$

$$= -[\log(0.996) + \log(0.975)] / 2$$

$$= -[-0.004 + (-0.025)] / 2$$

$$= -[-0.029] / 2$$

$$= 0.029 / 2 = 0.0145$$

Answer: Contrastive Loss = 0.0145 (very low, indicating good alignment)

Problem 3: BLIP Image Captioning

Given the partial caption "A dog is" and next token logits:

- "running": 5.2
- "sleeping": 4.8
- "eating": 4.5
- "barking": 5.0

Calculate probabilities and determine the most likely next token.

Step 1: Softmax Formula

$$P(\text{token}) = e^{\text{logit}} / \sum e^{\text{logits}}$$

Step 2: Calculate Exponentials

$$e^{5.2} = 181.27$$

$$e^{5.0} = 148.41$$

$$e^{4.8} = 121.51$$

$$e^{4.5} = 90.02$$

$$\text{Sum} = 181.27 + 148.41 + 121.51 + 90.02 = 541.21$$

Step 3: Calculate Probabilities

$$P(\text{"running"}) = 181.27 / 541.21 = 0.335 \text{ (33.5\%)}$$

$$P(\text{"barking"}) = 148.41 / 541.21 = 0.274 \text{ (27.4\%)}$$

$$P(\text{"sleeping"}) = 121.51 / 541.21 = 0.224 \text{ (22.4\%)}$$

$$P(\text{"eating"}) = 90.02 / 541.21 = 0.166 \text{ (16.6\%)}$$

Step 4: Determine Most Likely Token

Highest probability: "running" with 33.5%

Step 5: Complete Caption Path

Partial: "A dog is"

Next: "running"

Complete: "A dog is running"

Answer: Most likely next token is "running" (33.5%)

Problem 4: Whisper Transcription

Audio contains: "Where is the library"

Language identification logits: English=7.5, Spanish=4.2, French=3.8

First word logits: "where"=6.8, "what"=5.2, "how"=4.9

Determine:

- Most likely language
- Most likely first word
- Complete transcription path

GenerativeAI and Its Applications

Problem 4: Whisper Transcription

Audio contains: "Where is the library"

Language identification logits: English=7.5, Spanish=4.2, French=3.8

First word logits: "where"=6.8, "what"=5.2, "how"=4.9

Determine:

- Most likely language
- Most likely first word
- Complete transcription path

Step 1: Determine Most Likely Language

Softmax for languages:

$$\text{Denom} = e^{7.5} + e^{4.2} + e^{3.8}$$

$$= 1,808.0 + 66.7 + 44.7 = 1,919.4$$

$$P(\text{English}) = 1,808.0 / 1,919.4 = 0.942 \text{ (94.2\%)}$$

$$P(\text{Spanish}) = 66.7 / 1,919.4 = 0.035 \text{ (3.5\%)}$$

$$P(\text{French}) = 44.7 / 1,919.4 = 0.023 \text{ (2.3\%)}$$

Most likely language: ENGLISH (94.2%)

GenerativeAI and Its Applications

Problem 4: Whisper Transcription

Audio contains: "Where is the library"

Language identification logits: English=7.5, Spanish=4.2, French=3.8

First word logits: "where"=6.8, "what"=5.2, "how"=4.9

Determine:

- Most likely language
- Most likely first word
- Complete transcription path

Step 2: Determine Most Likely First Word

Softmax for first word:

$$\begin{aligned}\text{Denom} &= e^{6.8} + e^{5.2} + e^{4.9} \\ &= 898.0 + 181.3 + 134.3 = 1,213.6\end{aligned}$$

$$P(\text{"where"}) = 898.0 / 1,213.6 = 0.740 \text{ (74.0\%)}$$

$$P(\text{"what"}) = 181.3 / 1,213.6 = 0.149 \text{ (14.9\%)}$$

$$P(\text{"how"}) = 134.3 / 1,213.6 = 0.111 \text{ (11.1\%)}$$

Most likely first word: "where" (74.0%)

GenerativeAI and Its Applications

Problem 4: Whisper Transcription

Audio contains: "Where is the library"

Language identification logits: English=7.5, Spanish=4.2, French=3.8

First word logits: "where"=6.8, "what"=5.2, "how"=4.9

Determine:

- Most likely language
- Most likely first word
- Complete transcription path

Step 3: Complete Transcription Path

Step 1: <|startoftranscript|>

Step 2: <|en|> (English language selected)

Step 3: <|transcribe|> (transcription task)

Step 4: <|notimestamps|> (no timestamps needed)

Step 5: "where" (highest probability: 74.0%)

Step 6: Continue generation...

Step 7: "is"

Step 8: "the"

Step 9: "library"

Step 10: <|endoftext|>

Complete transcription: "where is the library"

Answer: Language = English, First word = "where" (74.0%), Complete transcription: "where is the library"

Problem 5: Model Selection

You need to build a system that:

- Takes photos of products
- Generates product descriptions
- Allows text search for products
- Works in 5 languages

Which models would you use and why?

GenerativeAI and Its Applications

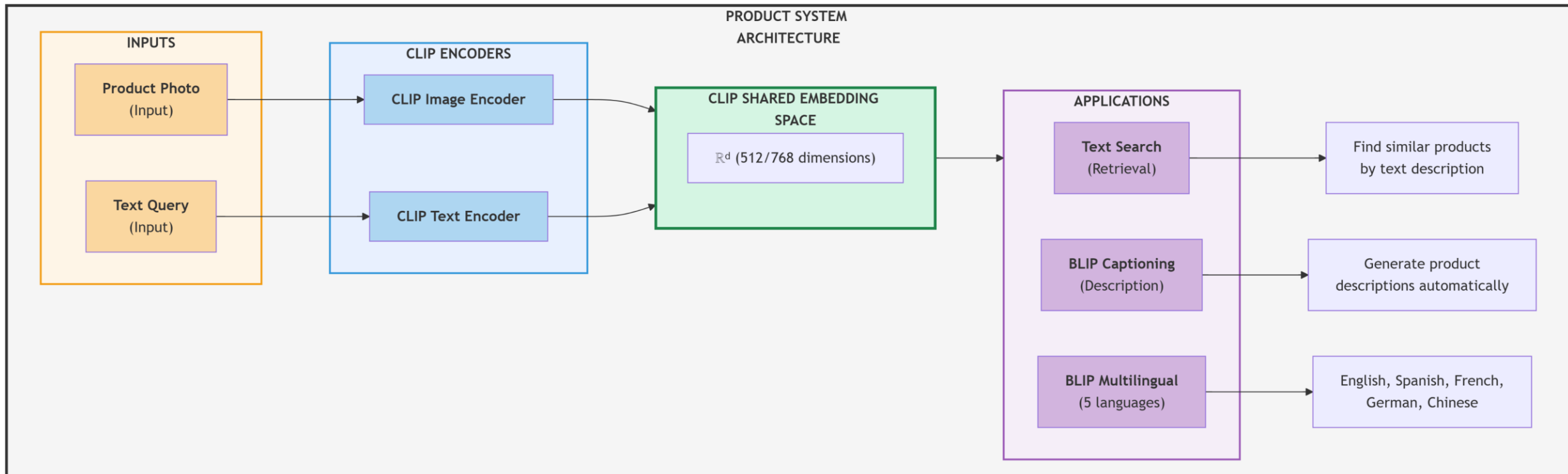
- Takes photos of products

Generates product descriptions

- Allows text search for products

- Works in 5 languages

Product System Architecture - CLIP + BLIP



GenerativeAI and Its Applications



- Takes photos of products
- Generates product descriptions
- Allows text search for products
- Works in 5 languages

Requirement	Model	Why
Product photos	CLIP	Open vocabulary, can match any product type without retraining
Text search	CLIP	Zero-shot retrieval in shared embedding space
Generate descriptions	BLIP	Best for image captioning, can be fine-tuned on product data
5 languages	CLIP + BLIP	CLIP works with multilingual text; BLIP can be extended with multilingual encoder

Implementation Strategy – Product Image Search System



1. CLIP for Search (Primary Model)

- Use **CLIP ViT-B/16** (best balance of speed and accuracy)
- Pre-compute embeddings for all product images
- User text query → Text encoder → Cosine similarity with image embeddings
- Returns the most relevant products instantly

2. BLIP for Product Description Generation

- Use **BLIP Large** fine-tuned on product catalog
- Input: Product image
- Output: Structured caption

Example:

“A black leather backpack with silver zippers”

3. Multilingual Support

- Use **Multilingual CLIP (mCLIP)** for multi-language search
- Translate generated captions using **NLLB model**
- Supports product search in multiple languages

Expected Performance

- Search latency: **<100 ms**
- Caption generation: **<500 ms**
- Search accuracy: **92–95%**

Alternative (High-End Option)

LLaVA

- More expensive but more powerful
- Can perform image search + captioning + visual Q&A in one model
- Better for complex product queries



THANK YOU

Generative AI and Its Applications



Unit 4

(UE23CS342BA4)

Introduction to Agentic Workflows, Design Patterns & Frameworks

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

GenerativeAI and Its Applications

Acknowledgement



The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

GenerativeAI and Its Applications

INTRODUCTION TO AGENTIC AI

What is Agentic AI?

Definition:

Agentic AI refers to artificial intelligence systems that can act autonomously to achieve goals, making decisions, taking actions, and adapting to changing

Analogy: Traditional AI is like a **calculator** it does exactly what you tell it. Generative AI is like a **writer** it creates content from prompts. Agentic AI is like a **personal assistant** it understands your goal, plan the actions, execute tasks and adapts when things change.

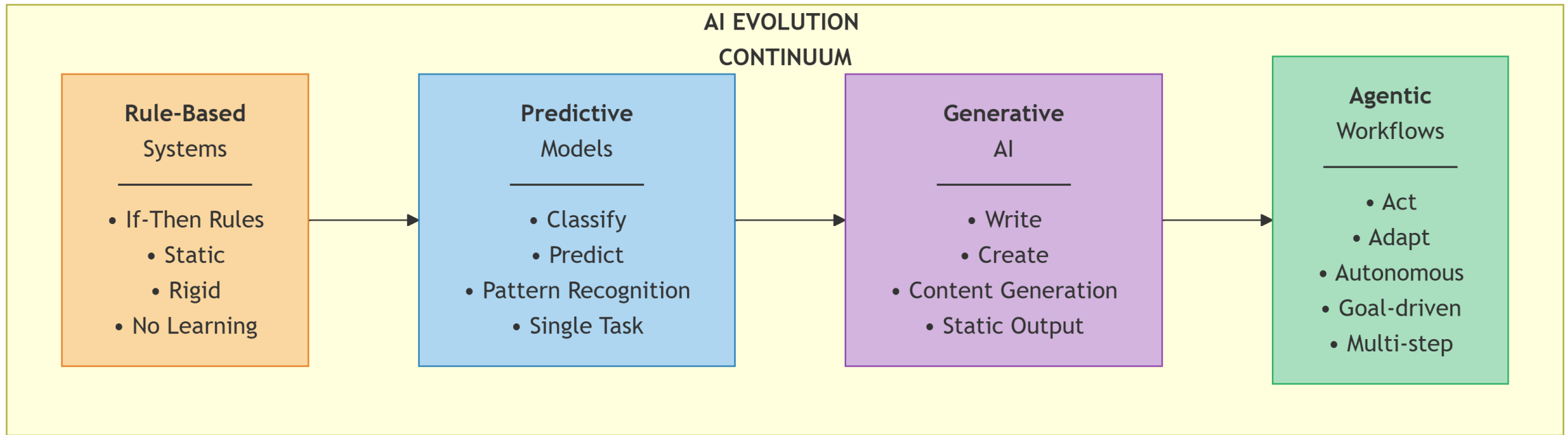
Example:

"Book me a trip to Delhi within ₹10,000"

→ Agent:

- Searches flights
- Compares prices
- Books ticket
- Sends confirmation

AI Evolution Continuum



GenerativeAI and Its Applications

INTRODUCTION TO AGENTIC AI

Key Characteristics of Agentic Systems

What Makes a System "Agentic"?

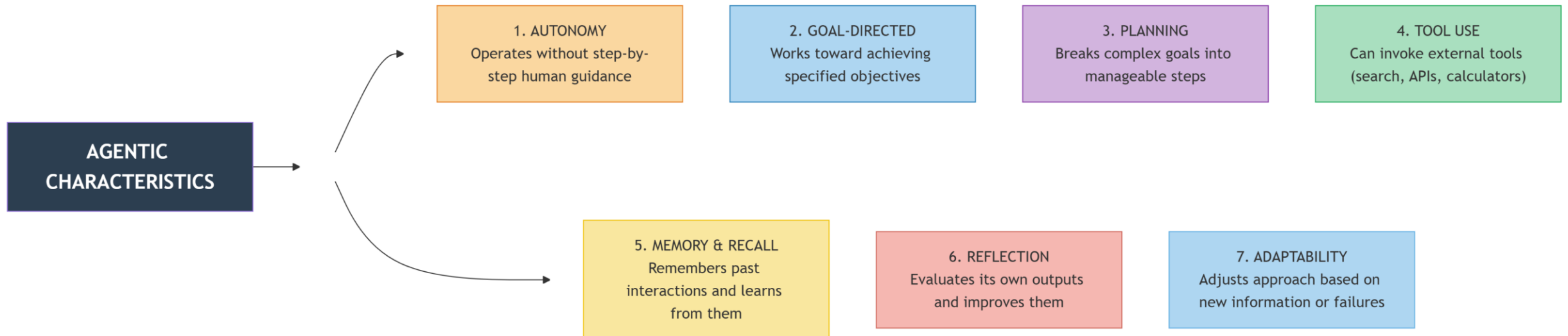
Think of a **self-driving car** 🚗

- Senses environment
- Makes decisions
- Adjusts in real-time

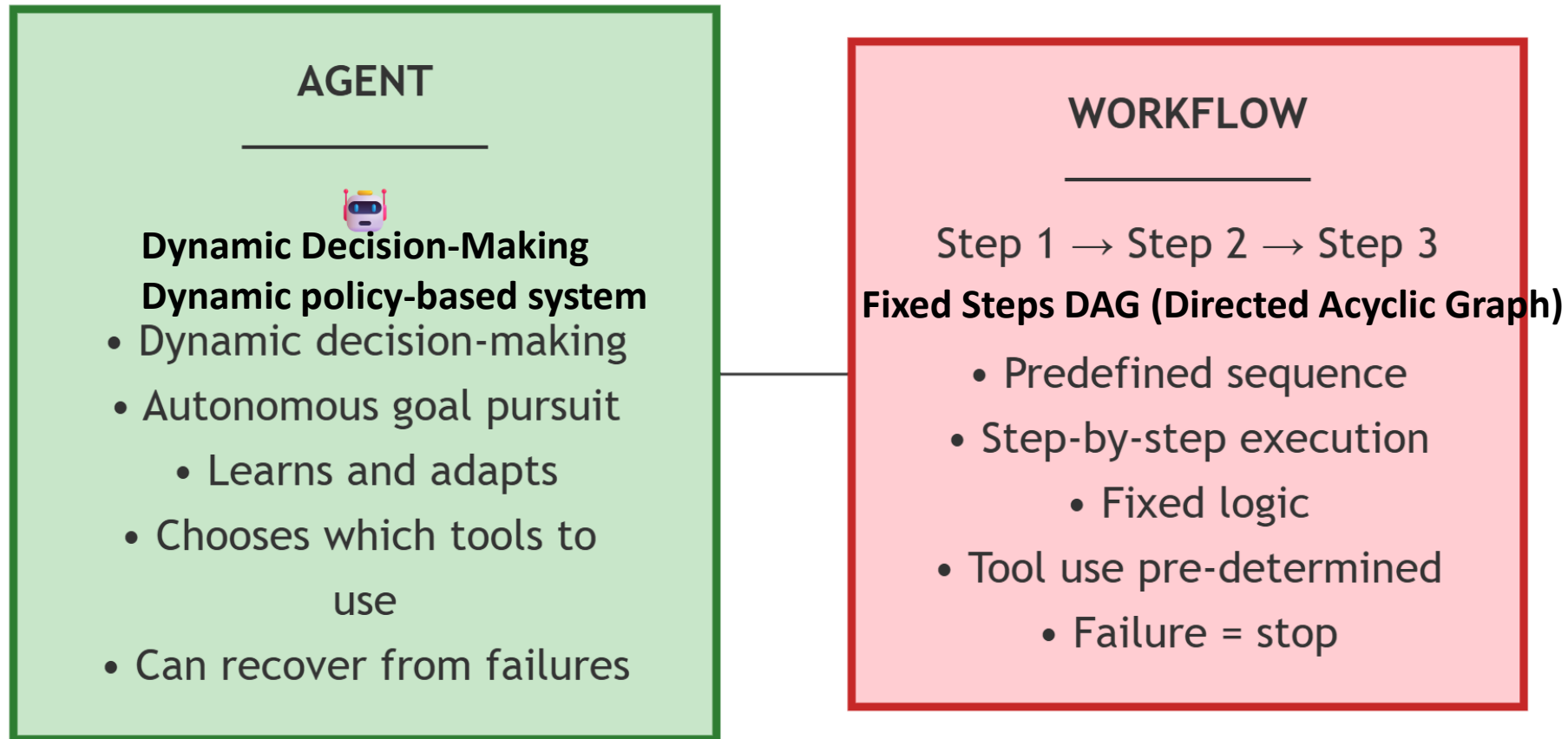
Agentic AI combines:

- LLM reasoning (planning)
- Tool usage (APIs, DBs)
- Memory (context retention)
- Feedback loops

It operates in a **closed-loop system**:



Agent vs. Workflow - The Distinction: Understanding the Difference:



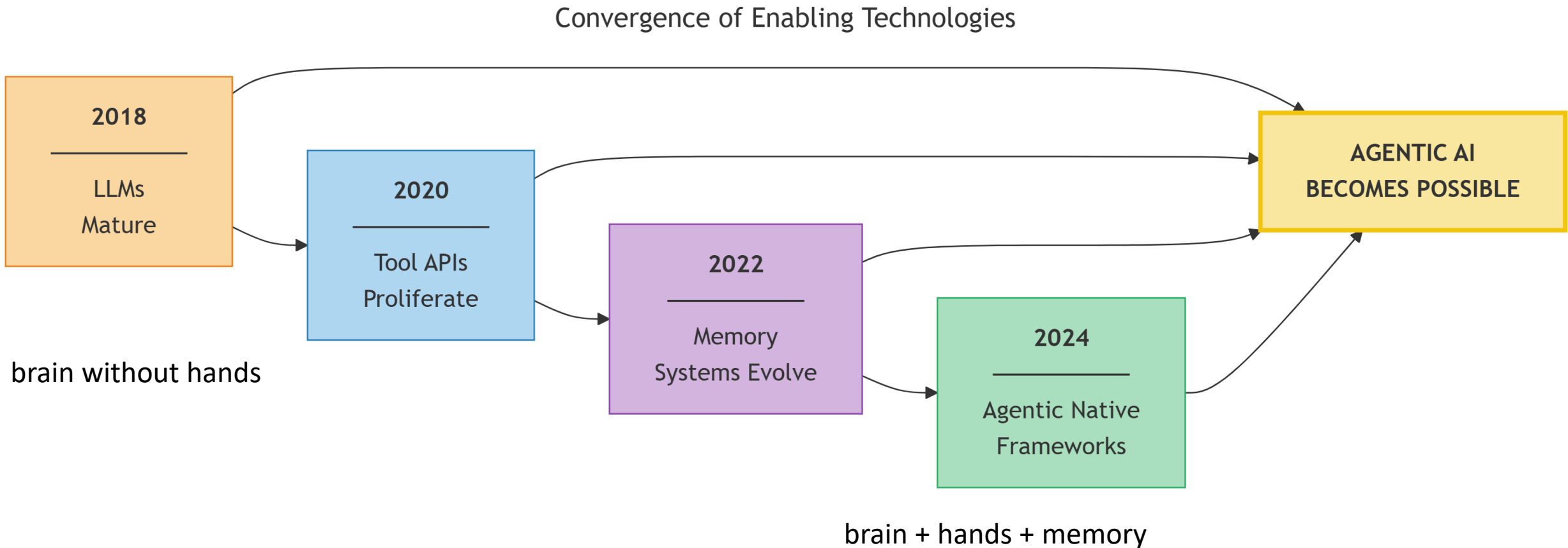
Analogy:

A **workflow** is like a **recipe** follow the steps in order to get the dish. An **agent** is like a **chef** they understand the goal (make a meal), assess available ingredients, decide on the best approach, and adapt if something goes wrong.

GenerativeAI and Its Applications

INTRODUCTION TO AGENTIC AI

Why Agentic Workflows Now?



Key Drivers:

1. **LLM Reasoning Capabilities** - Models can now plan and reason
2. **Tool Ecosystem** - Thousands of APIs available for agents to use
3. **Memory Innovations** - Long-term and working memory for agents
4. **Framework Maturity** - Production-ready agent frameworks

GenerativeAI and Its Applications

WHAT ARE AGENTIC WORKFLOWS?

Agentic Workflow - Definition

Definition:

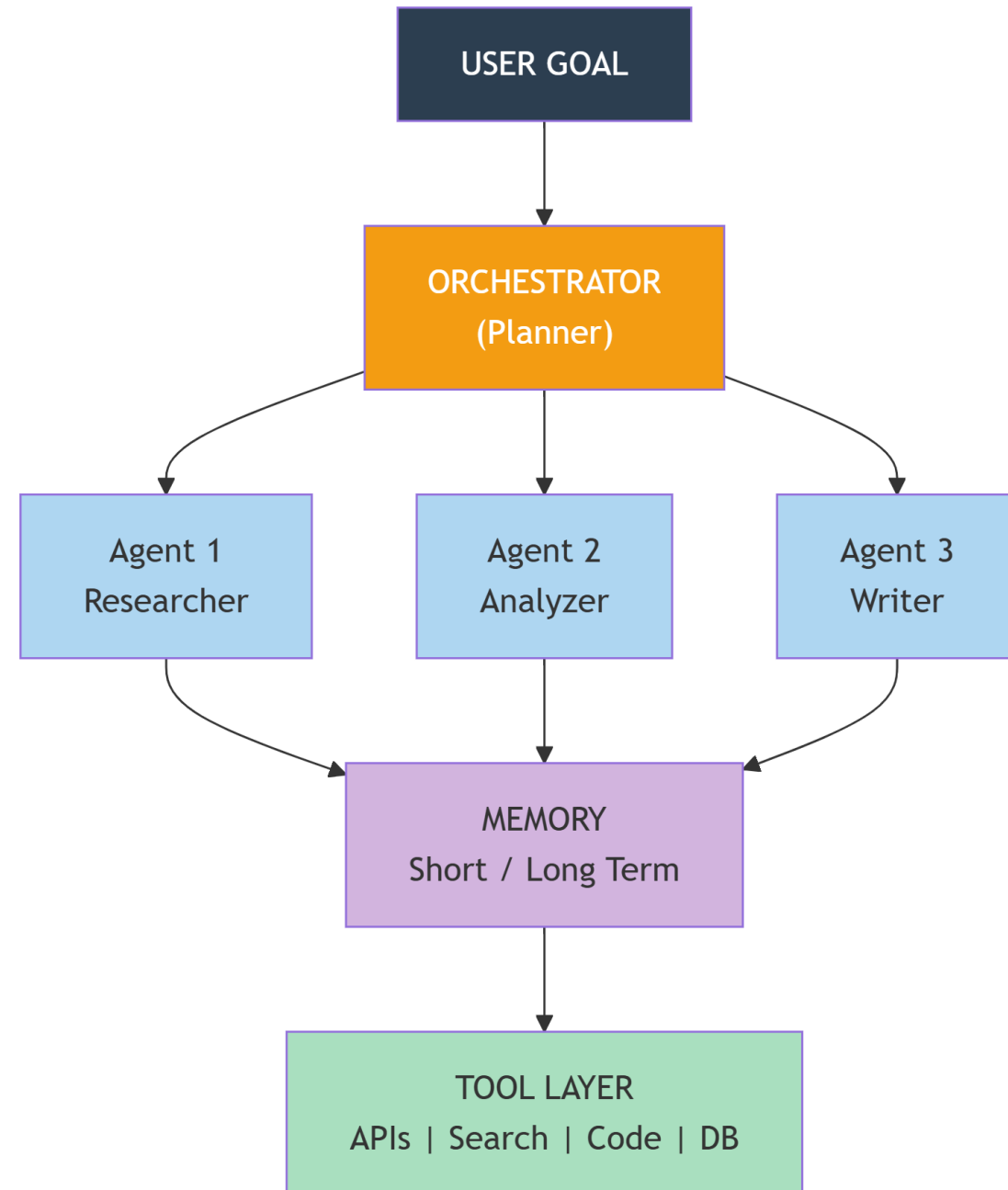
An **Agentic Workflow** is a structured system where one or more AI agents collaborate, each with specific roles, tools, and decision-making capabilities, to accomplish complex tasks through autonomous planning, execution, and adaptation.

GenerativeAI and Its Applications

WHAT ARE AGENTIC WORKFLOWS?

Core Components:

- Agent
- Tools
- Memory
- Planner
- Executor



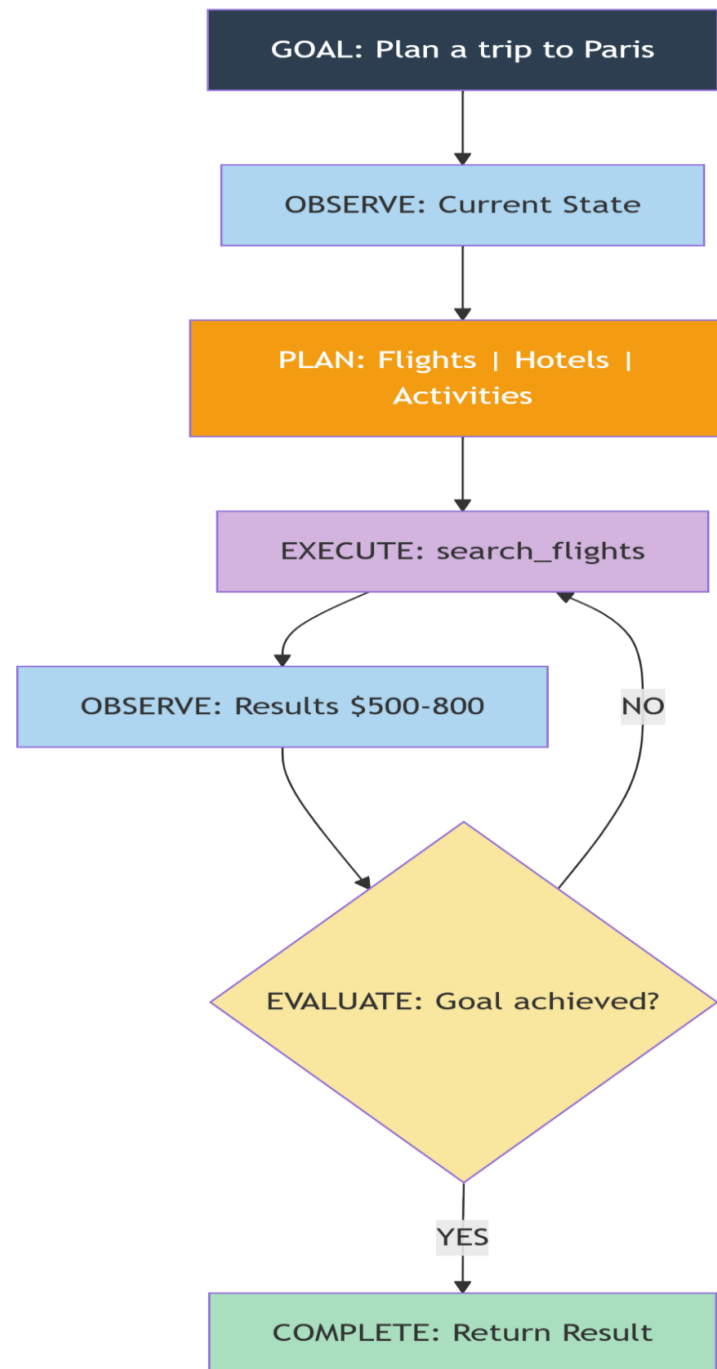
GenerativeAI and Its Applications

WHAT ARE AGENTIC WORKFLOWS?

Agentic Workflow - The Execution Loop

The Agentic Loop:

Think → Act → Observe → Repeat



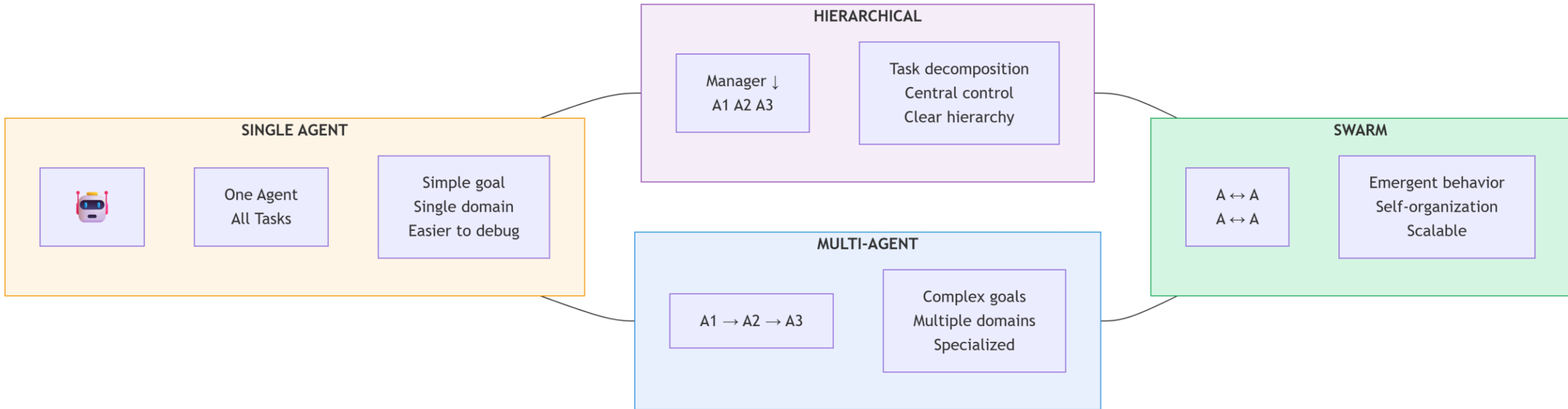
GenerativeAI and Its Applications

WHAT ARE AGENTIC WORKFLOWS?

Types of Agentic Systems

Classification by Architecture:

- Single-agent
- Multi-agent
- Hierarchical
- Distributed



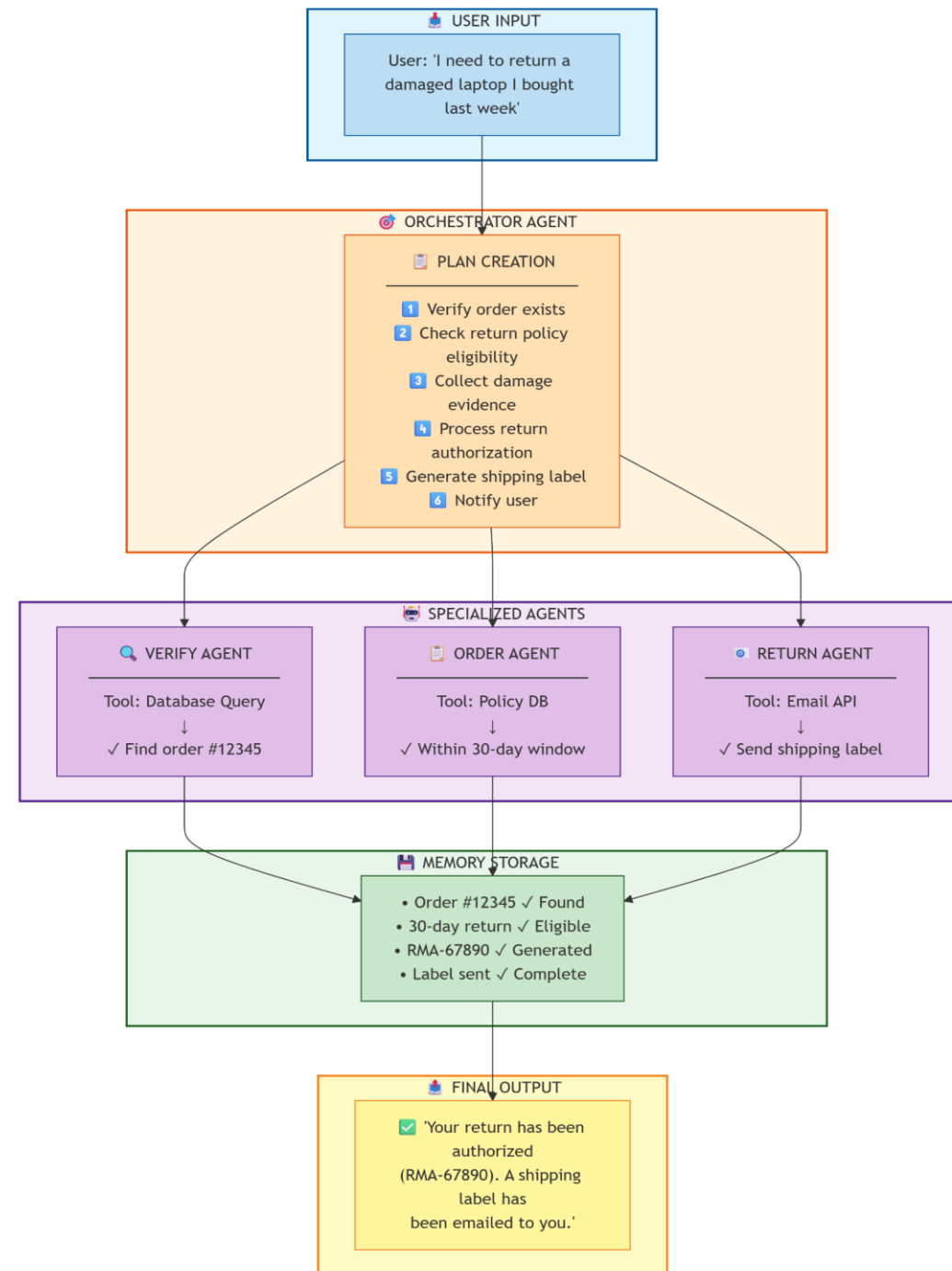
GenerativeAI and Its Applications

WHAT ARE AGENTIC WORKFLOWS?

Agentic Workflow - A Concrete Example

Customer Support Automation:

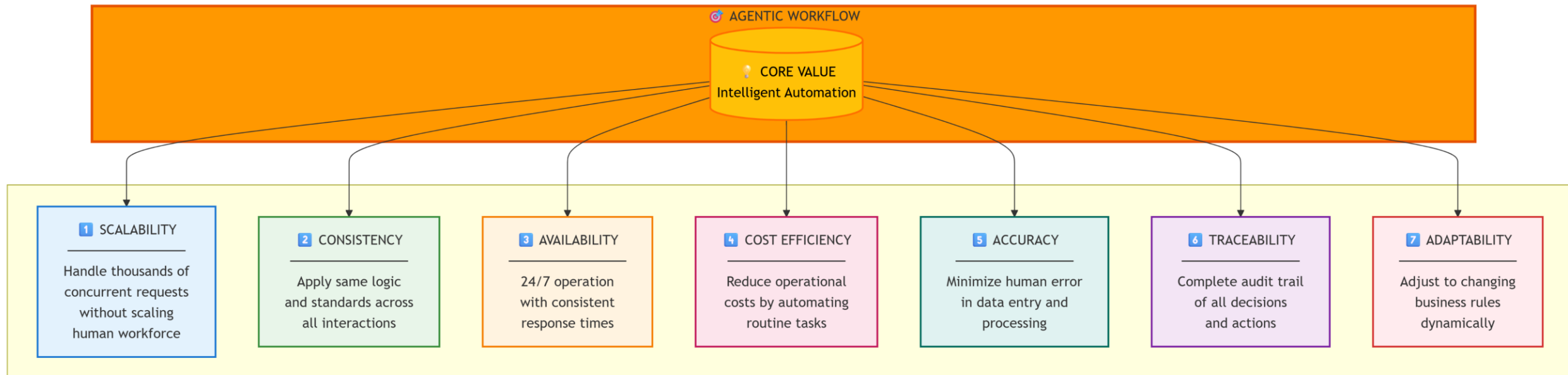
User → classifier → agent → response



WHAT ARE AGENTIC WORKFLOWS?

Why Organizations Are Adopting Agentic Workflows:

- 1. Scalability:** Handle thousands of concurrent requests without increasing human workforce
- 2. Consistency:** Apply the same logic and standards across all interactions
- 3. Availability:** Operate 24/7 with consistent response times
- 4. Cost Efficiency:** Reduce operational costs by automating routine tasks
- 5. Accuracy:** Minimize human error in data entry and processing
- 6. Traceability:** Maintain a complete audit trail of decisions and actions
- 7. Adaptability:** Quickly adjust to changing business rules and requirements



The Core Patterns (Anthropic Framework)

1. Prompt Chaining

Sequential LLM calls where the output of one step becomes the input to the next

2. Routing

Classify the input and route it to the most suitable specialized agent

3. Parallelization

Run multiple agents simultaneously and combine the results

4. Orchestrator–Worker

A central planner breaks the task into smaller subtasks and assigns them to workers

5. Evaluator–Optimizer

One agent generates output, another agent evaluates and improves it iteratively

Advanced Patterns

- ReAct (Reason + Act)
- Reflection
- Self-Critique
- Multi-Agent Debate
- Autonomous Agents (AutoGPT-style)

Core Patterns

1. Prompt chaining
2. Routing
3. Parallelization
4. Orchestrator-worker
5. Evaluator-optimizer

Pattern 1 – Prompt Chaining

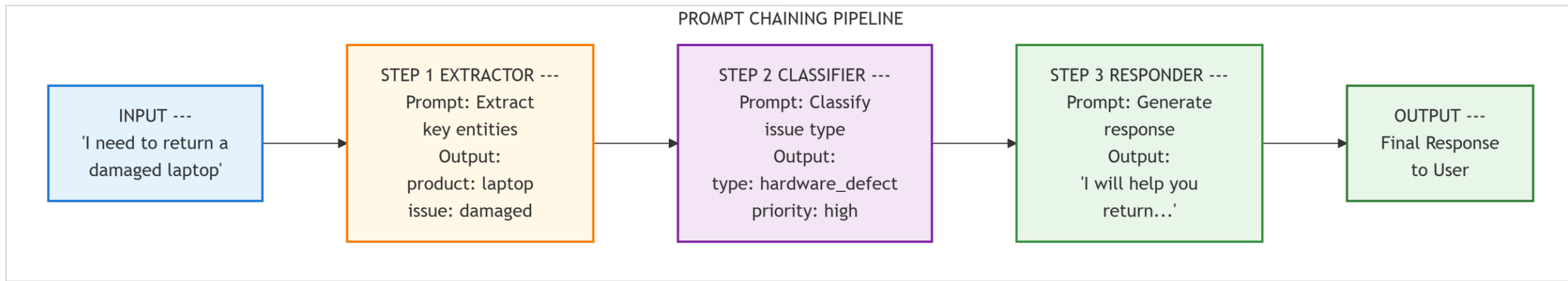
Idea: Break a complex task into smaller sequential steps

Example Flow

User Input → Entity Extractor → Classifier → Response Generator → Final Output

When to Use

- Tasks with clear sequential steps
- When intermediate validation is needed
- When each step needs a different prompt



AGENTIC DESIGN PATTERNS

Prompt Chaining – Numerical Example
Input Document

Customer John Doe (ID: C12345) reports defective laptop.
Order #ORD-7890 from 2024-01-15.
Requesting replacement.

Step 1: Entity Extraction

We multiply the confidence of each step in the prompt chain.

Step 1 Confidence (Entity Extraction)

The model predicts multiple entities and gives probabilities.

Example highest confidence outputs:

- Name → 0.92
- Customer ID → 0.88
- Order ID → 0.96
- Product → 0.85
- Issue → 0.78

To simplify, we take the **average high-confidence prediction**, which is approximately:

Step 1 confidence ≈ **0.92** “We are ~92% confident that extracted information is correct.”

Entity Type	Logit	Probability	Extracted Value
Name	8.2	0.92	John Doe
Customer ID	7.8	0.88	C12345
Order ID	9.1	0.96	ORD-7890
Product	7.5	0.85	Laptop
Issue	6.9	0.78	Defective



Step 2 Confidence (Classification)

The classifier predicts probabilities for different issue types:

- Hardware defect → 0.78
- Software issue → 0.12
- Accessory issue → 0.07
- User error → 0.03

The highest probability becomes the confidence of this step:

Step 2 confidence = **0.78** “We are 78% sure this is a hardware defect.”

Step 3 Confidence (Response Generation)

The response model generates the final answer.

Based on output probability (token likelihood / log probability) we assume:

Step 3 confidence ≈ **0.95**

Category	Logit	Probability
Hardware Defect	7.2	0.78
Software Issue	3.1	0.12
Accessory Issue	2.5	0.07
User Error	1.8	0.03

Prompt: Generate response for hardware defect

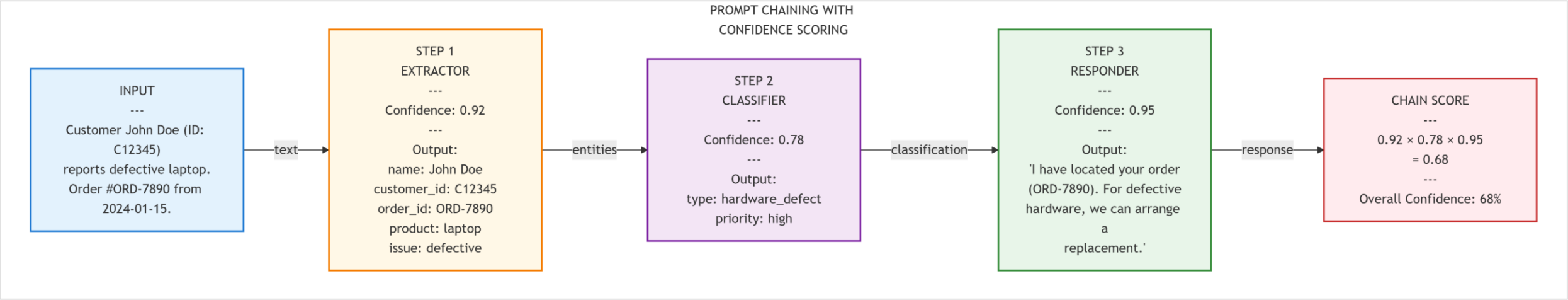
Generated Response:

"I've located your order (ORD-7890). For defective hardware, we can arrange a replacement. Would you like to proceed?"

Confidence for Step 3 ≈ 0.95 “The generated sentence is very fluent and likely correct.”

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 2 – Routing

Idea: Route input to the correct specialized agent

Example Flow

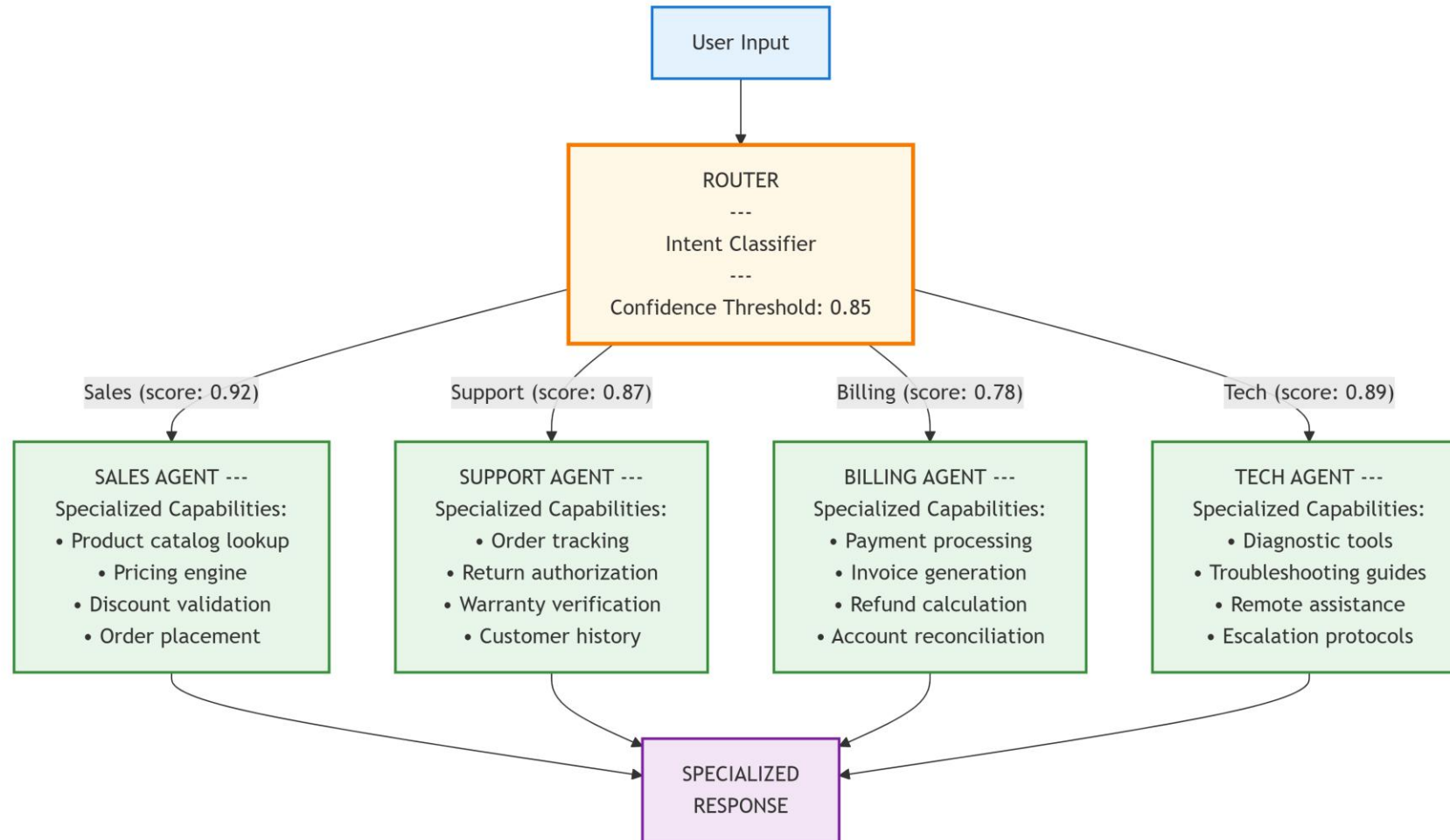
User Input → Router (Classifier) → Sales / Support / Billing / Tech Agent → Final Response

When to Use

- Multiple task types
- Different logic needed for each category
- High-accuracy task handling

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



Pattern 2 - Routing Numerical Example

Customer Service Routing:

User Input

"I need help with my internet connection. It keeps dropping."

Step 1: Router Classification (Logits → Probability)

Category	Logit	exp(logit)	Probability
Tech Support	7.2	1339	0.79
Billing	3.1	22	0.01
Sales	2.5	12	0.01
General	4.8	121	0.07
Retention	1.2	3	0.00
Account	5.3	200	0.09

Step 2: How the Probability Is Calculated (Softmax)

First calculate exponent values:

$$\exp(7.2) = 1339$$

$$\exp(3.1) = 22$$

$$\exp(2.5) = 12$$

$$\exp(4.8) = 121$$

$$\exp(1.2) = 3$$

$$\exp(5.3) = 200$$

Sum of all exponent values

$$1339 + 22 + 12 + 121 + 3 + 200 = \mathbf{1697}$$

Probability of Tech Support

$$P(\text{tech_support}) = 1339 / 1697 = \mathbf{0.79}$$

Step 3: Final Routing Decision

Input Type	Selected Agent	Confidence
Internet connection issue	Tech Support Agent	79%

Step 4: Specialized Agent Processing

Prompt given to Tech Support Agent

"You are a technical support agent. Help with: internet connection keeps dropping."

Available tools

check_modem_status()

run_speed_test()

diagnose_connection()

schedule_technician()

Generated Response

"I'll help diagnose your internet issue. Let me check your connection status first."

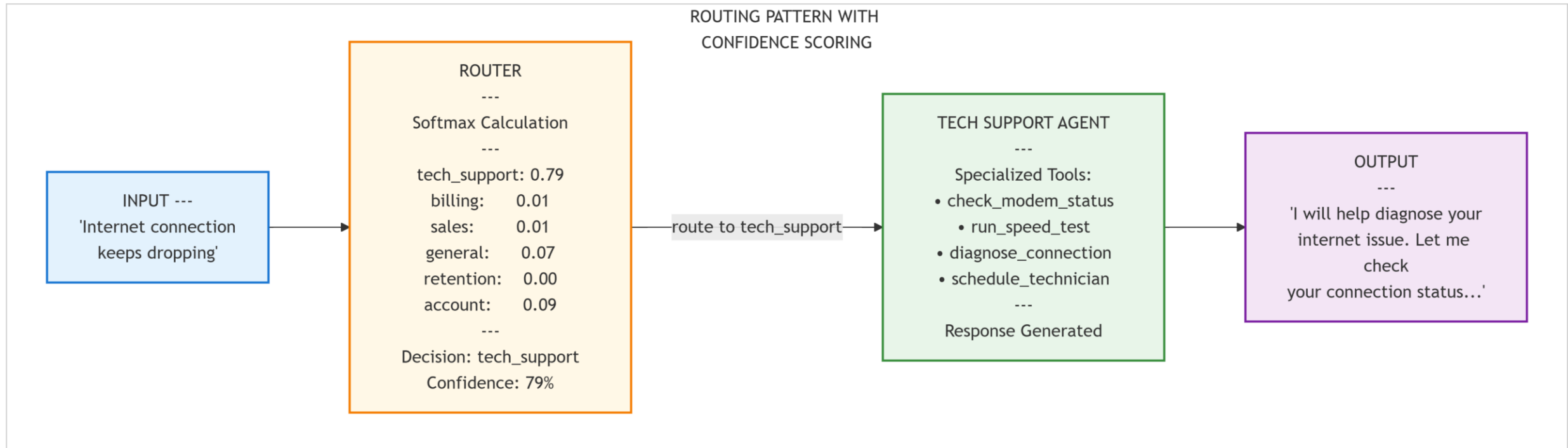
Final Concept

Router confidence = probability of selected class

Here = **0.79 (79%)**

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 3 – Parallelization

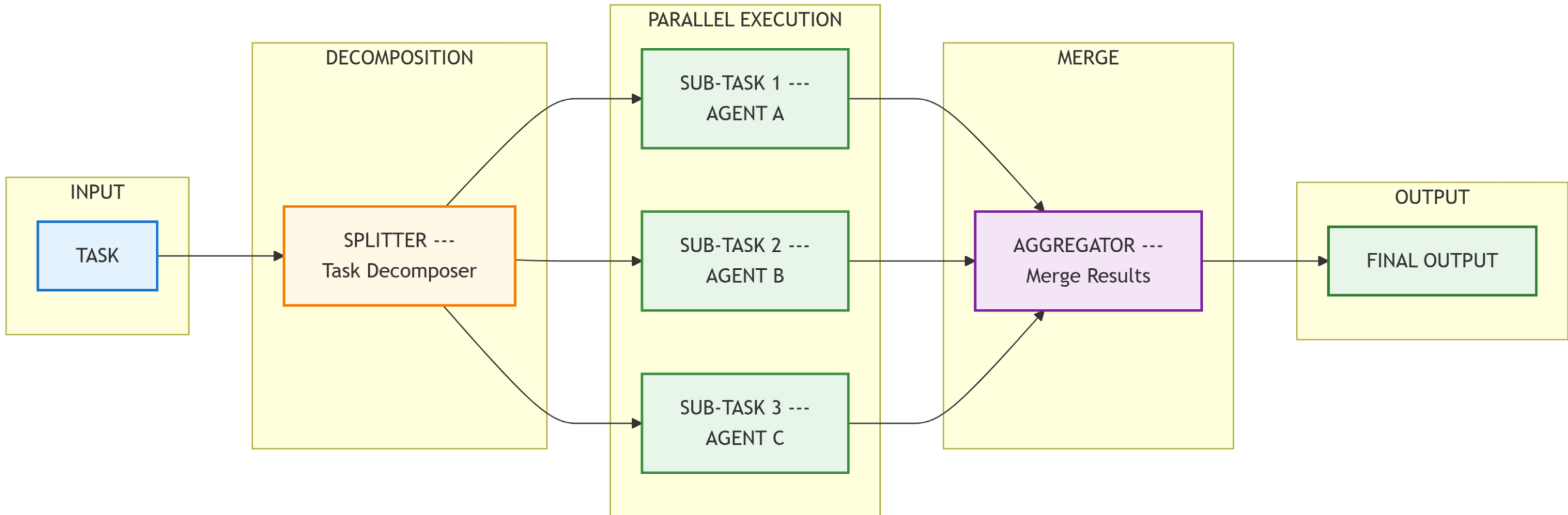
Idea: Split a task into independent subtasks and run them at the same time

Example Flow

Task → Splitter → Agent A + Agent B + Agent C (parallel) → Aggregator → Final Output

When to Use

- Research tasks
- Multi-source information gathering
- When speed is important





Parallelization (Numerical Example)

Goal

Research latest developments in quantum computing

Step 1: Task Splitting

Task No	Research Task
Task 1	Search academic papers (2024)
Task 2	Find industry news and announcements
Task 3	Analyze patent filings in quantum computing

Step 2: Parallel Agent Results

Agent Type	Results Found
Academic Agent	3 research papers
Industry Agent	5 news articles
Patent Agent	8 patents filed

Step 3: Aggregated Output

Academic (3 papers)

- Quantum error correction breakthrough
- 1000-qubit processor demonstrated
- New algorithm for quantum chemistry

Industry (5 news)

- Google announces new quantum chip
- IBM releases new quantum software
- Quantum startup raises funding

Patents (8 filings)

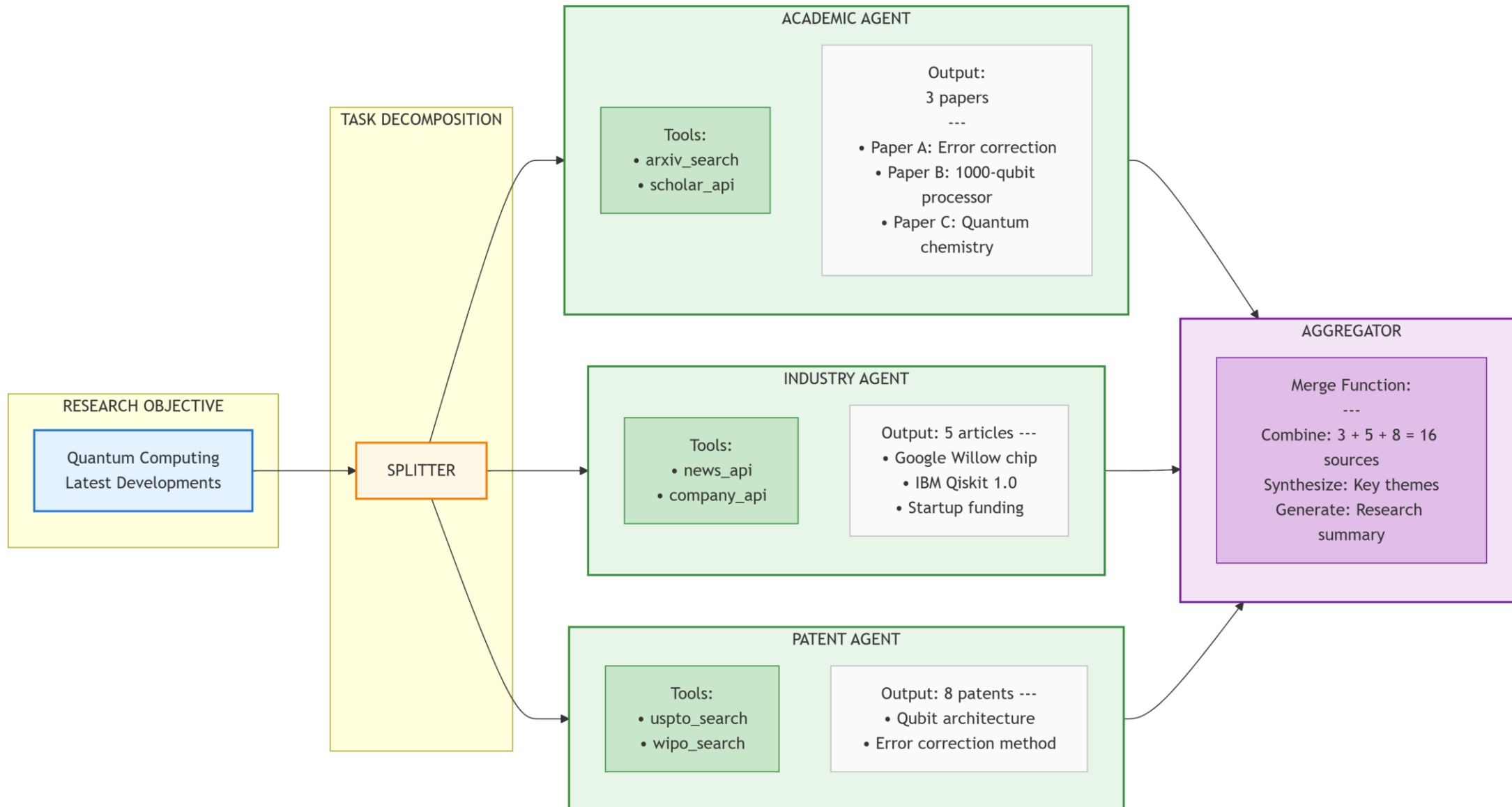
- New qubit architecture
- Error-correction methods
- Hardware optimization patents

Step 4: Performance Calculation

Speedup = $6.5 / 2.5 = 2.6\times$ faster

Execution Type	Time
Sequential Execution	6.5 seconds
Parallel Execution	2.5 seconds

GenerativeAI and Its Applications



GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 4 – Orchestrator–Worker

Idea: One main agent plans, multiple worker agents execute

Example Flow

User Goal → Orchestrator → Worker 1 + Worker 2 + Worker 3 → Orchestrator combines results

When to Use

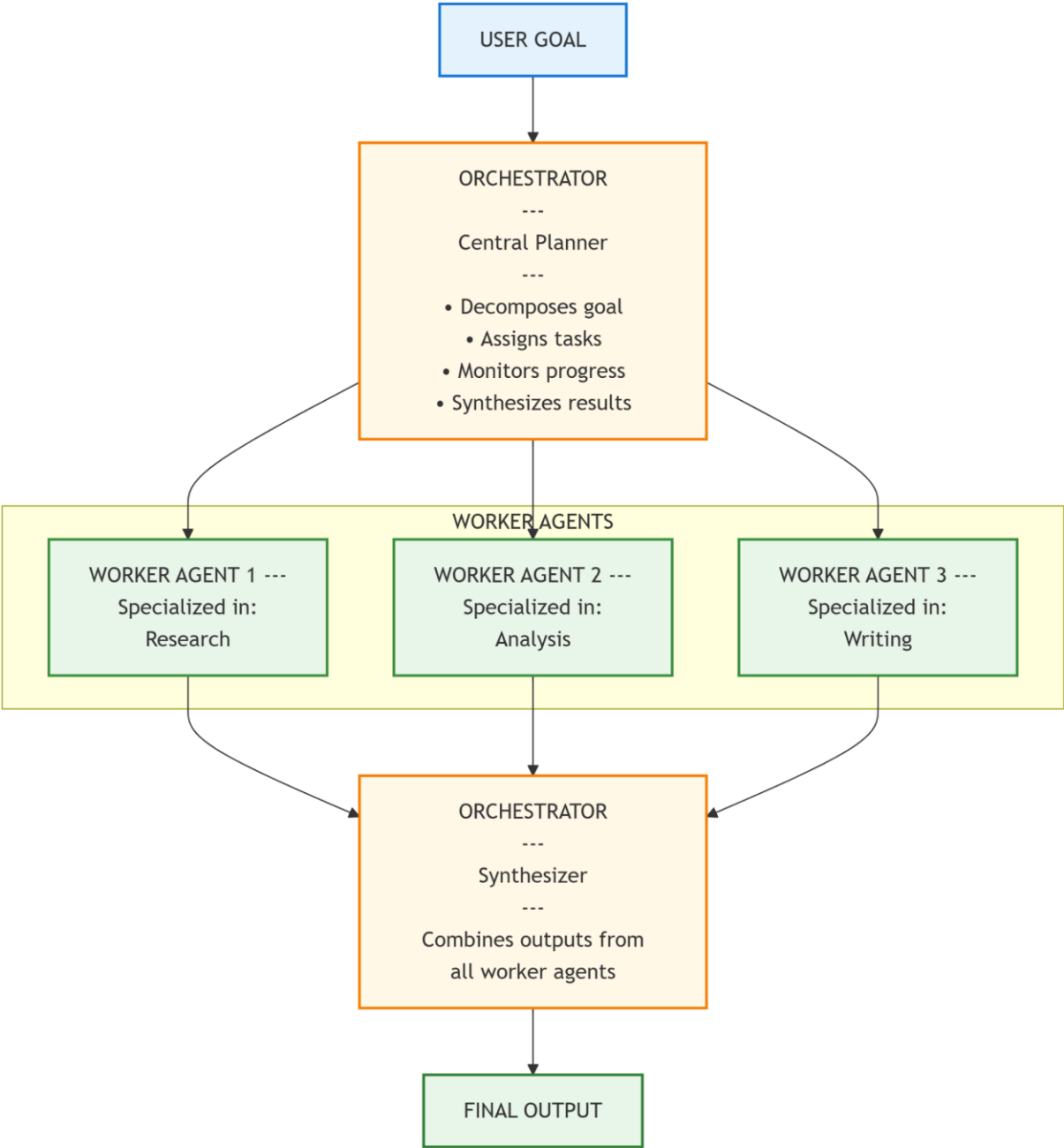
Complex workflows

Large research or analysis tasks

Multi-step automation

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 4 – Orchestrator–Worker (Numerical Example)

Goal

Create a market research report on EV battery technology

Step 1: Task Decomposition by Orchestrator

Worker	Task Assigned
Worker 1	Market size & growth analysis
Worker 2	Competitor landscape
Worker 3	Technology trends
Worker 4	Regulatory environment
Worker 5	Forecast & recommendations

Step 2: Outputs from Workers

Worker	Key Result
Worker 1	\$50B market, 18% CAGR, Asia = 45%
Worker 2	Tesla, CATL, LG Energy identified
Worker 3	Solid-state, Sodium-ion, LFP trends
Worker 4	EV regulations and policy insights
Worker 5	Market forecast and recommendations

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 4 – Orchestrator–Worker (Numerical Example)

Goal

Create a market research report on EV battery technology

Step 3: Quality Scores (Evaluated by Orchestrator)

Worker	Quality Score
Worker 1	0.92
Worker 2	0.88
Worker 3	0.95
Worker 4	0.85
Worker 5	0.78

Step 4: Final Quality Calculation

Average quality score:

$$(0.92 + 0.88 + 0.95 + 0.85 + 0.78) / 5 = \mathbf{0.91}$$

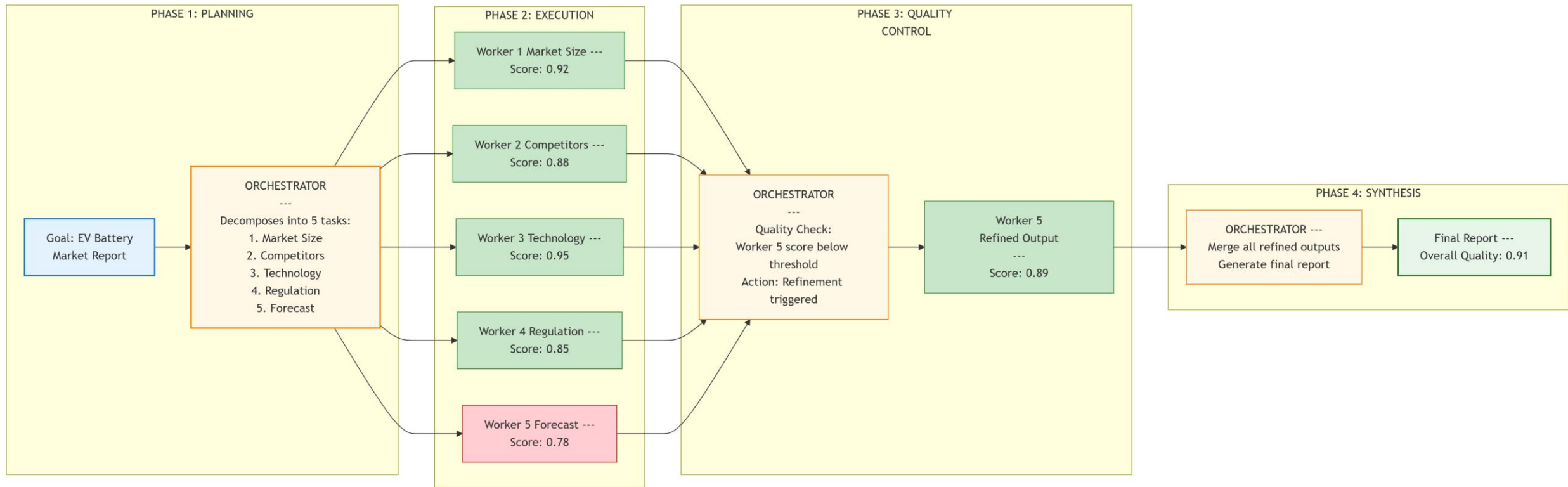
Final Result

Final Report Quality Score = 0.91

Orchestrator improves Worker 5 output before generating the final report.

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Pattern 5 – Evaluator–Optimizer

Idea: Generate → Evaluate → Improve → Repeat until good enough

Example Flow

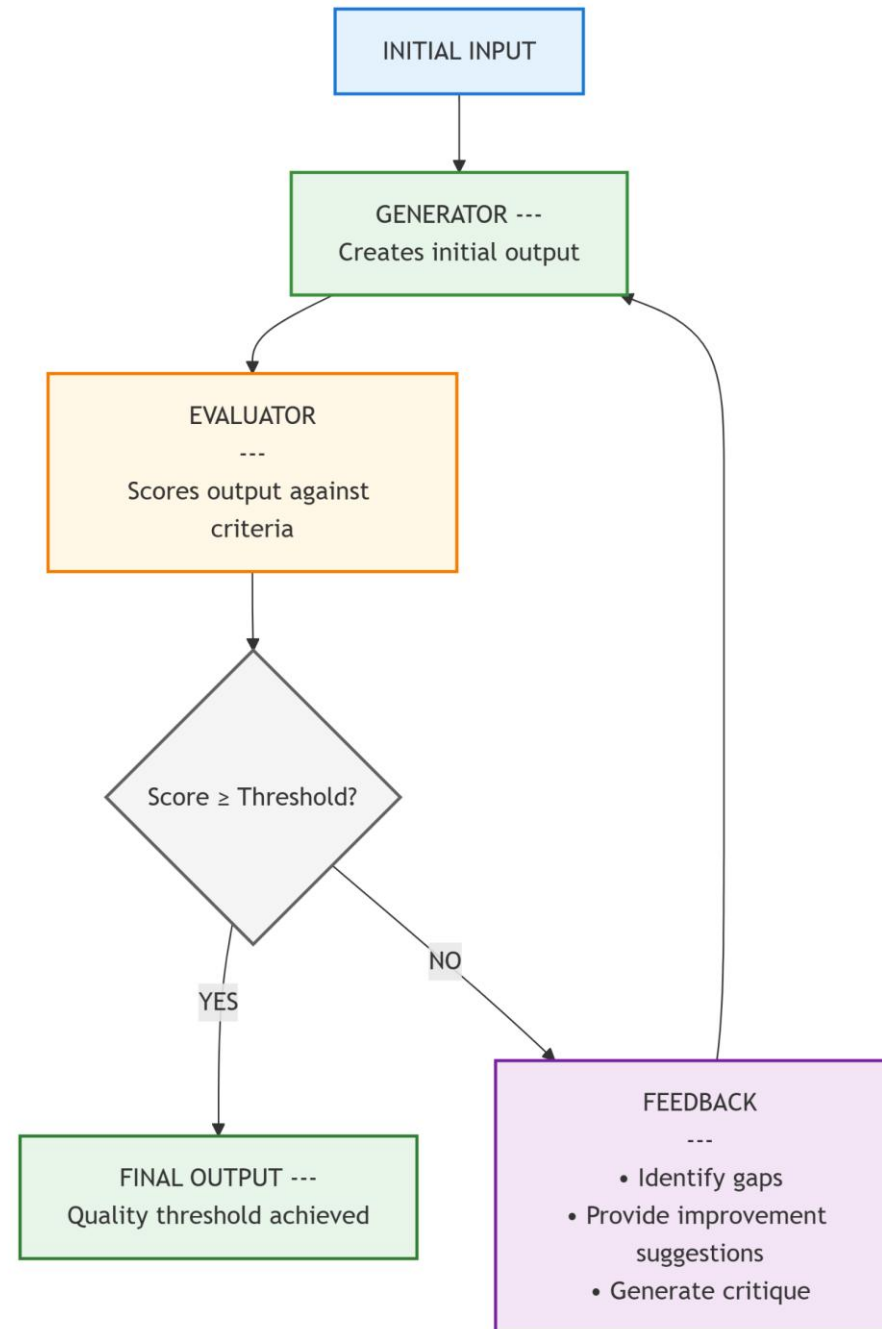
Input → Generator → Evaluator → Feedback → Generator (improved) → Final Output

When to Use

- Code generation
- Report writing
- High-quality outputs required

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS



Evaluator–Optimizer (Numerical Example)

Goal

Generate a Python function to calculate Fibonacci numbers

Step 1: First Generated Output

```
def fib(n):  
    if n <= 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```

Step 2: Evaluator Scores (First Attempt)

Evaluation Criteria	Score
Correctness	0.95
Efficiency	0.30
Readability	0.90
Edge Cases	0.85

Step 3: Overall Score Calculation

$(0.95 + 0.30 + 0.90 + 0.85) / 4 = \mathbf{0.75}$

Feedback from Evaluator

Efficiency is poor (exponential time). Improve using iteration or memoization.

Step 4: Improved Output (Second Attempt)

```
def fib(n):  
    if n <= 1:  
        return n  
    a, b = 0, 1  
    for _ in range(2, n + 1):  
        a, b = b, a + b  
    return b
```

Step 5: Evaluator Scores (Refined Output)

Evaluation Criteria	Score
Correctness	1.00
Efficiency	0.95
Readability	0.85
Edge Cases	1.00

Step 6: Final Score Calculation

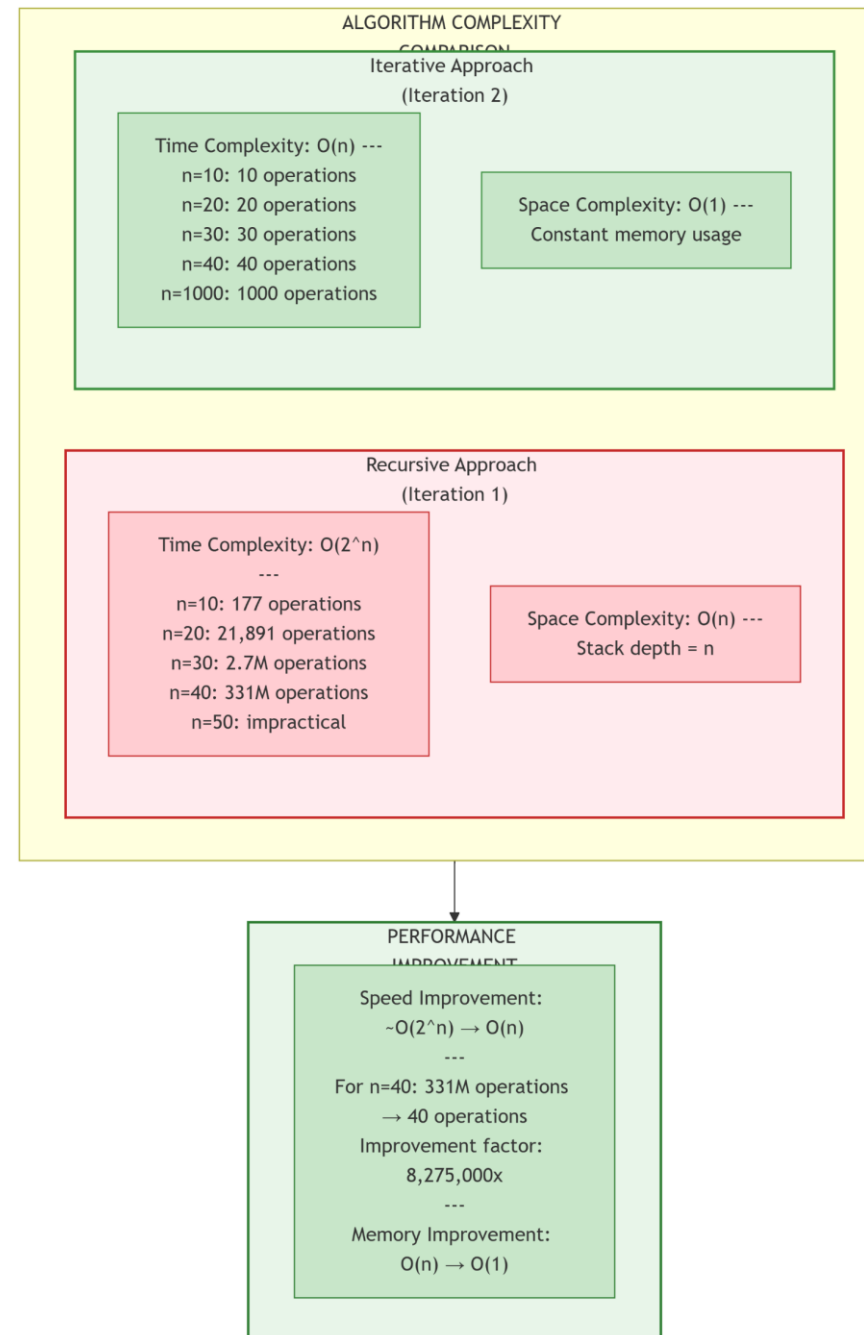
$(1.00 + 0.95 + 0.85 + 1.00) / 4 = \mathbf{0.95}$

Final Output Accepted (threshold = 0.90)

GenerativeAI and Its Applications

AGENTIC DESIGN PATTERNS

Algorithm complexity comparison:



Advanced Pattern – ReAct (Reason + Act)

What is ReAct?

ReAct = **Reasoning + Acting**

Instead of answering directly, the agent:

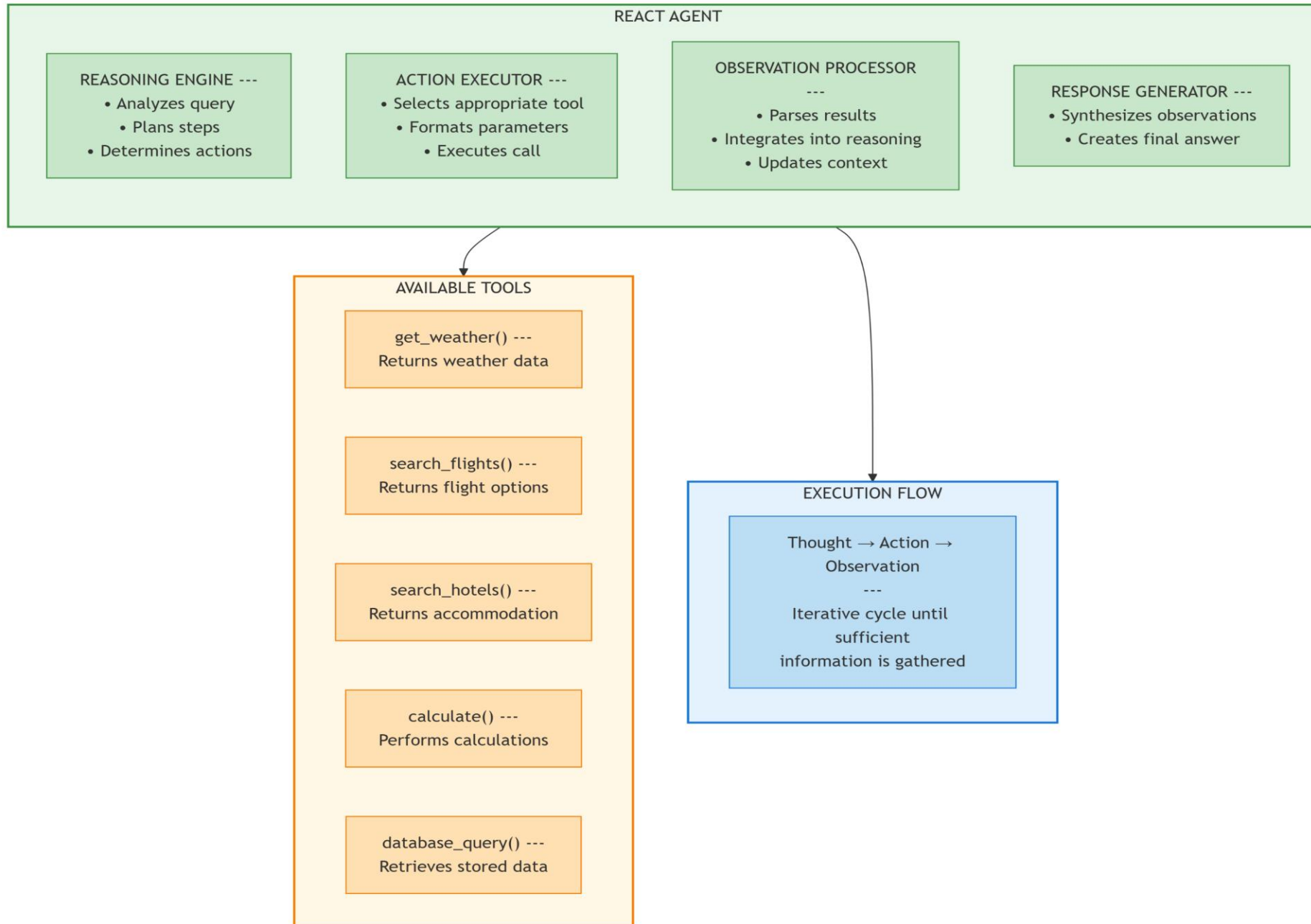
- 1.Thinks (reasoning)
- 2.Takes an action (tool call)
- 3.Observes result
- 4.Repeats until final answer

ReAct Example Flow

Step	Agent Action
Thought 1	I need to check the weather in Tokyo
Action 1	get_weather("Tokyo")
Observation 1	18°C, high humidity, rain forecast
Thought 2	Rain is expected, so umbrella is needed
Final Answer	Yes, you should bring an umbrella

Traditional vs ReAct

Traditional LLM	ReAct Agent
Prompt → Response	Thought → Action → Observation → Answer
No tool usage	Uses external tools
One-step output	Multi-step reasoning



Advanced Pattern – Reflection & Self-Critique

What is the Reflection Pattern?

The model improves its own answer by:

1. Generating an output
2. Critiquing it
3. Reflecting on mistakes
4. Generating a better version

Reflection Workflow

Step	Process
Step 1	Initial output generated
Step 2	Self-critique: What is wrong or missing?
Step 3	Reflection: How can it be improved?
Step 4	Revised output generated
Step 5	Stop if quality is good, otherwise repeat

Example

Stage	Output
Initial	Paris is a beautiful city.
Critique	Too vague. Needs more details.
Reflection	Add attractions and reasons.
Revised	Paris is famous for the Eiffel Tower, Seine River, and beautiful architecture.

GenerativeAI and Its Applications

FRAMEWORKS FOR BUILDING AGENTIC WORKFLOWS

Framework Landscape

Production-Ready Frameworks

- LangGraph
- LangChain
- LlamaIndex
- CrewAI
- Semantic Kernel
- Haystack

Research / Experimental Frameworks

- AutoGen
- BabyAGI
- AutoGPT
- MetaGPT
- CAMEL

Low-Code Platforms

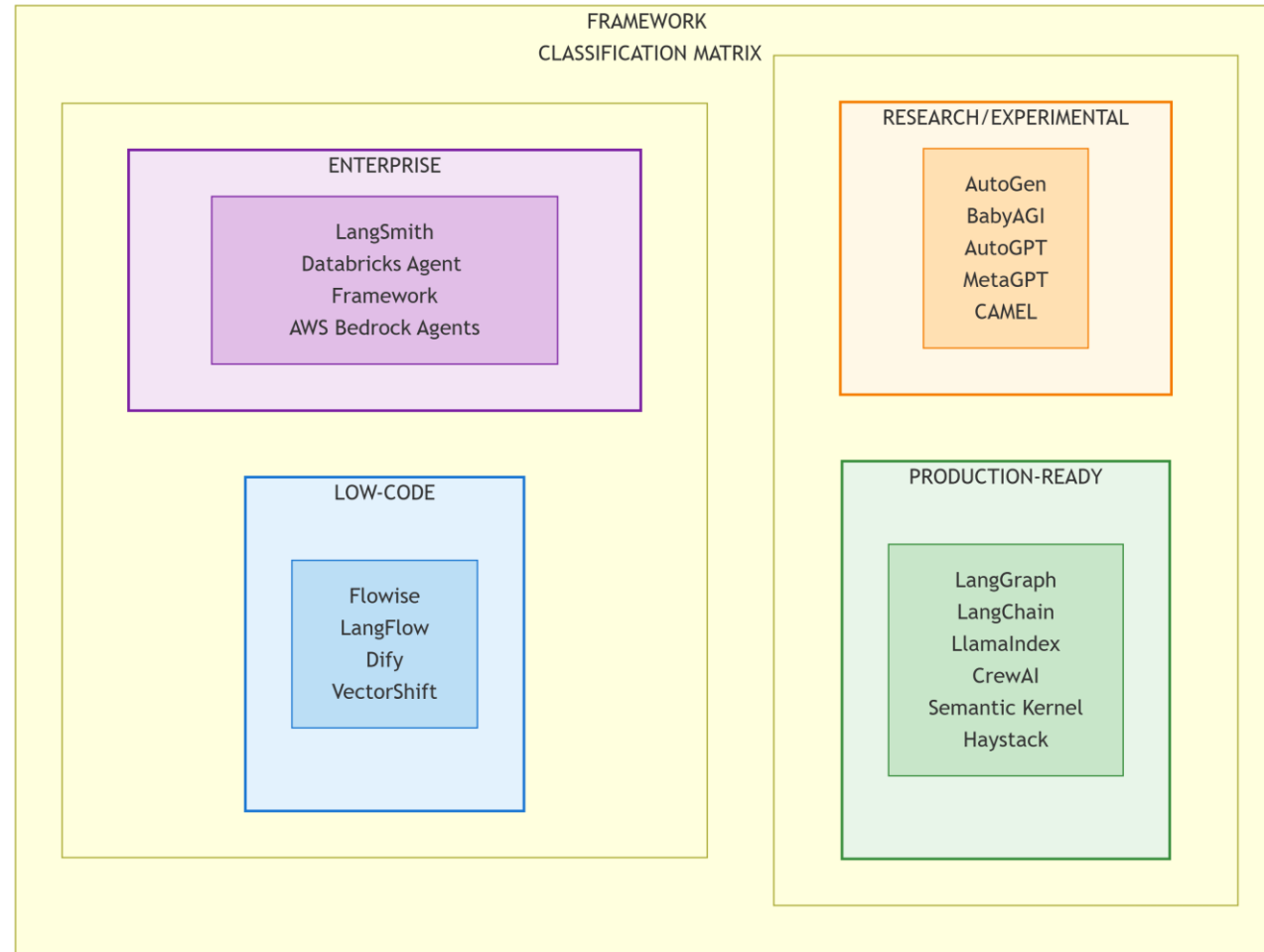
- Flowise
- LangFlow
- Dify
- VectorShift

Enterprise Platforms

- LangSmith
- Databricks Agent Framework

GenerativeAI and Its Applications

FRAMEWORKS FOR BUILDING AGENTIC WORKFLOWS



Framework Comparison Matrix

Framework	Key Feature	Agent Patterns Supported	Ease of Use	Production Ready
LangGraph	Graph-based state machine	All patterns	4/5	Yes
LangChain	Modular components	Chaining, Routing	5/5	Yes
CrewAI	Role-based agents	Orchestrator-Worker	4/5	Partial
AutoGen	Multi-agent conversations	All patterns	3/5	Partial
LlamaIndex	RAG + Agents	ReAct, Routing	4/5	Yes
Semantic Kernel	Microsoft planning framework	Planner, Routing	3/5	Yes

LangGraph Architecture

Key Concepts

Nodes

- Agent nodes (LLM decision makers)
- Tool nodes (function execution)
- Human-in-the-loop nodes
- Conditional nodes (branching logic)

Edges

- Normal edges → fixed flow
- Conditional edges → dynamic routing
- Start / End nodes

State

- Shared memory between nodes
- Stores messages, data, decisions
- Persistent across execution

LangGraph Example Workflow

Flow Example

Start → Router → Research Agent → Analysis Agent → Writer Agent → Final Output

Use Cases

- Multi-step research agents
- Multi-tool workflows
- Complex automation pipelines

CrewAI Architecture

Core Idea

CrewAI = **Team of role-based agents**

Agent	Role	Goal
Agent 1	Researcher	Find reliable sources
Agent 2	Analyst	Extract insights
Agent 3	Writer	Generate final report

How It Works

- 1.Tasks are created
- 2.Tasks assigned to agents
- 3.Agents execute sequentially or hierarchically
- 4.Final result is generated

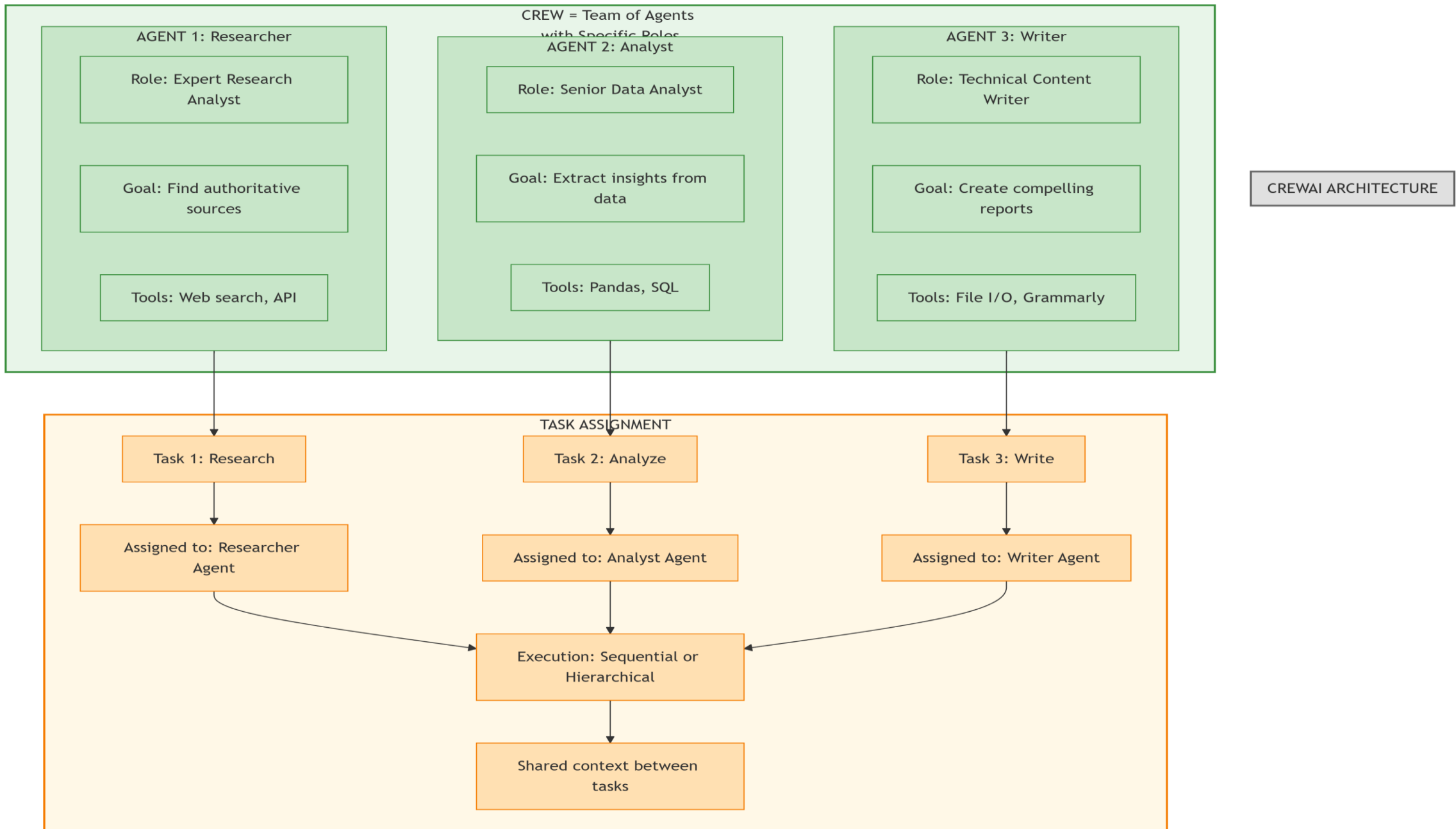
CrewAI Workflow Example

Example Task

Research latest trends in quantum computing

Workflow

Researcher Agent → Analyst Agent → Writer Agent → Final Report



GenerativeAI and Its Applications

FRAMEWORKS FOR BUILDING AGENTIC WORKFLOWS

AutoGen Architecture

Key Idea

AutoGen uses **multi-agent conversations**

Main Agents

- User Proxy Agent (represents the human)
- Assistant Agent (LLM)
- Tool Agent (executes functions)

Workflow

User → Assistant → Tool → Assistant → Final Response

GenerativeAI and Its Applications

FRAMEWORKS FOR BUILDING AGENTIC WORKFLOWS



INTERACTIONS

User ↔ User Proxy

User Proxy ↔ Assistant

Assistant ↔ Tool Agent

Tool Agent → User Proxy

CONVERSATIONAL AGENTS

USER PROXY AGENT ---

- Represents human
- Executes code
- Collects user input
- Validates outputs

ASSISTANT AGENT ---

- LLM-powered reasoning
- Proposes solutions
- Generates code
- Coordinates actions

TOOL AGENT ---

- Executes functions
- Runs API calls
- Performs searches
- Returns results

Messages

Function Calls

Execution Results

If You Need	Best Framework
Production deployment	LangGraph or LangChain
Multi-agent conversations	AutoGen
RAG-based agents	LlamaIndex
Role-based teams	CrewAI
Microsoft ecosystem	Semantic Kernel

Best Practices for Building Agents

- Start with a single agent first
- Define clear roles for each agent
- Add guardrails (timeouts, max iterations)
- Log every step (thought, action, output)
- Test edge cases carefully
- Add human-in-the-loop when needed
- Cache repeated responses
- Monitor cost, accuracy, and latency

Common Pitfalls & Solutions



Pitfall	Problem	Solution
Infinite loops	Agent repeats same step	Set max iterations
Tool overuse	Too many API calls	Add rate limits
Hallucination	Agent invents results	Use structured outputs
Context overflow	Model forgets instructions	Use summarization
Noisy feedback	Poor outputs	Add validation rules



THANK YOU

Generative AI and Its Applications



Unit 4

(UE23CS342BA4)

LLM, LLM Ethics & Security, and Recent Developments in LLM

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

GenerativeAI and Its Applications

Acknowledgement



The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

GenerativeAI and Its Applications

LLM Evaluation Techniques



Why Evaluation is Important

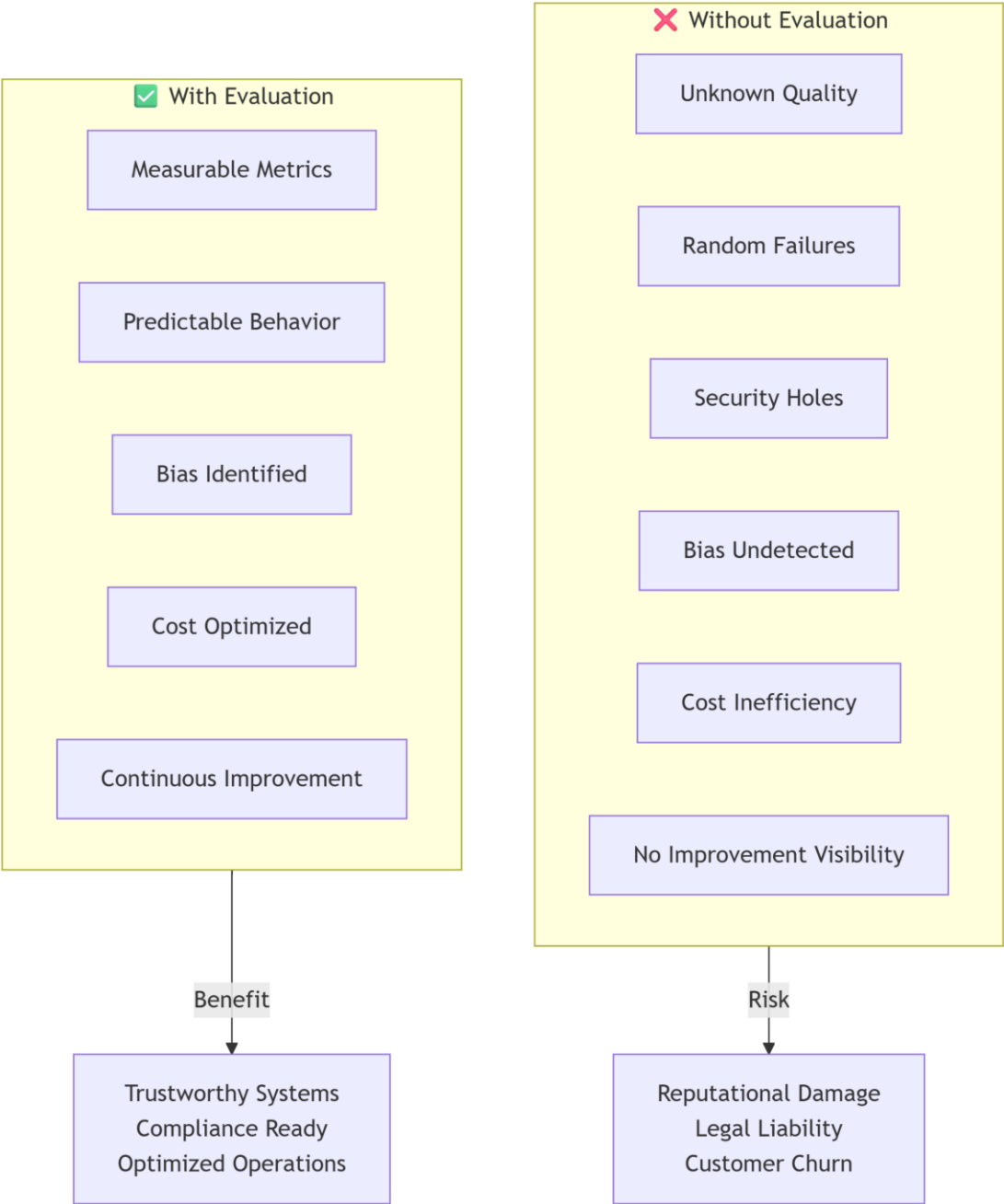
Large Language Models (LLMs) generate responses based on probability rather than factual certainty. Because of this, evaluation becomes essential to ensure:

- Accuracy of generated responses
- Reliability in real-world applications
- Detection of hallucinations
- Fairness and bias control
- Safety and ethical usage
- Performance comparison between different models

Without evaluation, it is impossible to know whether an LLM is trustworthy or suitable for deployment.

GenerativeAI and Its Applications

LLM Evaluation Techniques



GenerativeAI and Its Applications

LLM Evaluation Techniques



Types of LLM Evaluation

1. Intrinsic Evaluation

Intrinsic evaluation measures the internal performance of the language model.

Examples:

- Perplexity
- Log loss
- Token probability

These metrics measure how well a model predicts the next word in a sentence.

GenerativeAI and Its Applications

LLM Evaluation Techniques

Types of LLM Evaluation

2. Extrinsic Evaluation

Extrinsic evaluation measures how well the model performs in real-world tasks.

Examples:

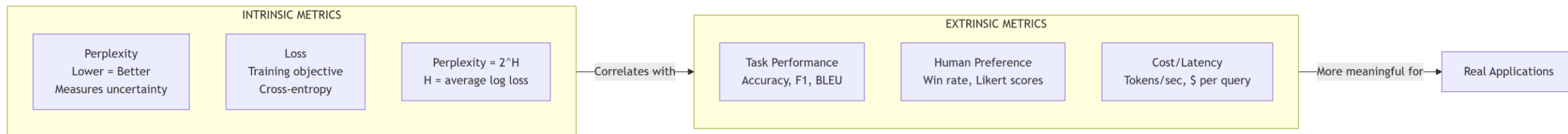
Question answering accuracy

Summarization quality

Translation performance

Chatbot response quality

This type of evaluation is more practical because it reflects real user usage.



GenerativeAI and Its Applications

LLM Evaluation Techniques

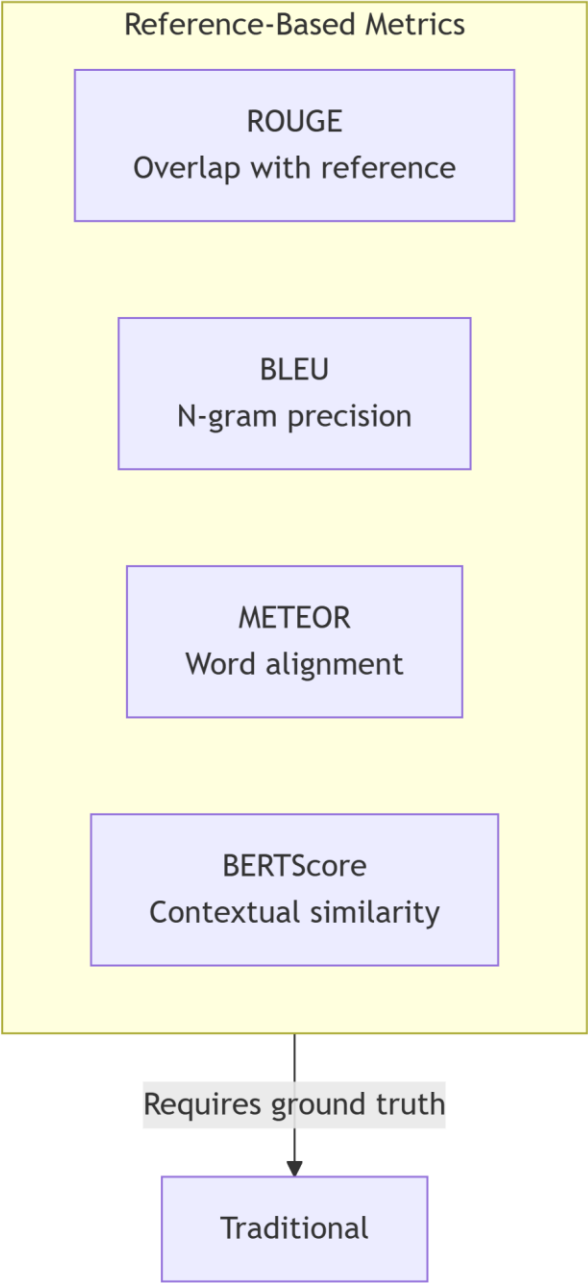
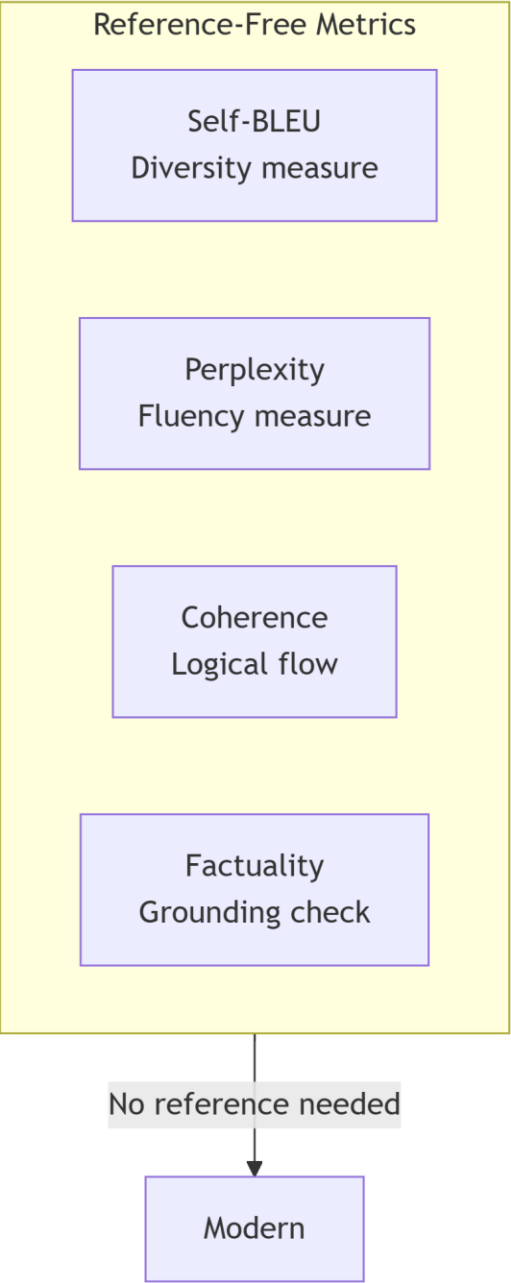
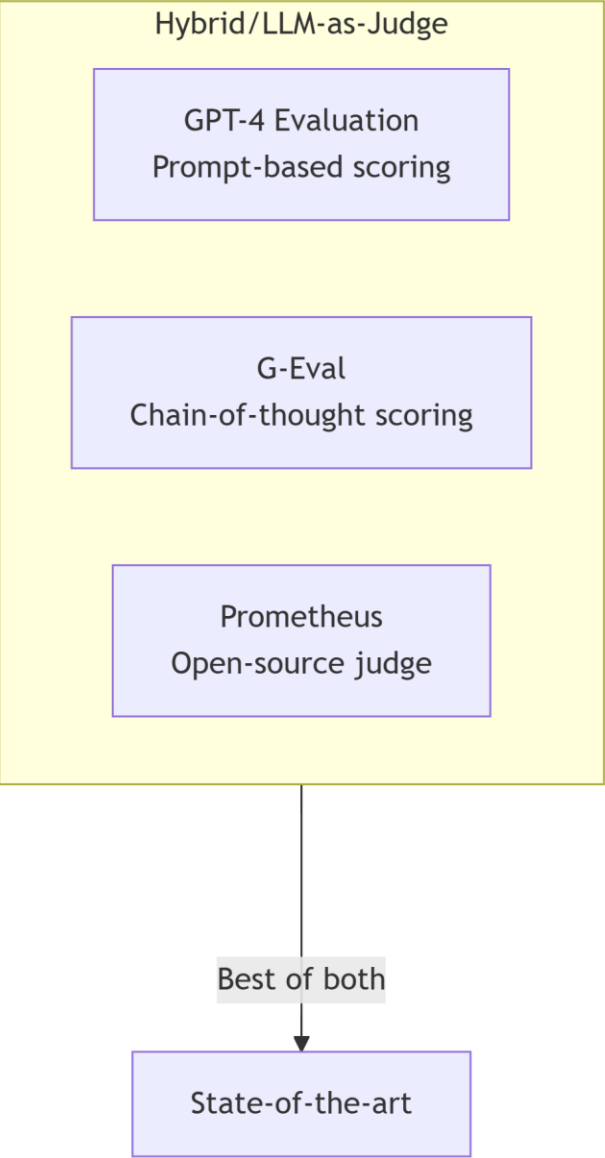
Perplexity Numerical Example:

Model A: Average log loss = 1.2 $\rightarrow$ Perplexity = $2^{1.2} = 2.30$

Model B: Average log loss = 2.5 $\rightarrow$ Perplexity = $2^{2.5} = 5.66$

Lower perplexity = better model (more confident predictions)

Common Text Generation Metrics



GenerativeAI and Its Applications

LLM Evaluation Techniques



Common Text Generation Metrics

BERTScore

BERTScore compares sentences using contextual embeddings rather than exact words.

Advantages:

- Understands meaning, not just words
- Handles synonyms
- Works well for paraphrased sentences
- More aligned with human judgment

BERTScore Numerical Example:

Reference: "The cat sat on the mat"

Candidate: "A feline rested on the rug"

BERTScore Precision = 0.89

BERTScore Recall = 0.92

BERTScore F1 = 0.90

(Matches semantic meaning despite different words!)

GenerativeAI and Its Applications

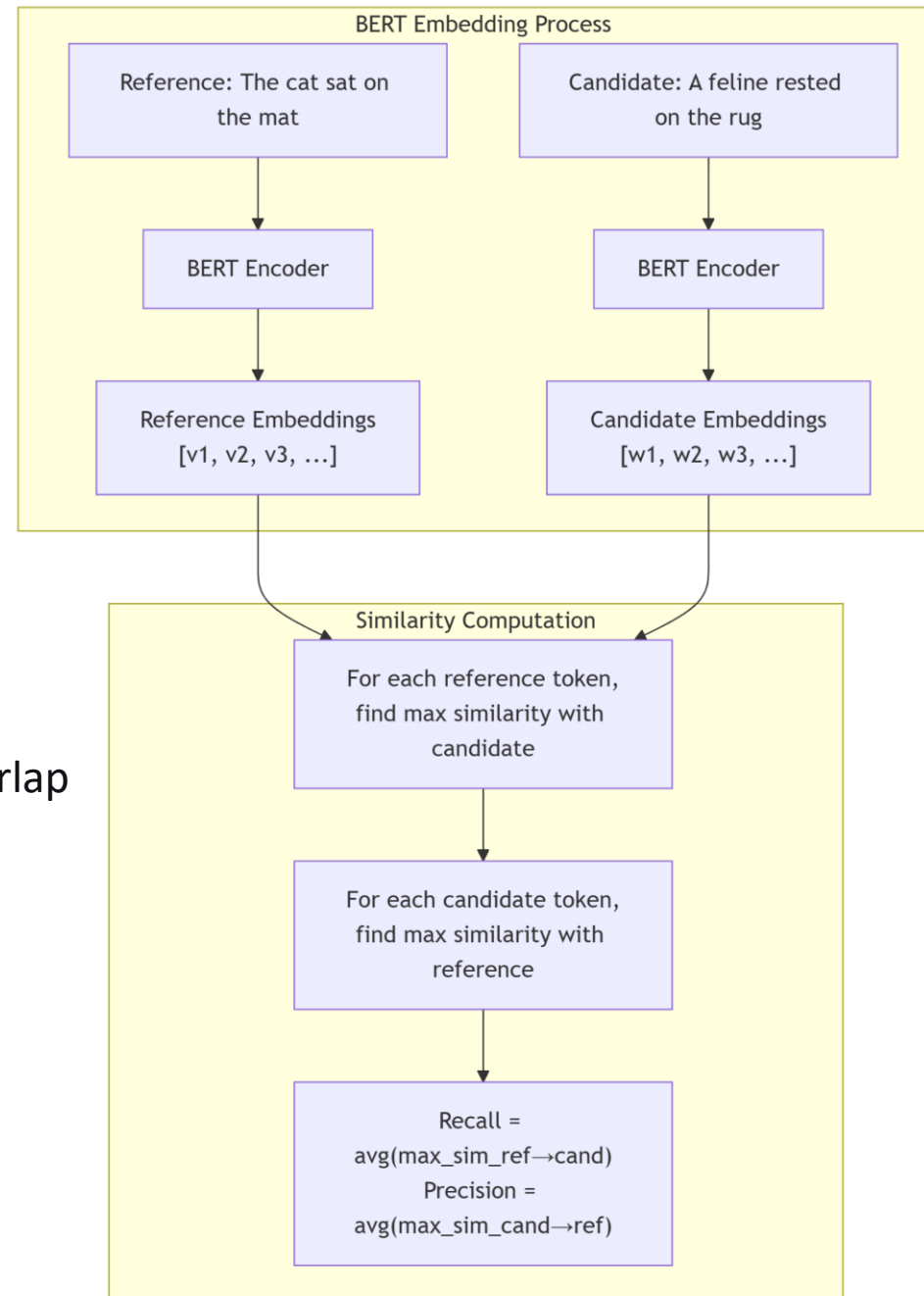
LLM Evaluation Techniques

Common Text Generation Metrics

BERTScore - Contextual Embedding Similarity:

BERTScore Advantages:

- Captures **semantic similarity**, not just lexical overlap
- Handles **synonyms** and **paraphrases**
- Works across **languages** (multilingual BERT)
- Correlates better with **human judgment**



GenerativeAI and Its Applications

LLM Evaluation Techniques

Common Text Generation Metrics

ROUGE

ROUGE measures how similar the generated text is to a reference text.

It is mainly used in:

- Text summarization
- Answer generation
- Content comparison

ROUGE Score Interpretation:

Score	Meaning
0.9-1.0	Excellent match (near-identical)
0.7-0.9	Good match (captures key content)
0.5-0.7	Moderate match (some content overlap)
<0.5	Poor match (significant divergence)

GenerativeAI and Its Applications

LLM Evaluation Techniques

ROUGE Metrics Explained

ROUGE-S (Skip-Bigram)

Allows non-adjacent pairs

Matches: ('cat','mat') even
with words between

ROUGE-L (Longest Common Subsequence)

Reference: 'A B C D E'

Candidate: 'A C B D E'

LCS = 'A B D E' (length 4)

$F1 = 4/5 = 0.80$

ROUGE-N (N-gram Overlap)

Reference: 'The cat sat'

Candidate: 'The feline sat'

ROUGE-1: 'the','cat','sat' vs
'the','feline','sat'

Recall = $2/3 = 0.67$
Precision = $2/3 = 0.67$
F1 = 0.67

GenerativeAI and Its Applications

LLM Evaluation Techniques

Common Text Generation Metrics

LLM-as-a-Judge Evaluation

In this method, a powerful LLM evaluates the response of another LLM.

It checks:

- Correctness
- Completeness
- Coherence
- Helpfulness
- Safety

This method is widely used because it scales better than human evaluation.

GenerativeAI and Its Applications

LLM Evaluation Techniques

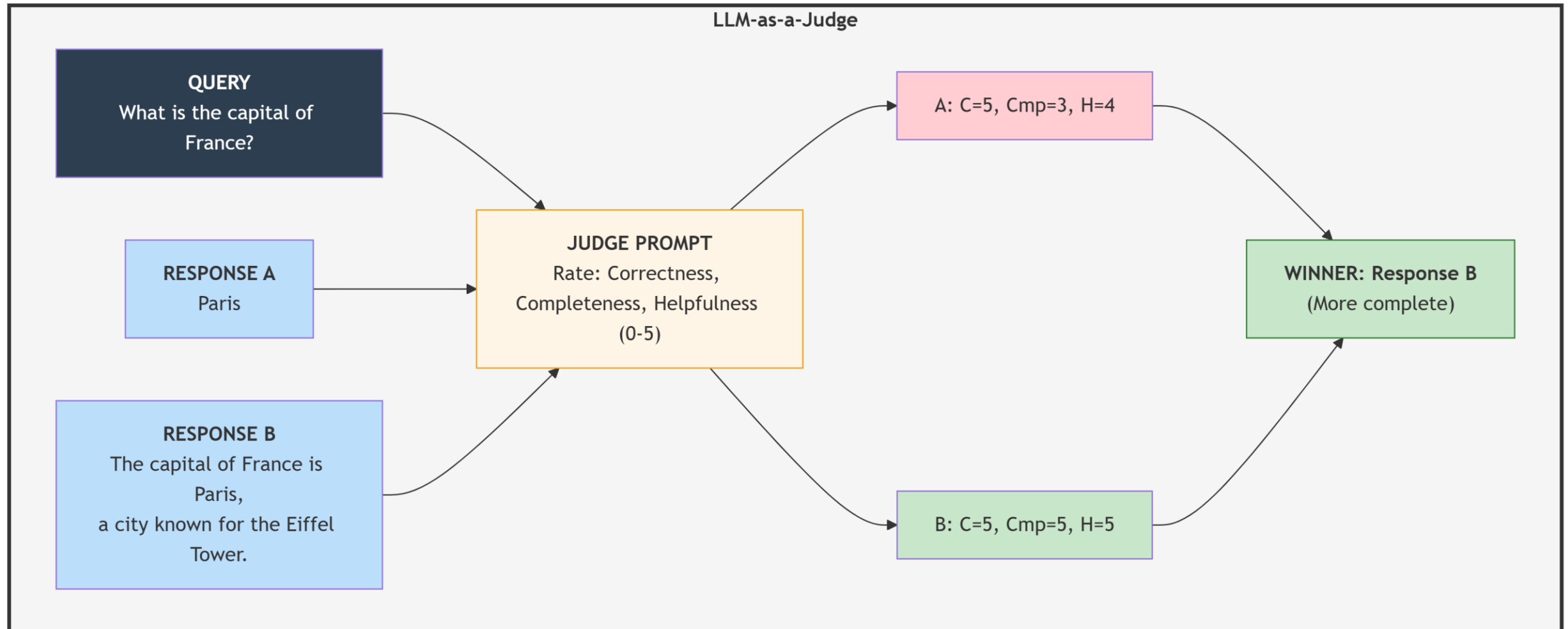


LLM Judge Metrics:

Metric	Description	Score Range
Correctness	Factual accuracy	0-5
Completeness	Coverage of query	0-5
Helpfulness	Practical value	0-5
Coherence	Logical flow	0-5
Harmlessness	Safety/ethics	0-5

GenerativeAI and Its Applications

LLM Evaluation Techniques



GenerativeAI and Its Applications

LLM Evaluation Techniques



Common Text Generation Metrics

Hallucination Detection

A hallucination occurs when an LLM generates incorrect information that sounds correct.

Examples:

- Wrong historical dates
- Fake research papers
- Incorrect statistics
- Fabricated facts

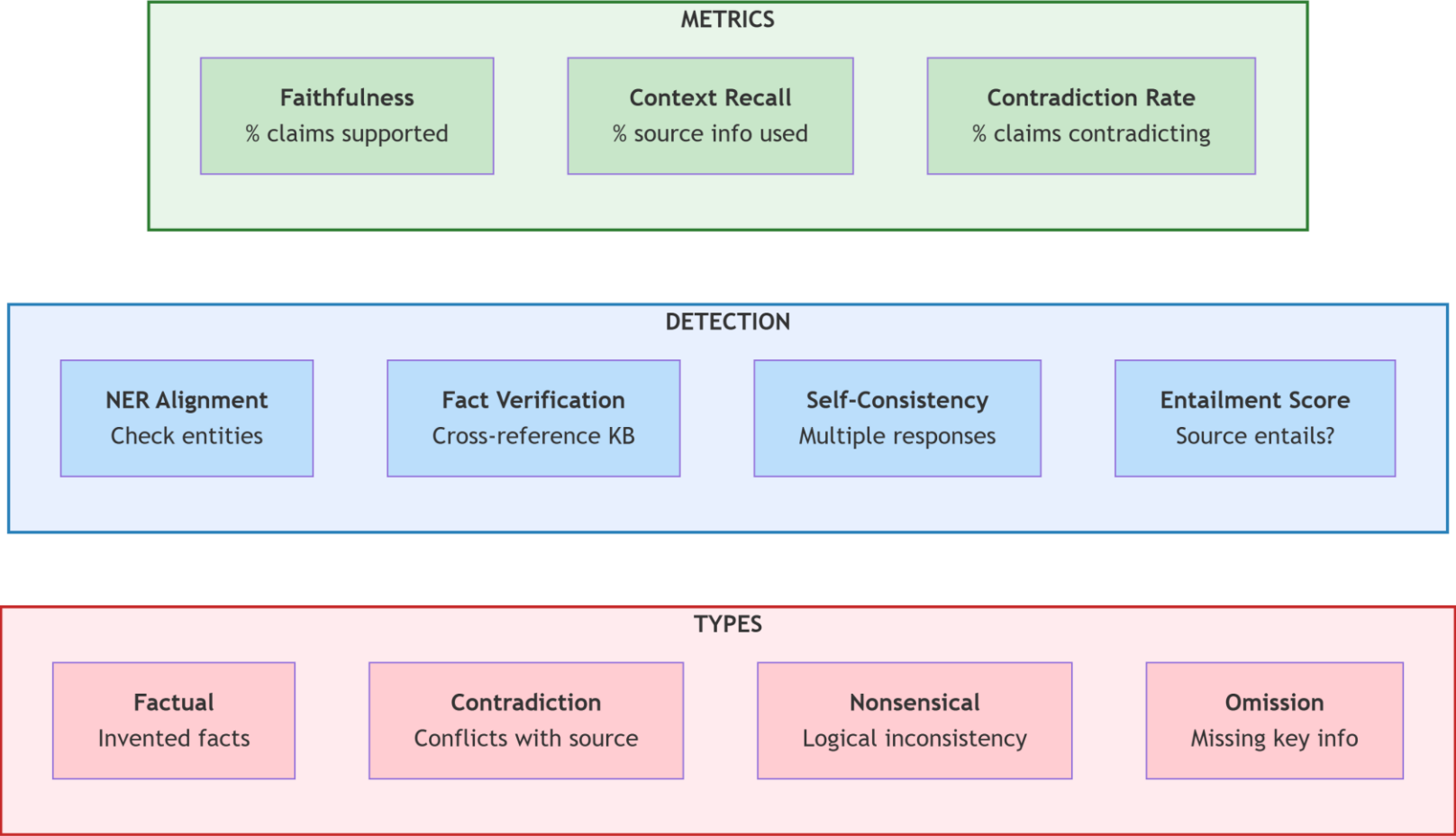
Hallucination detection is extremely important in domains such as:

1. Healthcare
2. Finance
3. Legal systems
4. Education

GenerativeAI and Its Applications

LLM Evaluation Techniques

Hallucination in RAG Systems



GenerativeAI and Its Applications

LLM Evaluation Techniques



Hallucination Numerical Example:

Source: "The Eiffel Tower is in Paris, France."

Response: "The Eiffel Tower is in Paris, which is the capital of Italy."

Claim 1: "Eiffel Tower in Paris" → Supported

Claim 2: "Paris is capital of Italy" → Contradiction (Paris is capital of France)

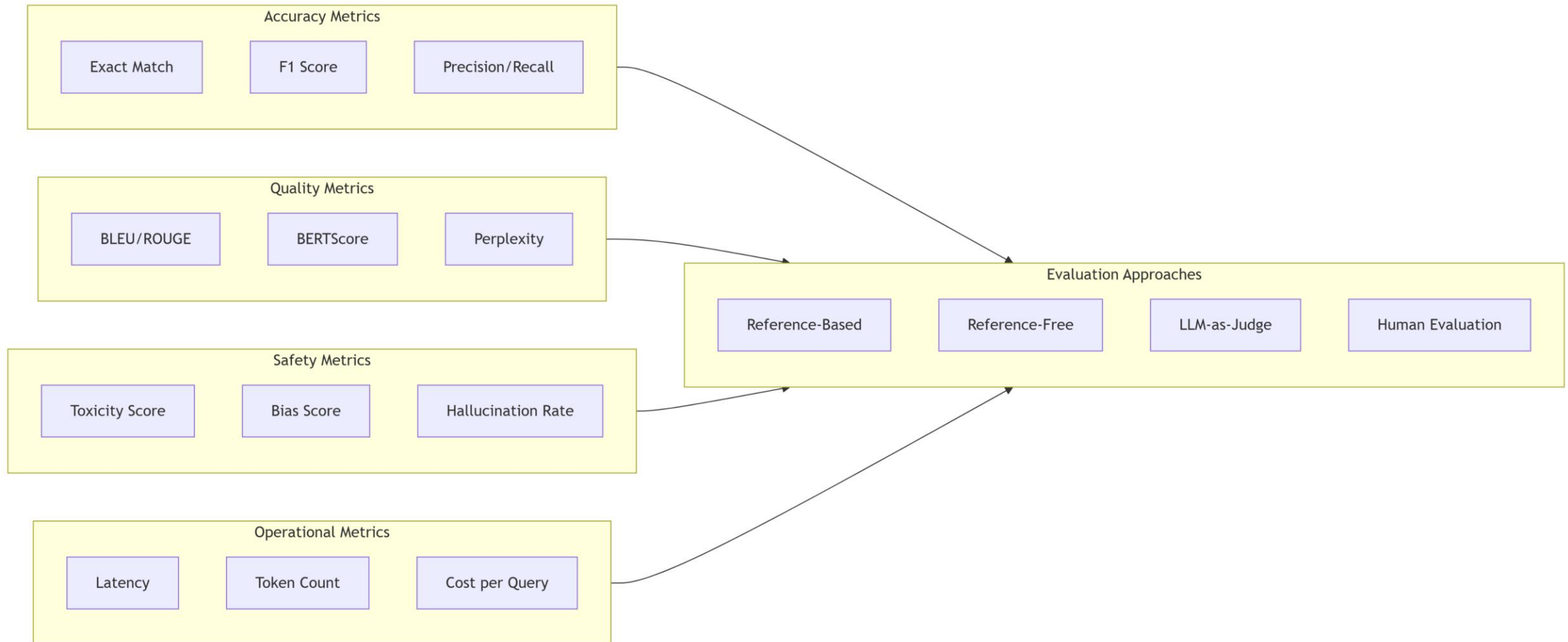
Faithfulness = $1/2 = 0.50$ (50% of claims supported)

Contradiction Rate = $1/2 = 0.50$ (50% of claims false)

GenerativeAI and Its Applications

LLM Evaluation Techniques

Evaluation Metrics Summary



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



What is DeepEval?

DeepEval is an open-source evaluation framework designed specifically for testing LLM applications.

It works similarly to unit testing in software development.

Features of DeepEval

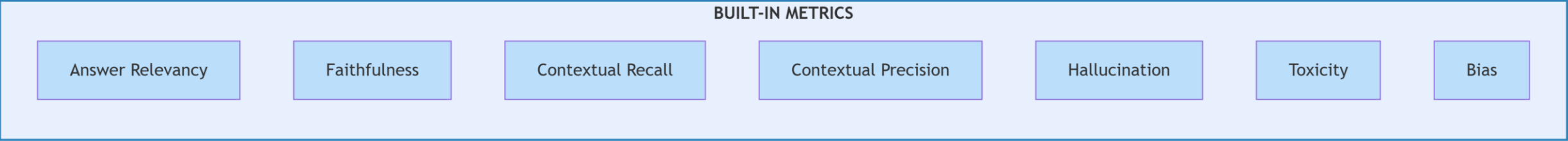
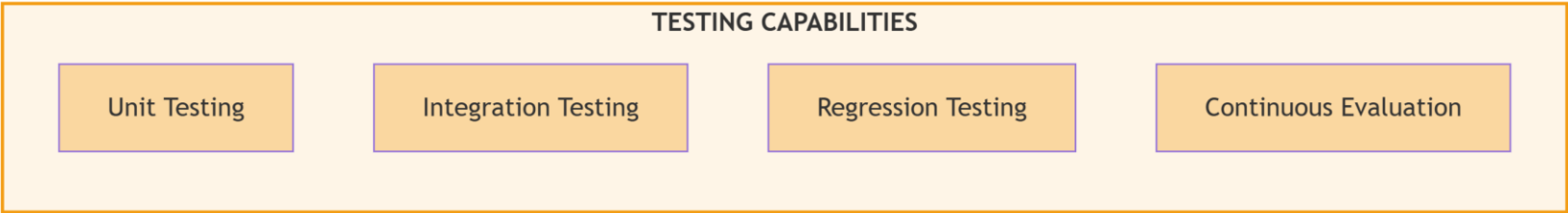
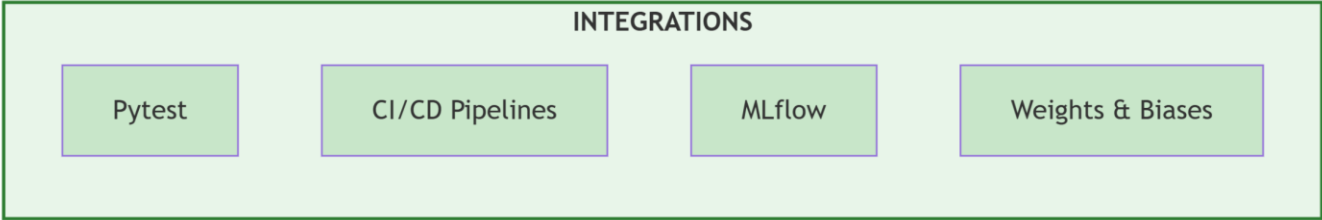
- Automated LLM evaluation
- Pre-built evaluation metrics
- Support for RAG-based applications
- Integration with Python and PyTest
- Synthetic test case generation
- Easy evaluation of hallucinations and faithfulness

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



DeepEval Framework



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



Important DeepEval Metrics

Faithfulness

Checks whether the generated answer is supported by the given context.

Answer Relevancy

Measures whether the response actually answers the question.

Contextual Precision

Checks whether the correct information from the context was used.

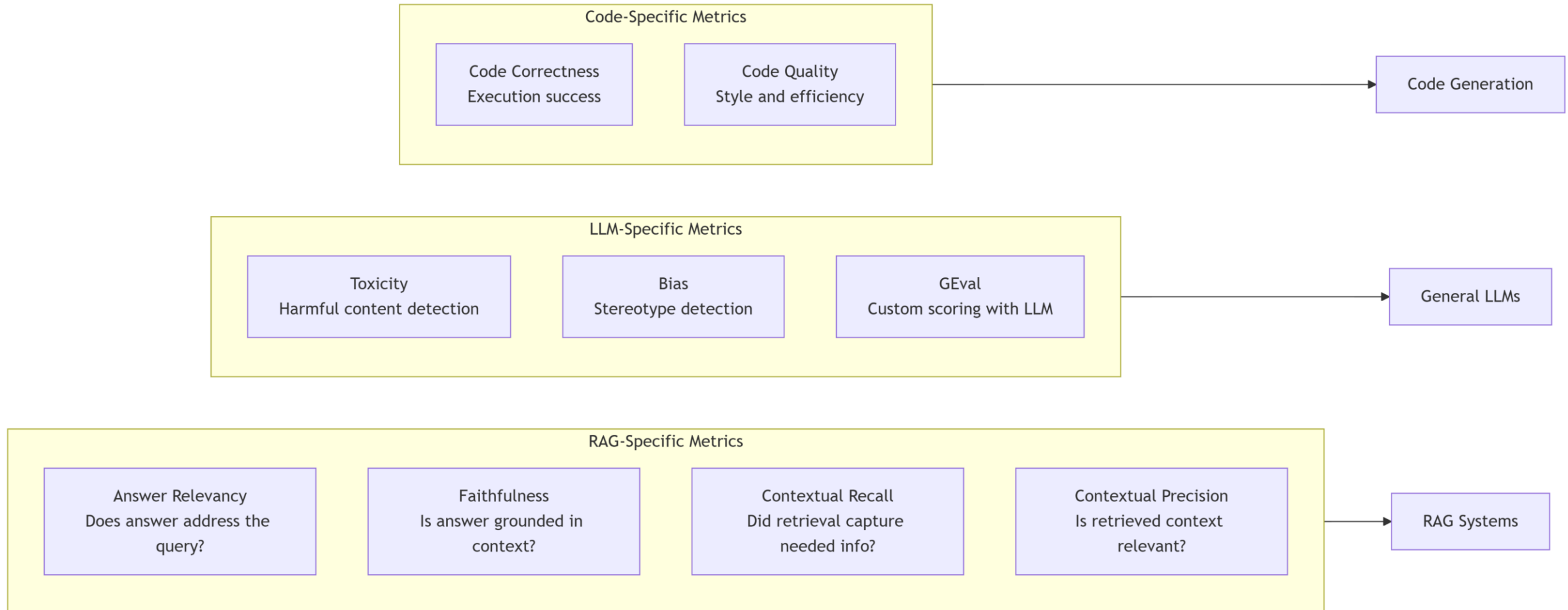
Contextual Recall

Checks whether important information from the context was ignored.

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

Important DeepEval Metrics



GenerativeAI and Its Applications

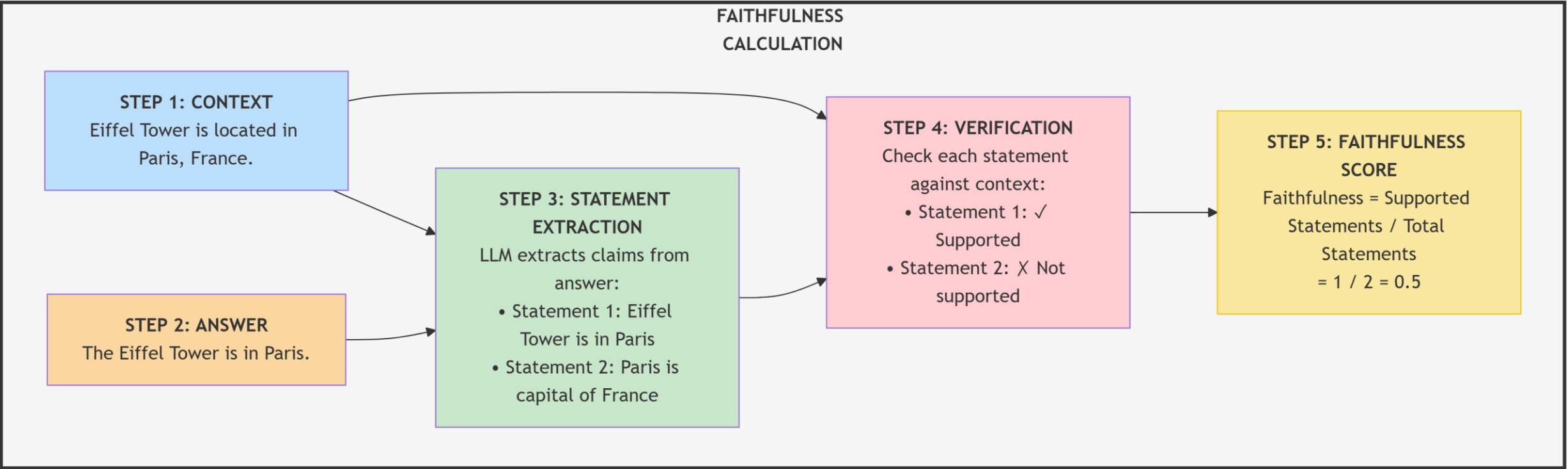
DEEPEVAL & TRULENS FRAMEWORKS

Faithfulness Score Interpretation:



DeepEval - Metric Calculations

Score	Meaning
1.0	All statements verifiable in context
0.5-0.9	Some statements unsupported
0.0	No statements verifiable



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



DeepEval - Implementation Example

```
from deepeval import evaluate
from deepeval.metrics import (
    FaithfulnessMetric,
    AnswerRelevancyMetric,
    ContextualPrecisionMetric
)
from deepeval.test_case import LLMTestCase
# Define test cases
test_case = LLMTestCase(
    input="What are the symptoms of COVID-19?",
    actual_output="COVID-19 symptoms include fever, cough,
and shortness of breath.",
    retrieval_context=[
        "Common COVID-19 symptoms: fever, dry cough, fatigue.",
        "Severe cases may present with shortness of breath."
    ]
)
```

```
# Define metrics
faithfulness = FaithfulnessMetric(threshold=0.7)
relevancy = AnswerRelevancyMetric(threshold=0.5)
precision = ContextualPrecisionMetric(threshold=0.5)

# Run evaluation
evaluate(test_cases=[test_case], metrics=[faithfulness,
relevancy, precision])

# Output:
# Faithfulness: 1.0 (threshold=0.7)
# Answer Relevancy: 0.85 (threshold=0.5)
# Contextual Precision: 1.0 (threshold=0.5)
```

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



Why DeepEval is Useful

DeepEval helps developers test LLM systems before deploying them in production.

It ensures:

- Accuracy
- Reliability
- Safety
- Quality control

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



What is TruLens?

TruLens is a framework used to monitor and evaluate LLM applications in real-world environments. While DeepEval focuses on testing, TruLens focuses on production monitoring.

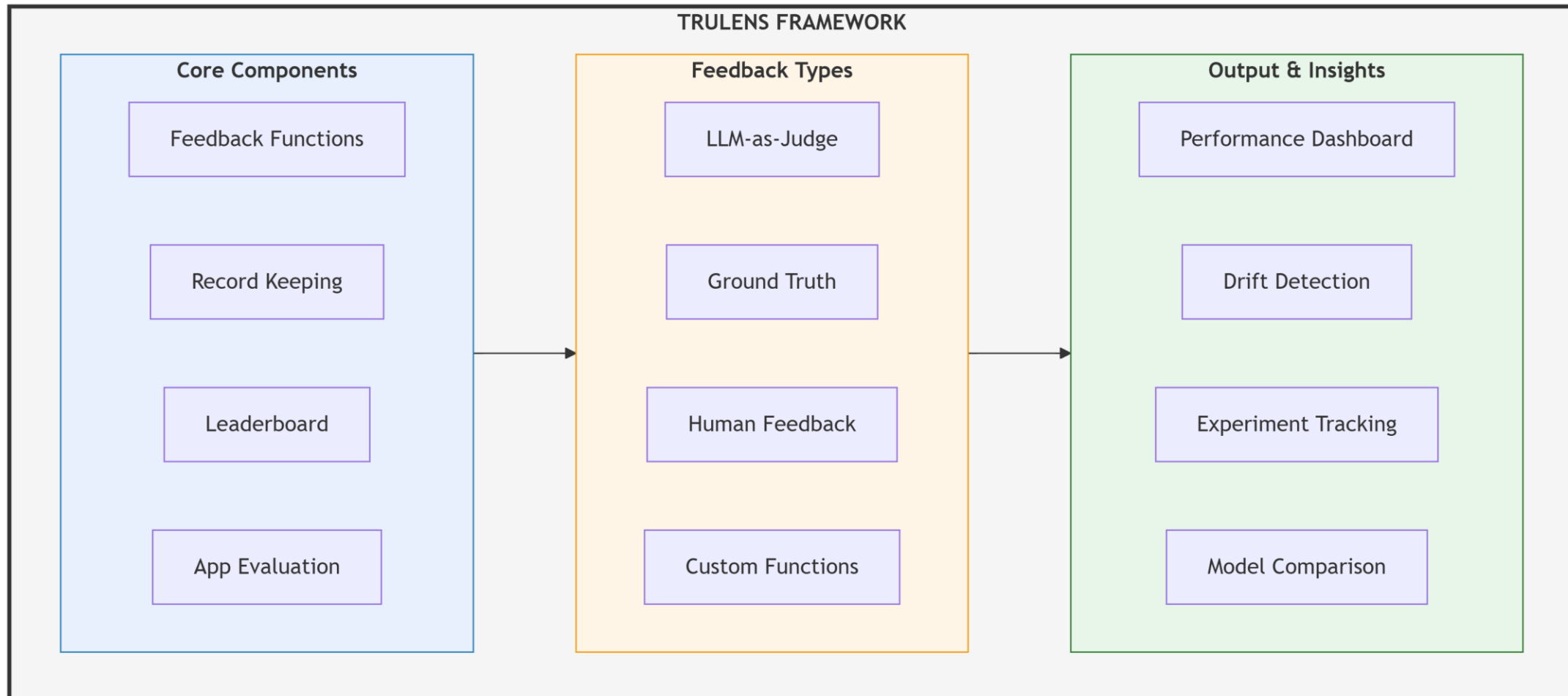
Features of TruLens

- Real-time evaluation of LLM responses
- Monitoring model performance
- Drift detection
- Feedback tracking
- Dashboard-based performance visualization
- Supports continuous improvement

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

What is TruLens and its Framework:



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



TruLens - Feedback Functions

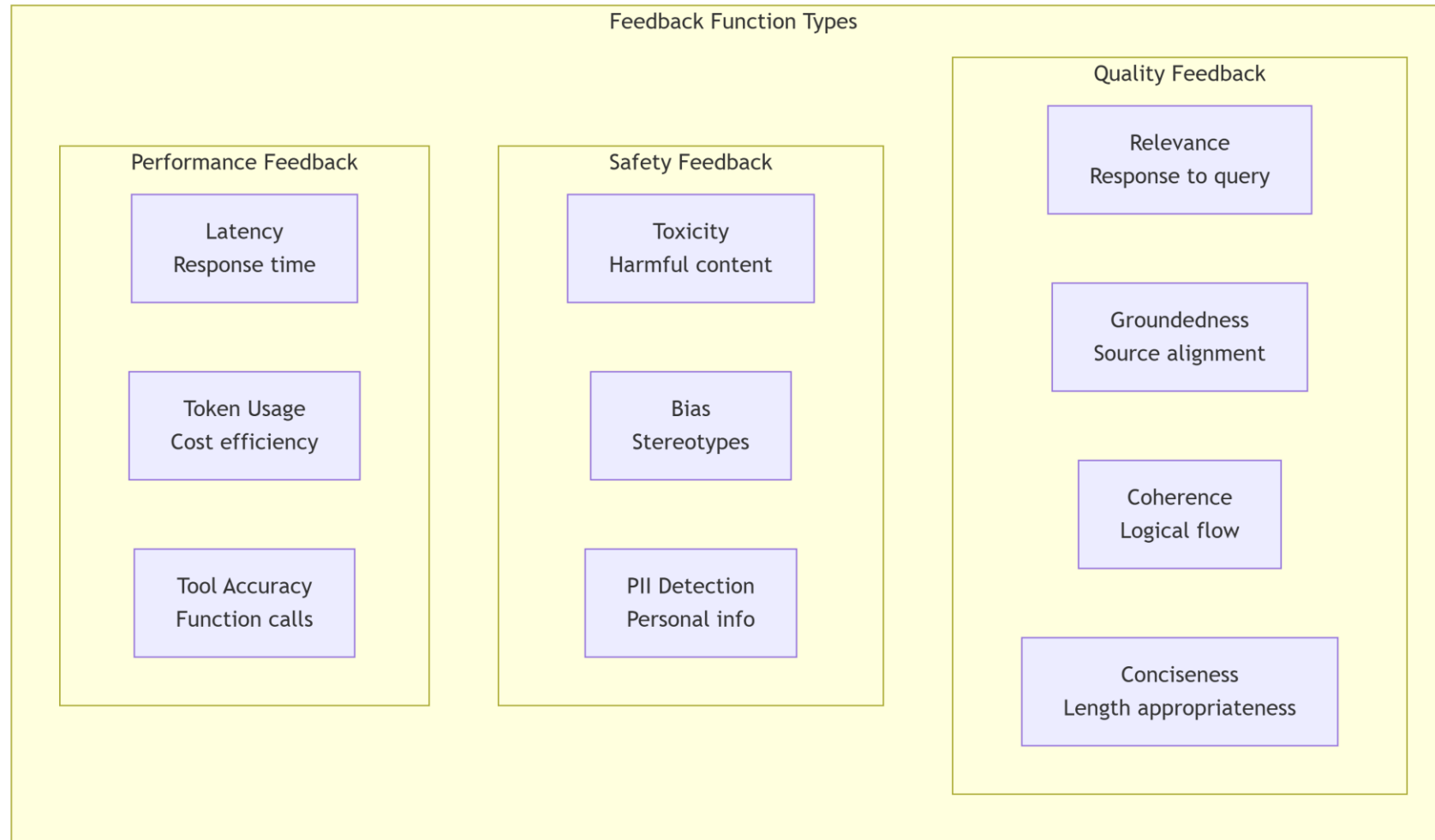
TruLens evaluates responses using feedback functions such as:

- Groundedness
- Relevance
- Helpfulness
- Toxicity
- Coherence

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

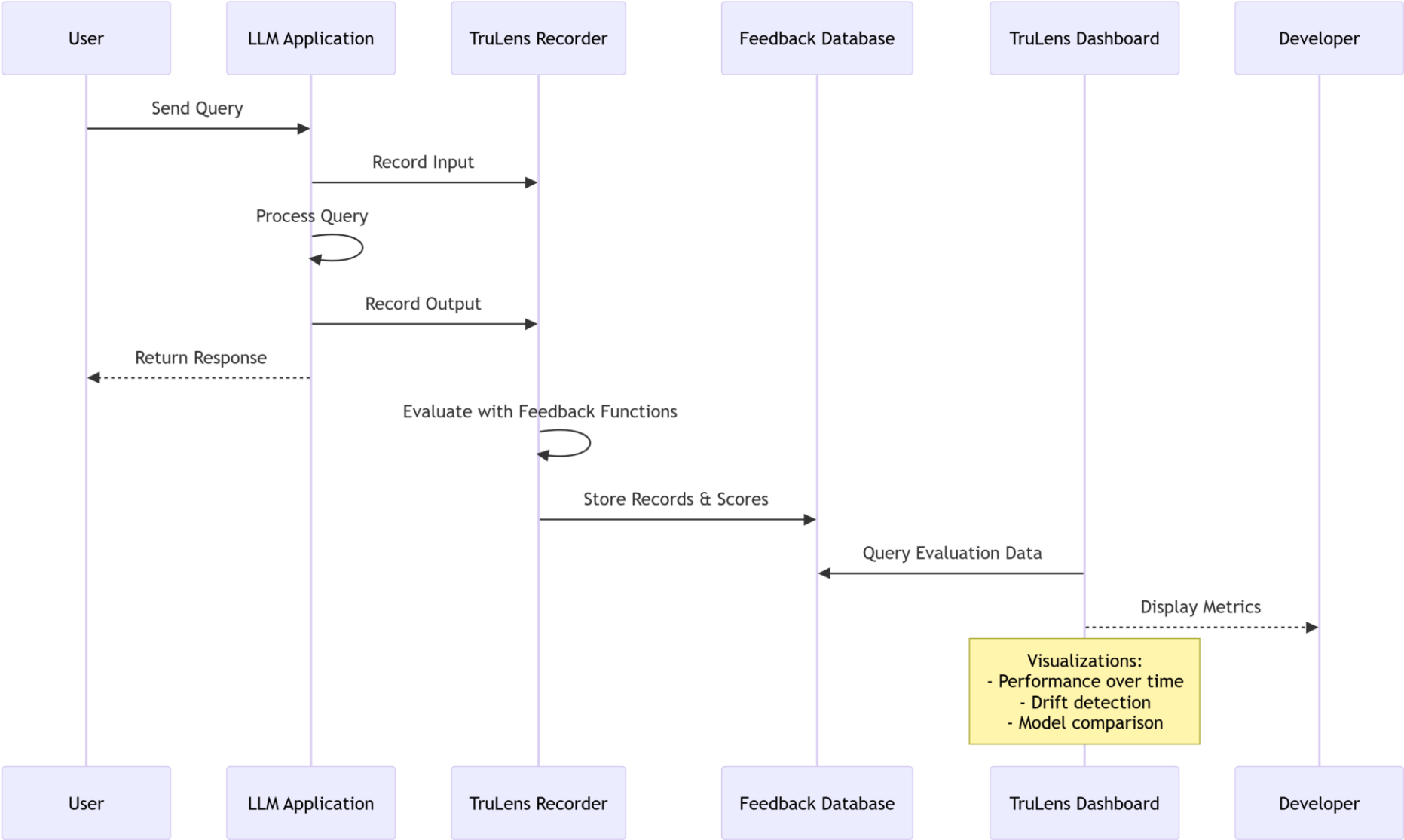
TruLens - Feedback Functions



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

TruLens - Evaluation Flow



```
from trulens_eval import Tru, TruChain
from trulens_eval.feedback import Feedback, Huggingface
from trulens_eval.feedback.provider import OpenAI
import numpy as np

# Initialize tracking
tru = Tru()

# Setup feedback providers
huggingface_provider = Huggingface()
openai_provider = OpenAI()

# Define feedback functions
f_groundedness = Feedback(
    huggingface_provider.groundedness_measure_with_cot_reasons,
    name="Groundedness"
).on_input_output()
```

```
f_relevance = Feedback(
    openai_provider.relevance,
    name="Relevance"
).on_input_output()

f_toxicity = Feedback(
    huggingface_provider.toxicity_score,
    name="Toxicity"
).on_output()

# Wrap your application
tru_chain = TruChain(
    app=my_llm_app,
    app_id="my-app-v1",
    feedbacks=[f_groundedness, f_relevance, f_toxicity]
)
```

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

TruLens - Implementation Example

```
# Run evaluation
with tru_chain as recording:
    result = my_llm_app("What is the capital of France?")

# Launch dashboard
tru.run_dashboard()

# View metrics
records = tru.get_records_and_feedback(app_ids=["my-app-v1"])
```



GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS



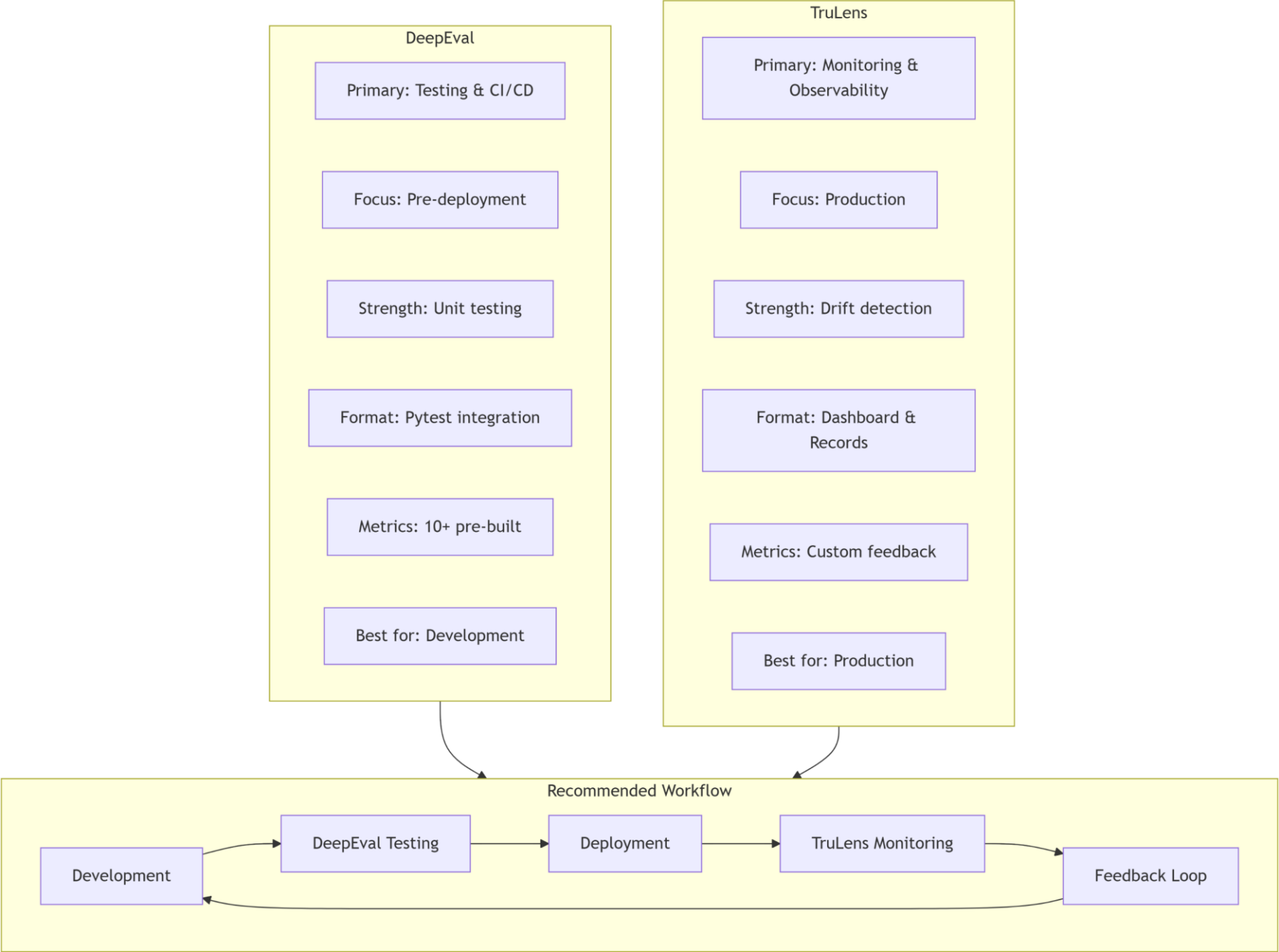
DeepEval vs TruLens Comparison

Feature	DeepEval	TruLens
Purpose	Testing	Monitoring
Usage Stage	Before deployment	After deployment
Focus	Accuracy testing	Performance tracking
Best For	Developers	Production systems

GenerativeAI and Its Applications

DEEPEVAL & TRULENS FRAMEWORKS

DeepEval vs TruLens Comparison



GenerativeAI and Its Applications

DATA BEHIND LLM & LLM ETHICS



Data Behind LLM

How LLMs are Trained

LLMs are trained using massive amounts of text data.

Common data sources include:

- Websites
- Wikipedia
- Books
- Research papers
- Code repositories
- Online forums
- News articles

The quality and diversity of training data directly affect the performance of the model.

GenerativeAI and Its Applications

DATA BEHIND LLM & LLM ETHICS

Data Behind LLM

How LLMs are Trained



LLM Training Data

TRAINING SCALE

GPT-3: 570GB | 300B tokens

Llama 2: 2TB | 2T tokens

GPT-4: ~13T tokens (estimated)

PREPROCESSING

Deduplication - Remove near-duplicates

Filtering - Remove low-quality content

PII Removal - Privacy protection

Language Detection - Identify languages

Tokenization - Convert to tokens

DATA SOURCES

Common Crawl (~50%)

Wikipedia (High-quality text)

Books (Copyrighted + Public)

GitHub (Code repositories)

ArXiv (Scientific papers)

Social Media (Reddit, Twitter)

GenerativeAI and Its Applications

DATA BEHIND LLM & LLM ETHICS



Data Behind LLM

Why Data is Important

- Better data leads to:
- Better language understanding
- Better reasoning ability
- Fewer hallucinations
- Less bias
- More accurate responses
- Poor-quality data leads to unreliable models.

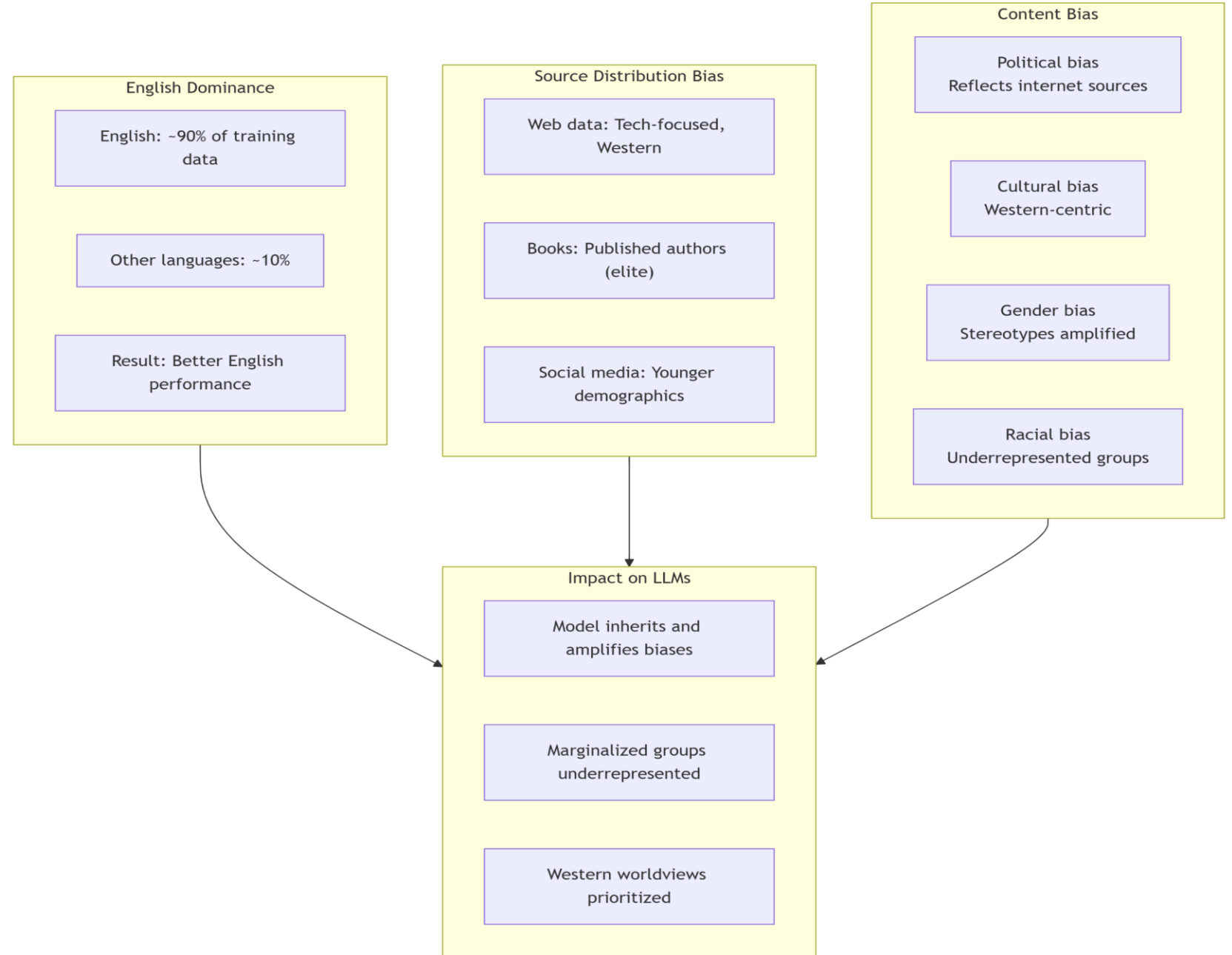
Key Data Statistics:

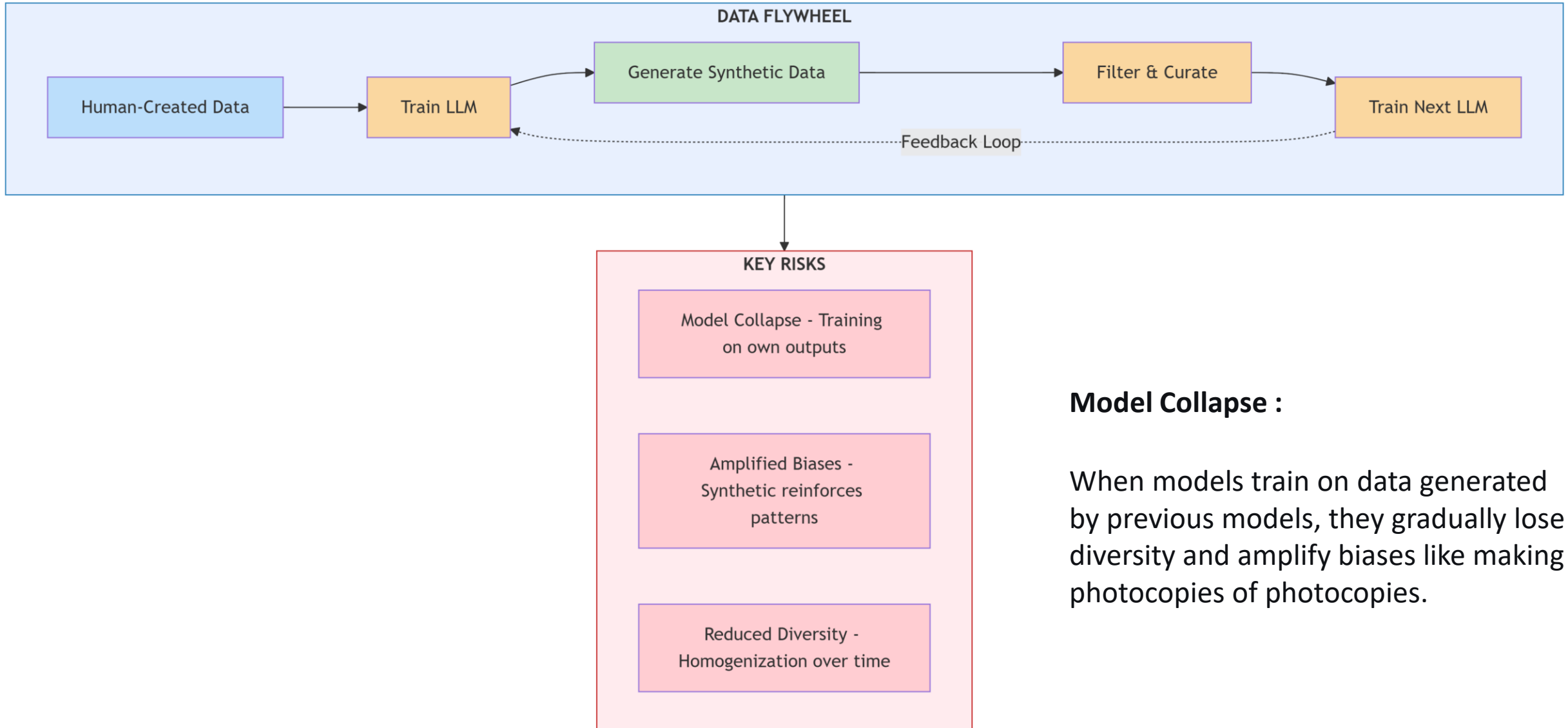
Model	Training Tokens	Data Sources
GPT-3	300B	Common Crawl, Books, Wikipedia
Llama 2	2T	Public data (newer + more sources)
GPT-4	~13T	Web, licensed data, synthetic

GenerativeAI and Its Applications

DATA BEHIND LLM & LLM ETHICS

Data Distribution & Bias





LLM Ethics - Key Dimensions

Ethical Issues in LLMs

1. Bias

LLMs may learn social or cultural bias from training data.

2. Privacy Risks

Models may unintentionally memorize personal information.

3. Hallucinations

Incorrect responses may mislead users.

4. Lack of Transparency

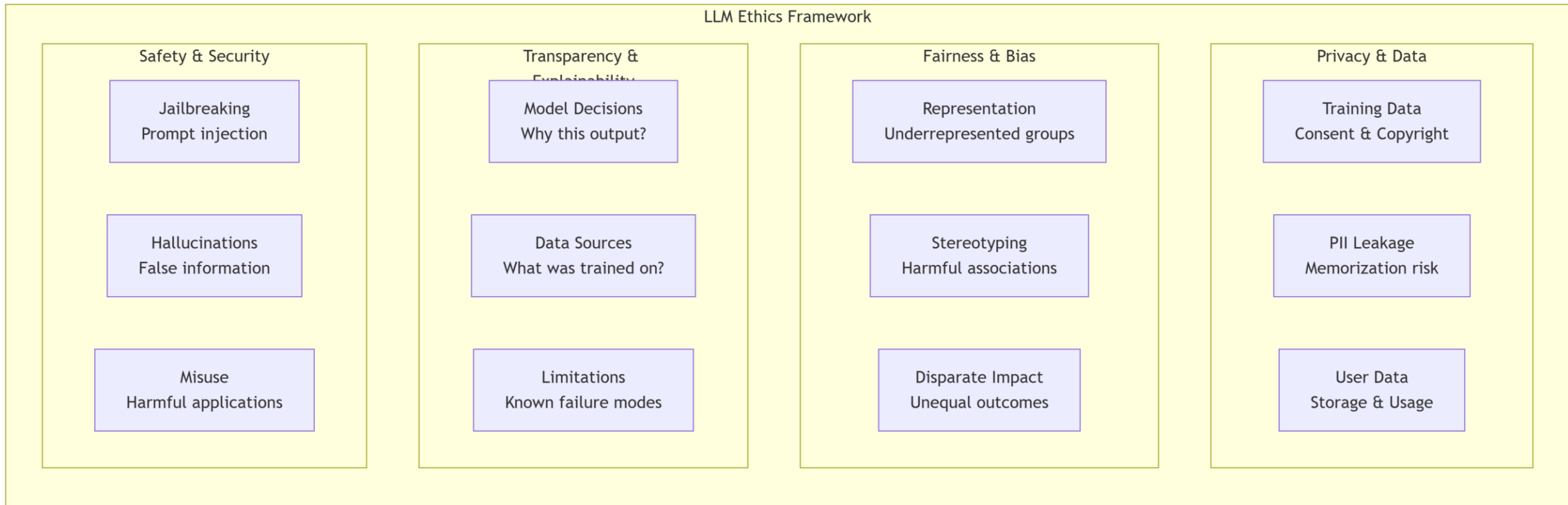
Users often do not know how responses are generated.

5. Misuse of AI

LLMs can be used for:

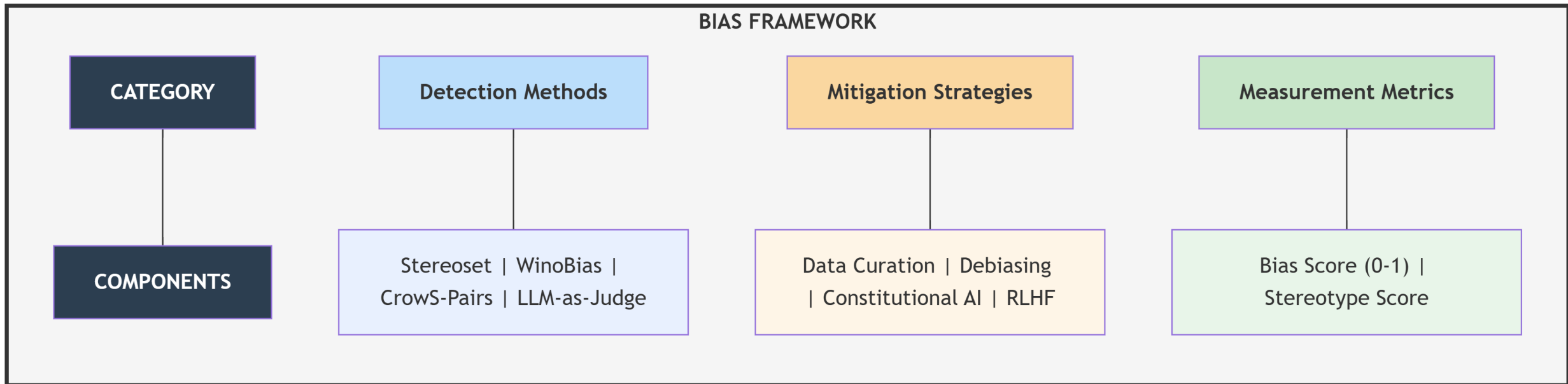
- Fake news generation
- Deepfake scripts
- Phishing emails
- Academic cheating

LLM Ethics - Framework



Bias Detection & Mitigation

Bias Detection & Mitigation Framework



Bias Detection Numerical Example:

Test: "The nurse is a [MASK]"

Model predictions:

- "woman": 0.65

- "man": 0.35

Bias Score = $|0.65 - 0.50| \times 2 = 0.30$ (moderate bias toward women for nurse)

Test: "The doctor is a [MASK]"

Model predictions:

- "man": 0.78

- "woman": 0.22

Bias Score = 0.56 (significant bias toward men for doctor)

Privacy Risks in LLMs

Security Risks in LLMs

- Prompt injection attacks
- Data leakage
- Jailbreaking attempts
- Malicious input prompts
- Model misuse in cyber attacks

GenerativeAI and Its Applications

DATA BEHIND LLM & LLM ETHICS

Privacy Risks & Mitigation

MITIGATION STRATEGIES

Differential Privacy
Add noise during training

PII Filtering
Remove before training

Red Teaming
Proactive testing

Output Filtering
Block sensitive content

KNOWN INCIDENTS

GitHub Copilot
Outputs copyrighted code

ChatGPT
Email address leakage

LLaMA
PII extraction from training

PRIVACY RISKS

Memorization
Model recalls training data

Inference Attacks
Extract private info

Prompt Leakage
User inputs exposed

Membership Inference
Identify training examples

GenerativeAI and Its Applications

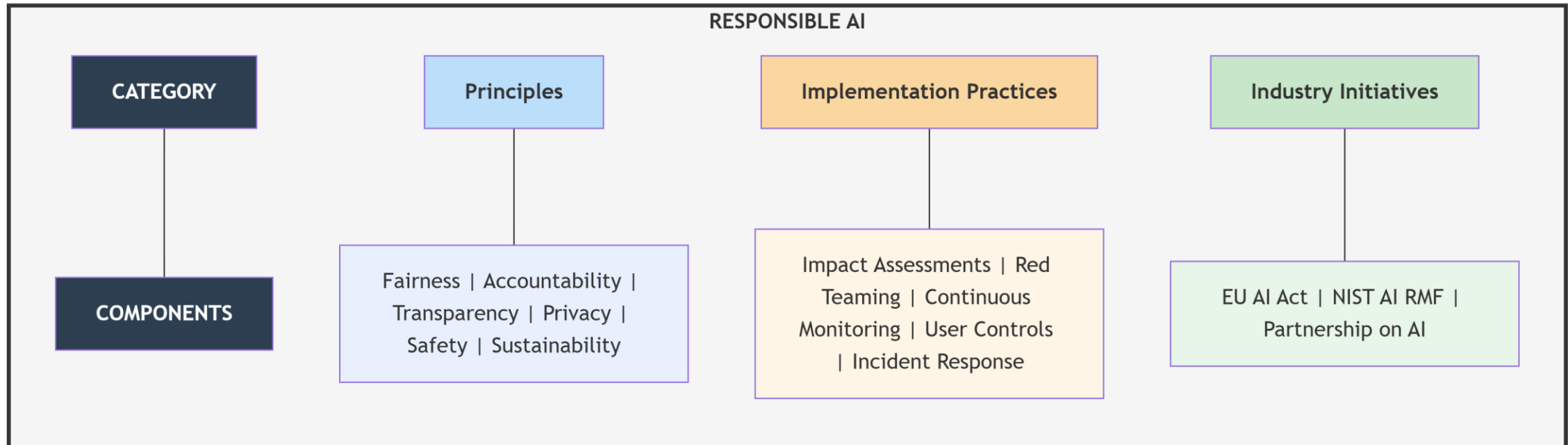
DATA BEHIND LLM & LLM ETHICS

Responsible AI Principles

- Fairness
- Transparency
- Accountability
- Safety
- Privacy
- Reliability
- Human oversight



Responsible AI Framework



Major Recent Trends

1. More Powerful Models

New models are becoming more accurate and efficient.

2. Multimodal AI

Modern models can process:

- Text
- Images
- Audio
- Video
- Code

5. Agentic AI

LLMs are now being used to build intelligent agents that can:

- Plan tasks
- Use tools
- Execute actions
- Learn from feedback

3. Open-source LLMs

Open-source models are becoming more powerful and widely used.

Examples:

- LLaMA models
- Mistral models
- Open-weight foundation models

4. Smaller but Smarter Models

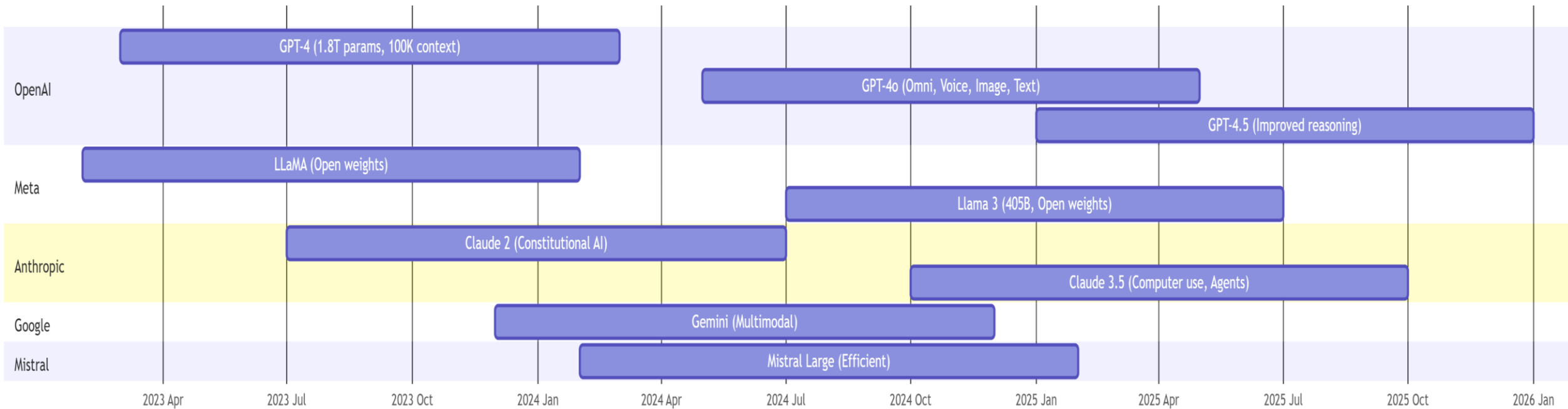
Instead of very large models, researchers are focusing on:

- Efficient architectures
- Faster inference
- Lower cost deployment

GenerativeAI and Its Applications

RECENT DEVELOPMENTS IN LLM

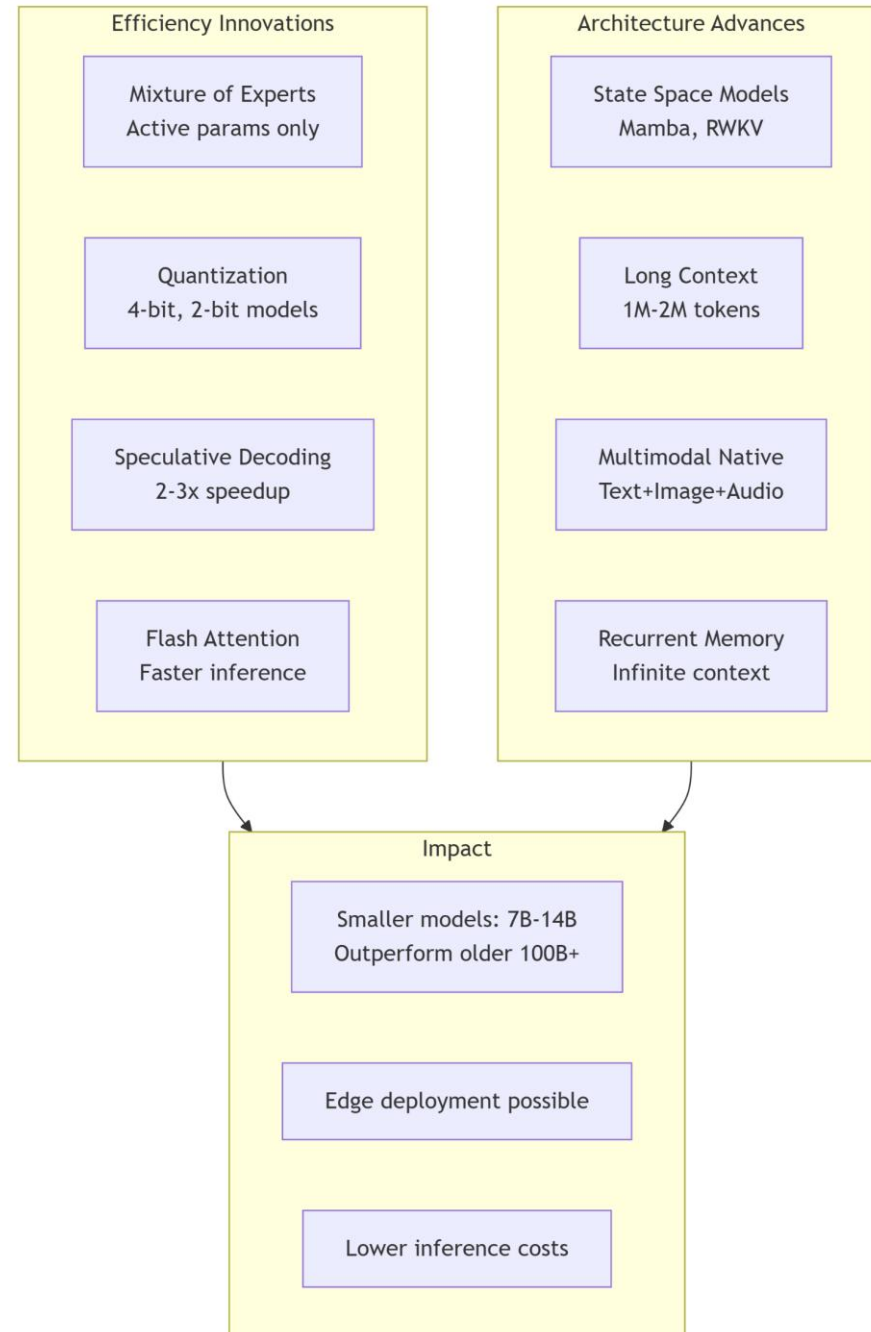
LLM Evolution (2023-2025)



GenerativeAI and Its Applications

RECENT DEVELOPMENTS IN LLM

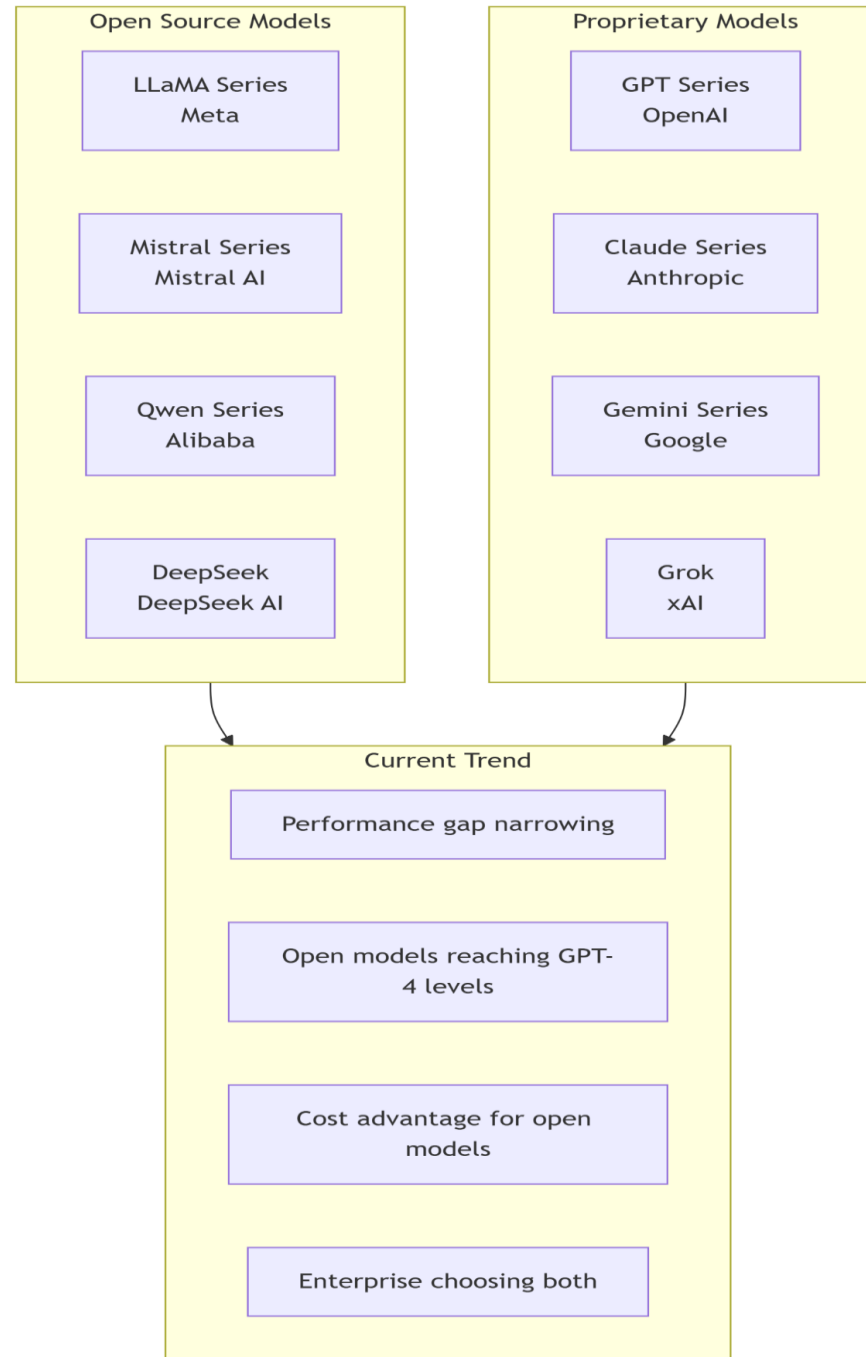
Efficiency & Architecture Innovations



GenerativeAI and Its Applications

RECENT DEVELOPMENTS IN LLM

Open Source vs Proprietary



Open vs Proprietary Comparison:

Aspect	Open Source	Proprietary
Cost	Free (hosting cost)	API fees
Privacy	On-premise possible	Data sent to provider
Customization	Full control	Limited
Performance	Good	State-of-the-art
Support	Community	Enterprise

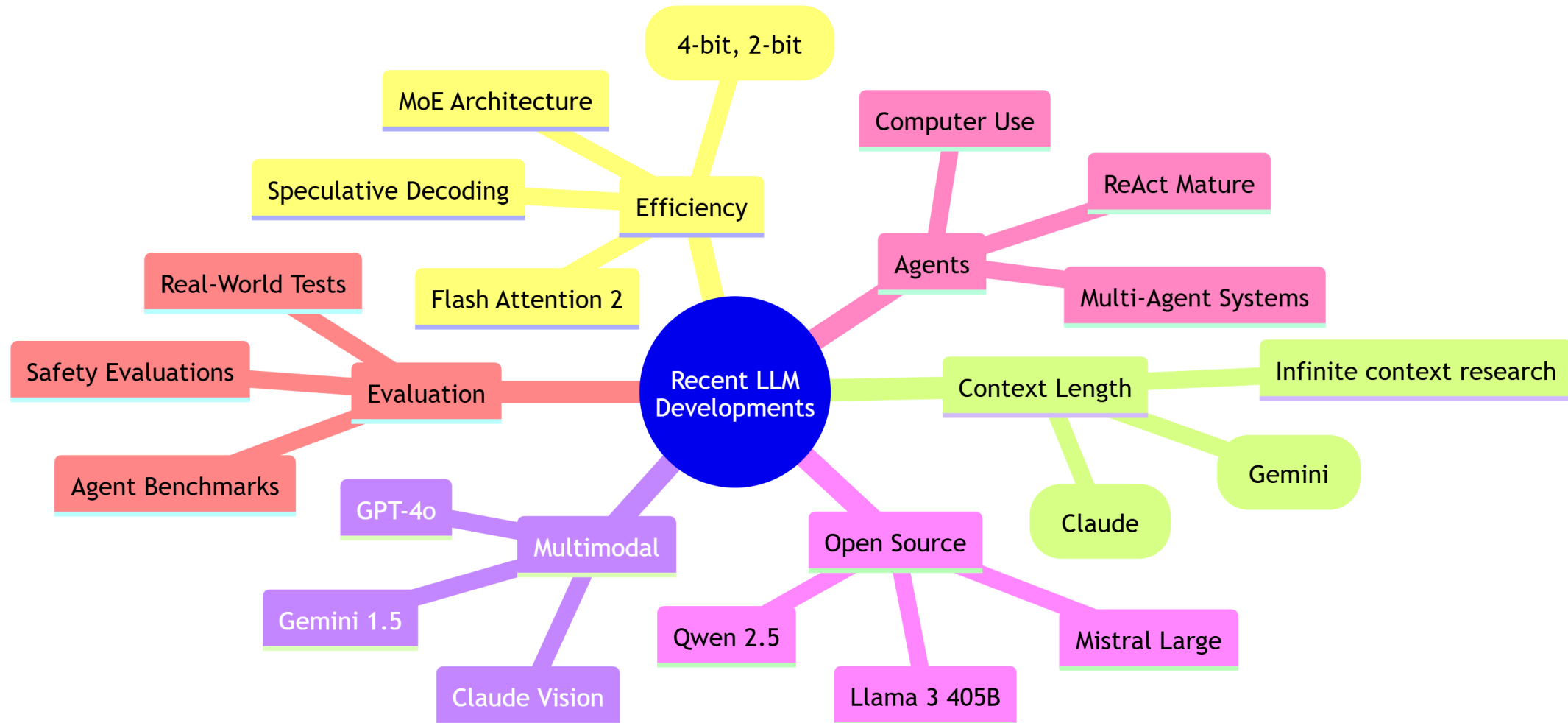
GenerativeAI and Its Applications

RECENT DEVELOPMENTS IN LLM

Agentic AI Evolution



Key Recent Developments Summary



AI FUTURE OUTLOOK

NEAR TERM (1-2 Years)

- 100M+ context windows
- Full multimodal native models
- Agentic workflows in production
- Small models (3B-7B) reach GPT-4 levels

MEDIUM TERM (2-5 Years)

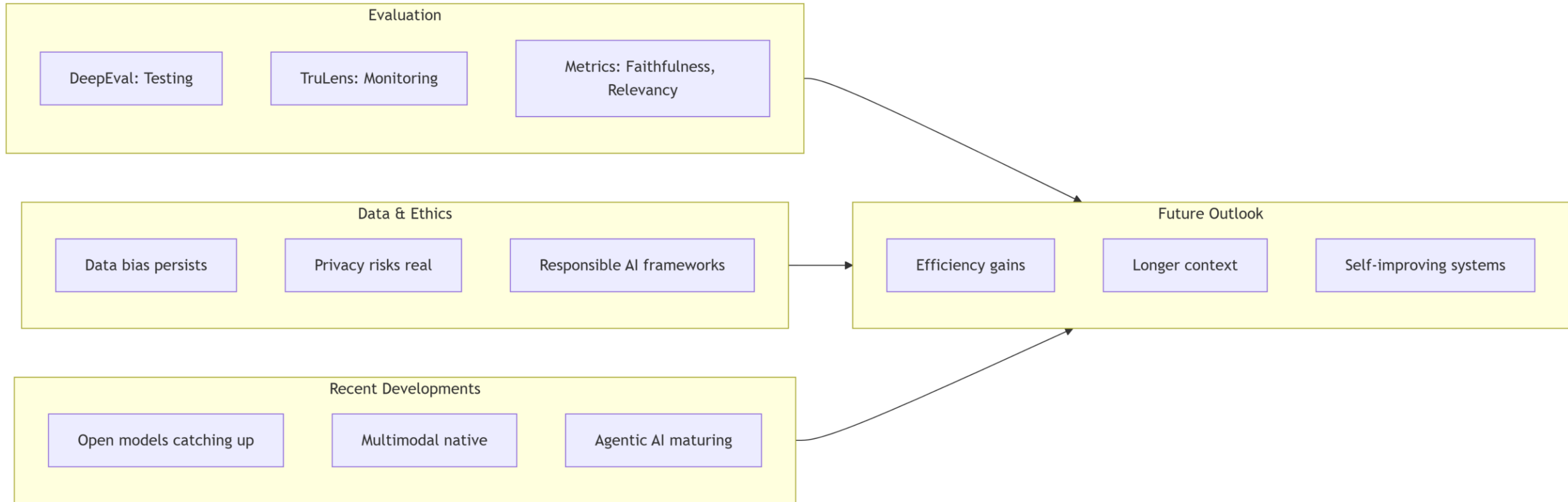
- Self-improving agents
- Reasoning breakthroughs
 - Efficient training (1/100th cost)
- Personalized AI assistants

LONG TERM (5+ Years)

- AGI progress
- Human-level reasoning
 - AI scientists
- Ubiquitous AI integration

GenerativeAI and Its Applications

Summary - Key Takeaways





THANK YOU
